

Introduction to the Theory of
COMPUTATION

THIRD EDITION

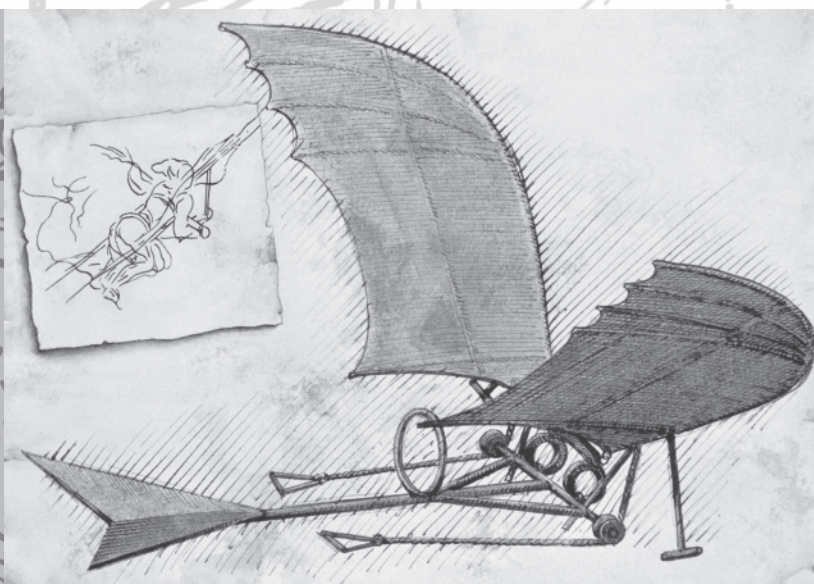


MICHAEL SIPSER

This is an electronic version of the print textbook. Due to electronic rights restrictions, some third party content may be suppressed. Editorial review has deemed that any suppressed content does not materially affect the overall learning experience. The publisher reserves the right to remove content from this title at any time if subsequent rights restrictions require it. For valuable information on pricing, previous editions, changes to current editions, and alternate formats, please visit www.cengage.com/highered to search by ISBN#, author, title, or keyword for materials in your areas of interest.

Introduction to the Theory of COMPUTATION

THIRD EDITION



MICHAEL SIPSER



Australia • Brazil • Japan • Korea • Mexico • Singapore • Spain • United Kingdom • United States

**Introduction to the Theory of
Computation, Third Edition**
Michael Sipser

Editor-in-Chief: Marie Lee

Senior Product Manager:
Alyssa Pratt

Associate Product Manager:
Stephanie Lorenz

Content Project Manager:
Jennifer Feltri-George

Art Director: GEX Publishing Services

Associate Marketing Manager:
Shanna Shelton

Cover Designer: Wing-ip Ngan,
Ink design, inc

Cover Image Credit: @Superstock

© 2013 Cengage Learning

ALL RIGHTS RESERVED. No part of this work covered by the copyright herein may be reproduced, transmitted, stored or used in any form or by any means graphic, electronic, or mechanical, including but not limited to photocopying, recording, scanning, digitizing, taping, Web distribution, information networks, or information storage and retrieval systems, except as permitted under Section 107 or 108 of the 1976 United States Copyright Act, without the prior written permission of the publisher. States Copyright Act, without the prior written permission of the publisher.

For product information and technology assistance, contact us at

**Cengage Learning Customer & Sales Support,
1-800-354-9706**

For permission to use material from this text or product,
submit all requests online at cengage.com/permissions

Further permissions questions can be emailed to
permissionrequest@cengage.com

Library of Congress Control Number: 2012938665

ISBN-13: 978-1-133-18779-0

ISBN-10: 1-133-18779-X

Cengage Learning
20 Channel Center Street
Boston, MA 02210
USA

Cengage Learning is a leading provider of customized learning solutions with office locations around the globe, including Singapore, the United Kingdom, Australia, Mexico, Brazil, and Japan. Locate your local office at: international.cengage.com/region

Cengage Learning products are represented in Canada by Nelson Education, Ltd.

For your lifelong learning solutions, visit www.cengage.com

Cengage Learning reserves the right to revise this publication and make changes from time to time in its content without notice.

The programs in this book are for instructional purposes only. They have been tested with care, but are not guaranteed for any particular intent beyond educational purposes. The author and the publisher do not offer any warranties or representations, nor do they accept any liabilities with respect to the programs.

Printed in the United States of America
1 2 3 4 5 6 7 8 16 15 14 13 12

To Ina, Rachel, and Aaron

CONTENTS

Preface to the First Edition	xi
To the student	xi
To the educator	xii
The first edition	xiii
Feedback to the author	xiii
Acknowledgments	xiv
Preface to the Second Edition	xvii
Preface to the Third Edition	xxi
0 Introduction	1
0.1 Automata, Computability, and Complexity	1
Complexity theory	2
Computability theory	3
Automata theory	3
0.2 Mathematical Notions and Terminology	3
Sets	3
Sequences and tuples	6
Functions and relations	7
Graphs	10
Strings and languages	13
Boolean logic	14
Summary of mathematical terms	16
0.3 Definitions, Theorems, and Proofs	17
Finding proofs	17
0.4 Types of Proof	21
Proof by construction	21
Proof by contradiction	21
Proof by induction	22
<i>Exercises, Problems, and Solutions</i>	25



Part One: Automata and Languages 29

1	Regular Languages	31
1.1	Finite Automata	31
	Formal definition of a finite automaton	35
	Examples of finite automata	37
	Formal definition of computation	40
	Designing finite automata	41
	The regular operations	44
1.2	Nondeterminism	47
	Formal definition of a nondeterministic finite automaton	53
	Equivalence of NFAs and DFAs	54
	Closure under the regular operations	58
1.3	Regular Expressions	63
	Formal definition of a regular expression	64
	Equivalence with finite automata	66
1.4	Nonregular Languages	77
	The pumping lemma for regular languages	77
	<i>Exercises, Problems, and Solutions</i>	82
2	Context-Free Languages	101
2.1	Context-Free Grammars	102
	Formal definition of a context-free grammar	104
	Examples of context-free grammars	105
	Designing context-free grammars	106
	Ambiguity	107
	Chomsky normal form	108
2.2	Pushdown Automata	111
	Formal definition of a pushdown automaton	113
	Examples of pushdown automata	114
	Equivalence with context-free grammars	117
2.3	Non-Context-Free Languages	125
	The pumping lemma for context-free languages	125
2.4	Deterministic Context-Free Languages	130
	Properties of DCFLs	133
	Deterministic context-free grammars	135
	Relationship of DPDAs and DCFGs	146
	Parsing and LR(k) Grammars	151
	<i>Exercises, Problems, and Solutions</i>	154

Part Two: Computability Theory 163

3	The Church–Turing Thesis	165
3.1	Turing Machines	165
	Formal definition of a Turing machine	167

Examples of Turing machines	170
3.2 Variants of Turing Machines	176
Multitape Turing machines	176
Nondeterministic Turing machines	178
Enumerators	180
Equivalence with other models	181
3.3 The Definition of Algorithm	182
Hilbert's problems	182
Terminology for describing Turing machines	184
<i>Exercises, Problems, and Solutions</i>	187
4 Decidability	193
4.1 Decidable Languages	194
Decidable problems concerning regular languages	194
Decidable problems concerning context-free languages	198
4.2 Undecidability	201
The diagonalization method	202
An undecidable language	207
A Turing-unrecognizable language	209
<i>Exercises, Problems, and Solutions</i>	210
5 Reducibility	215
5.1 Undecidable Problems from Language Theory	216
Reductions via computation histories	220
5.2 A Simple Undecidable Problem	227
5.3 Mapping Reducibility	234
Computable functions	234
Formal definition of mapping reducibility	235
<i>Exercises, Problems, and Solutions</i>	239
6 Advanced Topics in Computability Theory	245
6.1 The Recursion Theorem	245
Self-reference	246
Terminology for the recursion theorem	249
Applications	250
6.2 Decidability of logical theories	252
A decidable theory	255
An undecidable theory	257
6.3 Turing Reducibility	260
6.4 A Definition of Information	261
Minimal length descriptions	262
Optimality of the definition	266
Incompressible strings and randomness	267
<i>Exercises, Problems, and Solutions</i>	270

Part Three: Complexity Theory 273

7	Time Complexity	275
7.1	Measuring Complexity	275
	Big- O and small- o notation	276
	Analyzing algorithms	279
	Complexity relationships among models	282
7.2	The Class P	284
	Polynomial time	284
	Examples of problems in P	286
7.3	The Class NP	292
	Examples of problems in NP	295
	The P versus NP question	297
7.4	NP-completeness	299
	Polynomial time reducibility	300
	Definition of NP-completeness	304
	The Cook–Levin Theorem	304
7.5	Additional NP-complete Problems	311
	The vertex cover problem	312
	The Hamiltonian path problem	314
	The subset sum problem	319
	<i>Exercises, Problems, and Solutions</i>	322
8	Space Complexity	331
8.1	Savitch’s Theorem	333
8.2	The Class PSPACE	336
8.3	PSPACE-completeness	337
	The TQBF problem	338
	Winning strategies for games	341
	Generalized geography	343
8.4	The Classes L and NL	348
8.5	NL-completeness	351
	Searching in graphs	353
8.6	NL equals coNL	354
	<i>Exercises, Problems, and Solutions</i>	356
9	Intractability	363
9.1	Hierarchy Theorems	364
	Exponential space completeness	371
9.2	Relativization	376
	Limits of the diagonalization method	377
9.3	Circuit Complexity	379
	<i>Exercises, Problems, and Solutions</i>	388
10	Advanced Topics in Complexity Theory	393
10.1	Approximation Algorithms	393

10.2 Probabilistic Algorithms	396
The class BPP	396
Primality	399
Read-once branching programs	404
10.3 Alternation	408
Alternating time and space	410
The Polynomial time hierarchy	414
10.4 Interactive Proof Systems	415
Graph nonisomorphism	415
Definition of the model	416
IP = PSPACE	418
10.5 Parallel Computation	427
Uniform Boolean circuits	428
The class NC	430
P-completeness	432
10.6 Cryptography	433
Secret keys	433
Public-key cryptosystems	435
One-way functions	435
Trapdoor functions	437
<i>Exercises, Problems, and Solutions</i>	439

Selected Bibliography	443
------------------------------	------------

Index	448
--------------	------------

PREFACE TO THE FIRST EDITION

TO THE STUDENT

Welcome!

You are about to embark on the study of a fascinating and important subject: the theory of computation. It comprises the fundamental mathematical properties of computer hardware, software, and certain applications thereof. In studying this subject, we seek to determine what can and cannot be computed, how quickly, with how much memory, and on which type of computational model. The subject has obvious connections with engineering practice, and, as in many sciences, it also has purely philosophical aspects.

I know that many of you are looking forward to studying this material but some may not be here out of choice. You may want to obtain a degree in computer science or engineering, and a course in theory is required—God knows why. After all, isn't theory arcane, boring, and worst of all, irrelevant?

To see that theory is neither arcane nor boring, but instead quite understandable and even interesting, read on. Theoretical computer science does have many fascinating big ideas, but it also has many small and sometimes dull details that can be tiresome. Learning any new subject is hard work, but it becomes easier and more enjoyable if the subject is properly presented. My primary objective in writing this book is to expose you to the genuinely exciting aspects of computer theory, without getting bogged down in the drudgery. Of course, the only way to determine whether theory interests you is to try learning it.

Theory is relevant to practice. It provides conceptual tools that practitioners use in computer engineering. Designing a new programming language for a specialized application? What you learned about *grammars* in this course comes in handy. Dealing with string searching and pattern matching? Remember *finite automata* and *regular expressions*. Confronted with a problem that seems to require more computer time than you can afford? Think back to what you learned about *NP-completeness*. Various application areas, such as modern cryptographic protocols, rely on theoretical principles that you will learn here.

Theory also is relevant to you because it shows you a new, simpler, and more elegant side of computers, which we normally consider to be complicated machines. The best computer designs and applications are conceived with elegance in mind. A theoretical course can heighten your aesthetic sense and help you build more beautiful systems.

Finally, theory is good for you because studying it expands your mind. Computer technology changes quickly. Specific technical knowledge, though useful today, becomes outdated in just a few years. Consider instead the abilities to think, to express yourself clearly and precisely, to solve problems, and to know when you haven't solved a problem. These abilities have lasting value. Studying theory trains you in these areas.

Practical considerations aside, nearly everyone working with computers is curious about these amazing creations, their capabilities, and their limitations. A whole new branch of mathematics has grown up in the past 30 years to answer certain basic questions. Here's a big one that remains unsolved: If I give you a large number—say, with 500 digits—can you find its factors (the numbers that divide it evenly) in a reasonable amount of time? Even using a supercomputer, no one presently knows how to do that in all cases *within the lifetime of the universe!* The factoring problem is connected to certain secret codes in modern cryptosystems. Find a fast way to factor, and fame is yours!

TO THE EDUCATOR

This book is intended as an upper-level undergraduate or introductory graduate text in computer science theory. It contains a mathematical treatment of the subject, designed around theorems and proofs. I have made some effort to accommodate students with little prior experience in proving theorems, though more experienced students will have an easier time.

My primary goal in presenting the material has been to make it clear and interesting. In so doing, I have emphasized intuition and “the big picture” in the subject over some lower level details.

For example, even though I present the method of proof by induction in Chapter 0 along with other mathematical preliminaries, it doesn't play an important role subsequently. Generally, I do not present the usual induction proofs of the correctness of various constructions concerning automata. If presented clearly, these constructions convince and do not need further argument. An induction may confuse rather than enlighten because induction itself is a rather sophisticated technique that many find mysterious. Belaboring the obvious with

an induction risks teaching students that a mathematical proof is a formal manipulation instead of teaching them what is and what is not a cogent argument.

A second example occurs in Parts Two and Three, where I describe algorithms in prose instead of pseudocode. I don't spend much time programming Turing machines (or any other formal model). Students today come with a programming background and find the Church–Turing thesis to be self-evident. Hence I don't present lengthy simulations of one model by another to establish their equivalence.

Besides giving extra intuition and suppressing some details, I give what might be called a classical presentation of the subject material. Most theorists will find the choice of material, terminology, and order of presentation consistent with that of other widely used textbooks. I have introduced original terminology in only a few places, when I found the standard terminology particularly obscure or confusing. For example, I introduce the term *mapping reducibility* instead of *many-one reducibility*.

Practice through solving problems is essential to learning any mathematical subject. In this book, the problems are organized into two main categories called *Exercises* and *Problems*. The Exercises review definitions and concepts. The Problems require some ingenuity. Problems marked with a star are more difficult. I have tried to make the Exercises and Problems interesting challenges.

THE FIRST EDITION

Introduction to the Theory of Computation first appeared as a Preliminary Edition in paperback. The first edition differs from the Preliminary Edition in several substantial ways. The final three chapters are new: Chapter 8 on space complexity; Chapter 9 on provable intractability; and Chapter 10 on advanced topics in complexity theory. Chapter 6 was expanded to include several advanced topics in computability theory. Other chapters were improved through the inclusion of additional examples and exercises.

Comments from instructors and students who used the Preliminary Edition were helpful in polishing Chapters 0–7. Of course, the errors they reported have been corrected in this edition.

Chapters 6 and 10 give a survey of several more advanced topics in computability and complexity theories. They are not intended to comprise a cohesive unit in the way that the remaining chapters are. These chapters are included to allow the instructor to select optional topics that may be of interest to the serious student. The topics themselves range widely. Some, such as *Turing reducibility* and *alternation*, are direct extensions of other concepts in the book. Others, such as *decidable logical theories* and *cryptography*, are brief introductions to large fields.

FEEDBACK TO THE AUTHOR

The internet provides new opportunities for interaction between authors and readers. I have received much e-mail offering suggestions, praise, and criticism, and reporting errors for the Preliminary Edition. Please continue to correspond!

I try to respond to each message personally, as time permits. The e-mail address for correspondence related to this book is

sipserbook@math.mit.edu.

A web site that contains a list of errata is maintained. Other material may be added to that site to assist instructors and students. Let me know what you would like to see there. The location for that site is

<http://math.mit.edu/~sipser/book.html>.

ACKNOWLEDGMENTS

I could not have written this book without the help of many friends, colleagues, and my family.

I wish to thank the teachers who helped shape my scientific viewpoint and educational style. Five of them stand out. My thesis advisor, Manuel Blum, is due a special note for his unique way of inspiring students through clarity of thought, enthusiasm, and caring. He is a model for me and for many others. I am grateful to Richard Karp for introducing me to complexity theory, to John Addison for teaching me logic and assigning those wonderful homework sets, to Juris Hartmanis for introducing me to the theory of computation, and to my father for introducing me to mathematics, computers, and the art of teaching.

This book grew out of notes from a course that I have taught at MIT for the past 15 years. Students in my classes took these notes from my lectures. I hope they will forgive me for not listing them all. My teaching assistants over the years—Avrim Blum, Thang Bui, Benny Chor, Andrew Chou, Stavros Cosmadakis, Aditi Dhagat, Wayne Goddard, Parry Husbands, Dina Kravets, Jakov Kućan, Brian O'Neill, Ioana Popescu, and Alex Russell—helped me to edit and expand these notes and provided some of the homework problems.

Nearly three years ago, Tom Leighton persuaded me to write a textbook on the theory of computation. I had been thinking of doing so for some time, but it took Tom's persuasion to turn theory into practice. I appreciate his generous advice on book writing and on many other things.

I wish to thank Eric Bach, Peter Beebe, Cris Calude, Marek Chrobak, Anna Chefter, Guang-Ien Cheng, Elias Dahlhaus, Michael Fischer, Steve Fisk, Lance Fortnow, Henry J. Friedman, Jack Fu, Seymour Ginsburg, Oded Goldreich, Brian Grossman, David Harel, Micha Hofri, Dung T. Huynh, Neil Jones, H. Chad Lane, Kevin Lin, Michael Loui, Silvio Micali, Tadao Murata, Christos Papadimitriou, Vaughan Pratt, Daniel Rosenband, Brian Scassellati, Ashish Sharma, Nir Shavit, Alexander Shen, Ilya Shlyakhter, Matt Stallmann, Perry Susskind, Y. C. Tay, Joseph Traub, Osamu Watanabe, Peter Widmayer, David Williamson, Derick Wood, and Charles Yang for comments, suggestions, and assistance as the writing progressed.

The following people provided additional comments that have improved this book: Isam M. Abdelhameed, Eric Allender, Shay Artzi, Michelle Atherton, Rolfe Blodgett, Al Briggs, Brian E. Brooks, Jonathan Buss, Jin Yi Cai,

Steve Chapel, David Chow, Michael Ehrlich, Yaakov Eisenberg, Farzan Fallah, Shaun Flisakowski, Hjalmtyr Hafsteinsson, C. R. Hale, Maurice Herlihy, Vegard Holmedahl, Sandy Irani, Kevin Jiang, Rhys Price Jones, James M. Jowdy, David M. Martin Jr., Manrique Mata-Montero, Ryota Matsuura, Thomas Minka, Farooq Mohammed, Tadao Murata, Jason Murray, Hideo Nagahashi, Kazuo Ohta, Constantine Papageorgiou, Joseph Raj, Rick Regan, Rhonda A. Reumann, Michael Rintzler, Arnold L. Rosenberg, Larry Roske, Max Rozenoer, Walter L. Ruzzo, Sanatan Sahgal, Leonard Schulman, Steve Seiden, Joel Seiferas, Ambuj Singh, David J. Stucki, Jayram S. Thathachar, H. Venkateswaran, Tom Whaley, Christopher Van Wyk, Kyle Young, and Kyoung Hwan Yun.

Robert Sloan used an early version of the manuscript for this book in a class that he taught and provided me with invaluable commentary and ideas from his experience with it. Mark Herschberg, Kazuo Ohta, and Latanya Sweeney read over parts of the manuscript and suggested extensive improvements. Shafi Goldwasser helped me with material in Chapter 10.

I received expert technical support from William Baxter at Superscript, who wrote the $\text{\LaTeX}$ macro package implementing the interior design, and from Larry Nolan at the MIT mathematics department, who keeps things running.

It has been a pleasure to work with the folks at PWS Publishing in creating the final product. I mention Michael Sugarman, David Dietz, Elise Kaiser, Monique Calello, Susan Garland and Tanja Brull because I have had the most contact with them, but I know that many others have been involved, too. Thanks to Jerry Moore for the copy editing, to Diane Levy for the cover design, and to Catherine Hawkes for the interior design.

I am grateful to the National Science Foundation for support provided under grant CCR-9503322.

My father, Kenneth Sipser, and sister, Laura Sipser, converted the book diagrams into electronic form. My other sister, Karen Fisch, saved us in various computer emergencies, and my mother, Justine Sipser, helped out with motherly advice. I thank them for contributing under difficult circumstances, including insane deadlines and recalcitrant software.

Finally, my love goes to my wife, Ina, and my daughter, Rachel. Thanks for putting up with all of this.

*Cambridge, Massachusetts
October, 1996*

Michael Sipser

PREFACE TO THE SECOND EDITION

Judging from the email communications that I've received from so many of you, the biggest deficiency of the first edition is that it provides no sample solutions to any of the problems. So here they are. Every chapter now contains a new *Selected Solutions* section that gives answers to a representative cross-section of that chapter's exercises and problems. To make up for the loss of the solved problems as interesting homework challenges, I've also added a variety of new problems. Instructors may request an Instructor's Manual that contains additional solutions by contacting the sales representative for their region designated at www.course.com.

A number of readers would have liked more coverage of certain “standard” topics, particularly the Myhill–Nerode Theorem and Rice's Theorem. I've partially accommodated these readers by developing these topics in the solved problems. I did not include the Myhill–Nerode Theorem in the main body of the text because I believe that this course should provide only an introduction to finite automata and not a deep investigation. In my view, the role of finite automata here is for students to explore a simple formal model of computation as a prelude to more powerful models, and to provide convenient examples for subsequent topics. Of course, some people would prefer a more thorough treatment, while others feel that I ought to omit all references to (or at least dependence on) finite automata. I did not include Rice's Theorem in the main body of the text because, though it can be a useful “tool” for proving undecidability, some students might

use it mechanically without really understanding what is going on. Using reductions instead, for proving undecidability, gives more valuable preparation for the reductions that appear in complexity theory.

I am indebted to my teaching assistants—Ilya Baran, Sergi Elizalde, Rui Fan, Jonathan Feldman, Venkatesan Guruswami, Prahladh Harsha, Christos Kapoutsis, Julia Khodor, Adam Klivans, Kevin Matulef, Ioana Popescu, April Rasala, Sofya Raskhodnikova, and Iuliu Vasilescu—who helped me to craft some of the new problems and solutions. Ching Law, Edmond Kayi Lee, and Zulfikar Ramzan also contributed to the solutions. I thank Victor Shoup for coming up with a simple way to repair the gap in the analysis of the probabilistic primality algorithm that appears in the first edition.

I appreciate the efforts of the people at Course Technology in pushing me and the other parts of this project along, especially Alyssa Pratt and Aimee Poirier. Many thanks to Gerald Eisman, Weizhen Mao, Rupak Majumdar, Chris Umans, and Christopher Wilson for their reviews. I'm indebted to Jerry Moore for his superb job copy editing and to Laura Segel of ByteGraphics (lauras@bytegraphics.com) for her beautiful rendition of the figures.

The volume of email I've received has been more than I expected. Hearing from so many of you from so many places has been absolutely delightful, and I've tried to respond to all eventually—my apologies for those I missed. I've listed here the people who made suggestions that specifically affected this edition, but I thank everyone for their correspondence:

Luca Aceto, Arash Afkanpour, Rostom Aghanian, Eric Allender, Karun Bakshi, Brad Ballinger, Ray Bartkus, Louis Barton, Arnold Beckmann, Mihir Belhare, Kevin Trent Bergeson, Matthew Berman, Rajesh Bhatt, Somenath Biswas, Lenore Blum, Mauro A. Bonatti, Paul Bondin, Nicholas Bone, Ian Bratt, Gene Browder, Doug Burke, Sam Buss, Vladimir Bychkovsky, Bruce Carneal, Soma Chaudhuri, Rong-Jaye Chen, Samir Chopra, Benny Chor, John Clausen, Allison Coates, Anne Condon, Jeffrey Considine, John J. Crashell, Claude Crepeau, Shaun Cutts, Susheel M. Daswani, Geoff Davis, Scott Dexter, Peter Drake, Jeff Edmonds, Yaakov Eisenberg, Kurtcebe Eroglu, Georg Essl, Alexander T. Fader, Farzan Fallah, Faith Fich, Joseph E. Fitzgerald, Perry Fizzano, David Ford, Jeannie Fromer, Kevin Fu, Atsushi Fujioka, Michel Galley, K. Ganesan, Simson Garfinkel, Travis Gebhardt, Peymann Gohari, Ganesh Gopalakrishnan, Steven Greenberg, Larry Griffith, Jerry Grossman, Rudolf de Haan, Michael Halper, Nick Harvey, Mack Hendricks, Laurie Hiyakumoto, Steve Hockema, Michael Hoehle, Shahadat Hossain, Dave Isecke, Ghaith Issa, Raj D. Iyer, Christian Jacobi, Thomas Janzen, Mike D. Jones, Max Kanovitch, Aaron Kaufman, Roger Khazan, Sarfraz Khurshid, Kevin Killourhy, Seungjoo Kim, Victor Kuncak, Kanata Kuroda, Thomas Lasko, Suk Y. Lee, Edward D. Legenski, Li-Wei Lehman, Kong Lei, Zsolt Lengvarszky, Jeffrey Levetin, Baekjun Lim, Karen Livescu, Stephen Louie, TzerHung Low, Wolfgang Maass, Arash Madani, Michael Manapat, Wojciech Marchewka, David M. Martin Jr., Anders Martinson, Lyle McGeoch, Alberto Medina, Kurt Mehlhorn, Nihar Mehta, Albert R. Meyer, Thomas Minka, Mariya Minkova, Daichi Mizuguchi, G. Allen Morris III, Damon Mosk-Aoyama, Xiaolong Mou, Paul Muir, German Muller,

Donald Nelson, Gabriel Nivasch, Mary Obelnicki, Kazuo Ohta, Thomas M. Oleson, Jr., Curtis Oliver, Owen Ozier, Rene Peralta, Alexander Perlis, Holger Petersen, Detlef Plump, Robert Prince, David Pritchard, Bina Reed, Nicholas Riley, Ronald Rivest, Robert Robinson, Christi Rockwell, Phil Rogaway, Max Rozenoer, John Rupf, Teodor Rus, Larry Ruzzo, Brian Sanders, Cem Say, Kim Schioett, Joel Seiferas, Joao Carlos Setubal, Geoff Lee Seyon, Mark Skandera, Bob Sloan, Geoff Smith, Marc L. Smith, Stephen Smith, Alex C. Snoeren, Guy St-Denis, Larry Stockmeyer, Radu Stoleru, David Stucki, Hisham M. Sueyllam, Kenneth Tam, Elizabeth Thompson, Michel Toulouse, Eric Tria, Chittaranjan Tripathy, Dan Trubow, Hiroki Ueda, Giora Unger, Kurt L. Van Etten, Jesir Vargas, Bienvenido Velez-Rivera, Kobus Vos, Alex Vrenios, Sven Waibel, Marc Waldman, Tom Whaley, Anthony Widjaja, Sean Williams, Joseph N. Wilson, Chris Van Wyk, Guangming Xing, Vee Voon Yee, Cheng Yongxi, Neal Young, Timothy Yuen, Kyle Yung, Jinghua Zhang, Lilla Zollei.

I thank Suzanne Balik, Matthew Kane, Kurt L. Van Etten, Nancy Lynch, Gregory Roberts, and Cem Say for pointing out errata in the first printing.

Most of all, I thank my family—Ina, Rachel, and Aaron—for their patience, understanding, and love as I sat for endless hours here in front of my computer screen.

*Cambridge, Massachusetts
December, 2004*

Michael Sipser

PREFACE TO THE THIRD EDITION

The third edition contains an entirely new section on deterministic context-free languages. I chose this topic for several reasons. First of all, it fills an obvious gap in my previous treatment of the theory of automata and languages. The older editions introduced finite automata and Turing machines in deterministic and nondeterministic variants, but covered only the nondeterministic variant of pushdown automata. Adding a discussion of deterministic pushdown automata provides a missing piece of the puzzle.

Second, the theory of deterministic context-free grammars is the basis for $LR(k)$ grammars, an important and nontrivial application of automata theory in programming languages and compiler design. This application brings together several key concepts, including the equivalence of deterministic and nondeterministic finite automata, and the conversions between context-free grammars and pushdown automata, to yield an efficient and beautiful method for parsing. Here we have a concrete interplay between theory and practice.

Last, this topic seems underserved in existing theory textbooks, considering its importance as a genuine application of automata theory. I studied $LR(k)$ grammars years ago but without fully understanding how they work, and without seeing how nicely they fit into the theory of deterministic context-free languages. My goal in writing this section is to give an intuitive yet rigorous introduction to this area for theorists as well as practitioners, and thereby contribute to its broader appreciation. One note of caution, however: Some of the material in this section is rather challenging, so an instructor in a basic first theory course

may prefer to designate it as supplementary reading. Later chapters do not depend on this material.

Many people helped directly or indirectly in developing this edition. I'm indebted to reviewers Christos Kapoutsis and Cem Say who read a draft of the new section and provided valuable feedback. Several individuals at Cengage Learning assisted with the production, notably Alyssa Pratt and Jennifer Feltri-George. Suzanne Huizenga copyedited the text and Laura Segel of ByteGraphics created the new figures and modified some of the older figures.

I wish to thank my teaching assistants at MIT, Victor Chen, Andy Drucker, Michael Forbes, Elena Grigorescu, Brendan Juba, Christos Kapoutsis, Jon Kellner, Swastik Kopparty, Kevin Matulef, Amanda Redlich, Zack Remscrim, Ben Rossman, Shubhangi Saraf, and Oren Weimann. Each of them helped me by discussing new problems and their solutions, and by providing insight into how well our students understood the course content. I've greatly enjoyed working with such talented and enthusiastic young people.

It has been gratifying to receive email from around the globe. Thanks to all for your suggestions, questions, and ideas. Here is a list of those correspondents whose comments affected this edition:

Djihed Afifi, Steve Aldrich, Eirik Bakke, Suzanne Balik, Victor Bandur, Paul Beame, Elazar Birnbaum, Goutam Biswas, Rob Bittner, Marina Blanton, Rodney Bliss, Promita Chakraborty, Lewis Collier, Jonathan Deber, Simon Dexter, Matt Diephouse, Peter Dillinger, Peter Drake, Zhidian Du, Peter Fenger, Margaret Fleck, Atsushi Fujioka, Valerio Genovese, Evangelos Georgiadis, Joshua Grochow, Jerry Grossman, Andreas Guelzow, Hjalmtyr Hafsteinsson, Arthur Hall III, Cihat Imamoglu, Chinawat Isradisaikul, Kayla Jacobs, Flemming Jensen, Barbara Kaiser, Matthew Kane, Christos Kapoutsis, Ali Durlöv Khan, Edwin Sze Lun Khoo, Yongwook Kim, Akash Kumar, Eleazar Leal, Zsolt Lengvarszky, Cheng-Chung Li, Xiangdong Liang, Vladimir Lifschitz, Ryan Lortie, Jonathan Low, Nancy Lynch, Alexis Maciel, Kevin Matulef, Nelson Max, Hans-Rudolf Metz, Mladen Mikša, Sara Miner More, Rajagopal Nagarajan, Marvin Nakayama, Jonas Nyrup, Gregory Roberts, Ryan Romero, Santhosh Samarthyam, Cem Say, Joel Seiferas, John Sieg, Marc Smith, John Steinberger, Nuri Taşdemir, Tamir Tassa, Mark Testa, Jesse Tjang, John Trammell, Hiroki Ueda, Jeroen Vaelen, Kurt L. Van Etten, Guillermo Vázquez, Phanisekhar Botlaguduru Venkata, Benjamin Bing-Yi Wang, Lutz Warnke, David Warren, Thomas Watson, Joseph Wilson, David Wittenberg, Brian Wongchaowart, Kishan Yerubandi, Dai Yi.

Above all, I thank my family—my wife, Ina, and our children, Rachel and Aaron. Time is finite and fleeting. Your love is everything.

*Cambridge, Massachusetts
April, 2012*

Michael Sipser



0.1

separate part of this book. Here, we introduce these parts in reverse order because by starting from the end you can better understand the reason for the beginning.

COMPLEXITY THEORY

Computer problems come in different varieties; some are easy, and some are hard. For example, the sorting problem is an easy one. Say that you need to arrange a list of numbers in ascending order. Even a small computer can sort a million numbers rather quickly. Compare that to a scheduling problem. Say that you must find a schedule of classes for the entire university to satisfy some reasonable constraints, such as that no two classes take place in the same room at the same time. The scheduling problem seems to be much harder than the sorting problem. If you have just a thousand classes, finding the best schedule may require centuries, even with a supercomputer.

What makes some problems computationally hard and others easy?

This is the central question of complexity theory. Remarkably, **we don't know the answer to it**, though it has been intensively researched for over 40 years. Later, we explore this fascinating question and some of its ramifications.

In one important achievement of complexity theory thus far, researchers have discovered an elegant scheme for classifying problems according to their computational difficulty. It is analogous to the periodic table for classifying elements according to their chemical properties. Using this scheme, we can demonstrate a method for giving evidence that certain problems are computationally hard, even if we are unable to prove that they are.

You have several options when you confront a problem that appears to be computationally hard. First, by understanding which aspect of the problem is at the root of the difficulty, you may be able to alter it so that the problem is more easily solvable. Second, you may be able to settle for less than a perfect solution to the problem. In certain cases, finding solutions that only approximate the perfect one is relatively easy. Third, some problems are hard only in the worst case situation, but easy most of the time. Depending on the application, you may be satisfied with a procedure that occasionally is slow but usually runs quickly. Finally, you may consider alternative types of computation, such as randomized computation, that can speed up certain tasks.

One applied area that has been affected directly by complexity theory is the ancient field of **cryptography**. In most fields, an easy computational problem is preferable to a hard one because easy ones are cheaper to solve. **Cryptography is unusual because it specifically requires computational problems that are hard, rather than easy.** Secret codes should be hard to break without the secret key or password. Complexity theory has pointed cryptographers in the direction of computationally hard problems around which they have designed revolutionary new codes.

COMPUTABILITY THEORY

During the first half of the twentieth century, mathematicians such as Kurt Gödel, Alan Turing, and Alonzo Church discovered that certain basic problems cannot be solved by computers. One example of this phenomenon is the problem of determining whether a mathematical statement is **true or false**. This task is the bread and butter of mathematicians. It seems like a natural for solution by computer because it lies strictly within the realm of mathematics. But **no computer algorithm can perform this task**.

Among the consequences of this profound result was the development of ideas concerning theoretical models of computers that eventually would help lead to the construction of actual computers.

The theories of computability and complexity are closely related. In complexity theory, the objective is to classify problems as easy ones and hard ones; whereas in computability theory, the classification of problems is by those that are solvable and those that are not. Computability theory introduces several of the concepts used in complexity theory.

AUTOMATA THEORY

Automata theory deals with the definitions and properties of mathematical models of computation. These models play a role in several applied areas of computer science. One model, called the *finite automaton*, is used in text processing, compilers, and hardware design. Another model, called the *context-free grammar*, is used in programming languages and artificial intelligence.

Automata theory is an excellent place to begin the study of the theory of computation. The theories of computability and complexity require a precise definition of a *computer*. Automata theory allows practice with formal definitions of computation as it introduces concepts relevant to other nontheoretical areas of computer science.

0.2

MATHEMATICAL NOTIONS AND TERMINOLOGY

As in any mathematical subject, we begin with a discussion of the basic mathematical objects, tools, and notation that we expect to use.

SETS

A **set** is a group of objects represented as a unit. Sets may contain any type of object, including numbers, symbols, and even other sets. The objects in a set are called its **elements** or **members**. Sets may be described formally in several ways.

One way is by listing a set's elements inside braces. Thus the set

$$S = \{7, 21, 57\}$$

contains the elements 7, 21, and 57. The symbols $\in$ and $\notin$ denote set membership and nonmembership. We write $7 \in \{7, 21, 57\}$ and $8 \notin \{7, 21, 57\}$. For two sets A and B , we say that A is a **subset** of B , written $A \subseteq B$, if every member of A also is a member of B . We say that A is a **proper subset** of B , written $A \subsetneq B$, if A is a subset of B and not equal to B .

The order of describing a set doesn't matter, nor does repetition of its members. We get the same set S by writing $\{57, 7, 7, 7, 21\}$. If we do want to take the number of occurrences of members into account, we call the group a **multiset** instead of a set. Thus $\{7\}$ and $\{7, 7\}$ are different as multisets but identical as sets. An **infinite set** contains infinitely many elements. We cannot write a list of all the elements of an infinite set, so we sometimes use the " $\dots$ " notation to mean "continue the sequence forever." Thus we write the set of **natural numbers** $\mathcal{N}$ as

$$\{1, 2, 3, \dots\}.$$

The set of **integers** $\mathcal{Z}$ is written as

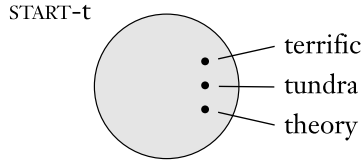
$$\{\dots, -2, -1, 0, 1, 2, \dots\}.$$

The set with zero members is called the **empty set** and is written $\emptyset$. A set with one member is sometimes called a **singleton set**, and a set with two members is called an **unordered pair**.

When we want to describe a set containing elements according to some rule, we write $\{n \mid \text{rule about } n\}$. Thus $\{n \mid n = m^2 \text{ for some } m \in \mathcal{N}\}$ means the set of perfect squares.

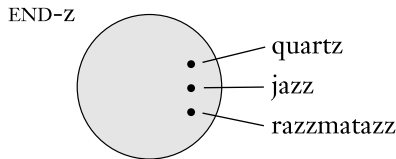
If we have two sets A and B , the **union** of A and B , written $A \cup B$, is the set we get by combining all the elements in A and B into a single set. The **intersection** of A and B , written $A \cap B$, is the set of elements that are in both A and B . The **complement** of A , written $\overline{A}$, is the set of all elements under consideration that are *not* in A .

As is often the case in mathematics, a picture helps clarify a concept. For sets, we use a type of picture called a **Venn diagram**. It represents sets as regions enclosed by circular lines. Let the set START-t be the set of all English words that start with the letter "t". For example, in the figure, the circle represents the set START-t. Several members of this set are represented as points inside the circle.

**FIGURE 0.1**

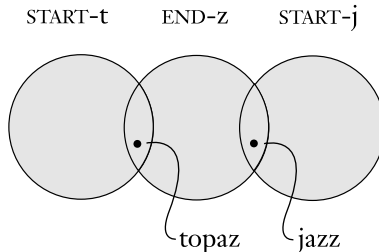
Venn diagram for the set of English words starting with “t”

Similarly, we represent the set END-z of English words that end with “z” in the following figure.

**FIGURE 0.2**

Venn diagram for the set of English words ending with “z”

To represent both sets in the same Venn diagram, we must draw them so that they overlap, indicating that they share some elements, as shown in the following figure. For example, the word *topaz* is in both sets. The figure also contains a circle for the set START-j. It doesn’t overlap the circle for START-t because no word lies in both sets.

**FIGURE 0.3**

Overlapping circles indicate common elements

The next two Venn diagrams depict the **union** and **intersection** of sets A and B .

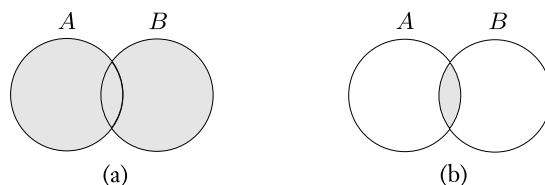


FIGURE 0.4

Diagrams for (a) $A \cup B$ and (b) $A \cap B$

SEQUENCES AND TUPLES

A **sequence** of objects is a list of these objects in some order. We usually designate a sequence by writing the list within parentheses. For example, the sequence 7, 21, 57 would be written

$$(7, 21, 57).$$

The order doesn't matter in a set, but in a sequence it does. Hence $(7, 21, 57)$ is not the same as $(57, 7, 21)$. Similarly, **repetition** does matter in a sequence, but it doesn't matter in a set. Thus $(7, 7, 21, 57)$ is different from both of the other sequences, whereas the set $\{7, 21, 57\}$ is identical to the set $\{7, 7, 21, 57\}$.

As with sets, sequences may be finite or infinite. **Finite sequences** often are called **tuples**. A sequence with k elements is a **k -tuple**. Thus $(7, 21, 57)$ is a 3-tuple. A 2-tuple is also called an **ordered pair**.

Sets and sequences may appear as elements of other sets and sequences. For example, the **power set** of A is the **set of all subsets of A** . If A is the set $\{0, 1\}$, the power set of A is the set $\{\emptyset, \{0\}, \{1\}, \{0, 1\}\}$. The set of all ordered pairs whose elements are 0s and 1s is $\{(0, 0), (0, 1), (1, 0), (1, 1)\}$.

If A and B are two sets, the **Cartesian product** or **cross product** of A and B , written $A \times B$, is the set of all ordered pairs wherein the first element is a member of A and the second element is a member of B .

EXAMPLE 0.5

If $A = \{1, 2\}$ and $B = \{x, y, z\}$,

$$A \times B = \{(1, x), (1, y), (1, z), (2, x), (2, y), (2, z)\}.$$

We can also take the Cartesian product of k sets, $A_1, A_2, \dots, A_k$, written $A_1 \times A_2 \times \dots \times A_k$. It is the set consisting of all k -tuples $(a_1, a_2, \dots, a_k)$ where $a_i \in A_i$.

EXAMPLE 0.6

If A and B are as in Example 0.5,

$$A \times B \times A = \{ (1, x, 1), (1, x, 2), (1, y, 1), (1, y, 2), (1, z, 1), (1, z, 2), \\ (2, x, 1), (2, x, 2), (2, y, 1), (2, y, 2), (2, z, 1), (2, z, 2) \}. \quad \blacksquare$$

If we have the Cartesian product of a set with itself, we use the shorthand

$$\overbrace{A \times A \times \cdots \times A}^k = A^k.$$

EXAMPLE 0.7

The set $\mathcal{N}^2$ equals $\mathcal{N} \times \mathcal{N}$. It consists of all ordered pairs of natural numbers. We also may write it as $\{(i, j) \mid i, j \geq 1\}$. $\blacksquare$

FUNCTIONS AND RELATIONS

Functions are central to mathematics. A **function** is an object that sets up an input–output relationship. A function takes an input and produces an output. In every function, the same input always produces the same output. If f is a function whose output value is b when the input value is a , we write

$$f(a) = b.$$

A function also is called a **mapping**, and, if $f(a) = b$, we say that f maps a to b .

For example, the absolute value function *abs* takes a number x as input and returns x if x is positive and $-x$ if x is negative. Thus $\text{abs}(2) = \text{abs}(-2) = 2$. Addition is another example of a function, written *add*. The input to the addition function is an ordered pair of numbers, and the output is the sum of those numbers.

The set of possible inputs to the function is called its **domain**. The outputs of a function come from a set called its **range**. The notation for saying that f is a function with domain D and range R is

$$f: D \longrightarrow R.$$

In the case of the function *abs*, if we are working with integers, the domain and the range are $\mathcal{Z}$, so we write $\text{abs}: \mathcal{Z} \longrightarrow \mathcal{Z}$. In the case of the addition function for integers, the domain is the set of pairs of integers $\mathcal{Z} \times \mathcal{Z}$ and the range is $\mathcal{Z}$, so we write $\text{add}: \mathcal{Z} \times \mathcal{Z} \longrightarrow \mathcal{Z}$. Note that a function may not necessarily use all the elements of the specified range. The function *abs* never takes on the value -1 even though $-1 \in \mathcal{Z}$. A function that does use all the elements of the range is said to be **onto** the range.

We may describe a specific function in several ways. One way is with a procedure for computing an output from a specified input. Another way is with a table that lists all possible inputs and gives the output for each input.

EXAMPLE 0.8 -----

Consider the function $f: \{0, 1, 2, 3, 4\} \rightarrow \{0, 1, 2, 3, 4\}$.

n	$f(n)$
0	1
1	2
2	3
3	4
4	0

This function adds 1 to its input and then outputs the result modulo 5. A number modulo m is the remainder after division by m . For example, the minute hand on a clock face counts modulo 60. When we do modular arithmetic, we define $\mathbb{Z}_m = \{0, 1, 2, \dots, m - 1\}$. With this notation, the aforementioned function f has the form $f: \mathbb{Z}_5 \rightarrow \mathbb{Z}_5$. ■

EXAMPLE 0.9 -----

Sometimes a two-dimensional table is used if the domain of the function is the Cartesian product of two sets. Here is another function, $g: \mathbb{Z}_4 \times \mathbb{Z}_4 \rightarrow \mathbb{Z}_4$. The entry at the row labeled i and the column labeled j in the table is the value of $g(i, j)$.

g	0	1	2	3
0	0	1	2	3
1	1	2	3	0
2	2	3	0	1
3	3	0	1	2

The function g is the addition function modulo 4. ■

When the domain of a function f is $A_1 \times \dots \times A_k$ for some sets $A_1, \dots, A_k$, the input to f is a k -tuple $(a_1, a_2, \dots, a_k)$ and we call the a_i the *arguments* to f . A function with k arguments is called a ***k-ary function***, and k is called the *arity* of the function. If k is 1, f has a single argument and f is called a ***unary function***. If k is 2, f is a ***binary function***. Certain familiar binary functions are written in a special ***infix notation***, with the symbol for the function placed between its two arguments, rather than in ***prefix notation***, with the symbol preceding. For example, the addition function *add* usually is written in infix notation with the $+$ symbol between its two arguments as in $a + b$ instead of in prefix notation $add(a, b)$.

A *predicate* or *property* is a function whose range is $\{\text{TRUE}, \text{FALSE}\}$. For example, let *even* be a property that is TRUE if its input is an even number and FALSE if its input is an odd number. Thus $\text{even}(4) = \text{TRUE}$ and $\text{even}(5) = \text{FALSE}$.

A property whose domain is a set of k -tuples $A \times \cdots \times A$ is called a *relation*, a *k -ary relation*, or a *k -ary relation on A* . A common case is a 2-ary relation, called a *binary relation*. When writing an expression involving a binary relation, we customarily use infix notation. For example, “less than” is a relation usually written with the infix operation symbol $<$. “Equality”, written with the $=$ symbol, is another familiar relation. If R is a binary relation, the statement aRb means that $aRb = \text{TRUE}$. Similarly, if R is a k -ary relation, the statement $R(a_1, \dots, a_k)$ means that $R(a_1, \dots, a_k) = \text{TRUE}$.

EXAMPLE 0.10

In a children’s game called Scissors–Paper–Stone, the two players simultaneously select a member of the set $\{\text{SCISSORS}, \text{PAPER}, \text{STONE}\}$ and indicate their selections with hand signals. If the two selections are the same, the game starts over. If the selections differ, one player wins, according to the relation *beats*.

<i>beats</i>	SCISSORS	PAPER	STONE
SCISSORS	FALSE	TRUE	FALSE
PAPER	FALSE	FALSE	TRUE
STONE	TRUE	FALSE	FALSE

From this table we determine that SCISSORS *beats* PAPER is TRUE and that PAPER *beats* SCISSORS is FALSE. ■

Sometimes describing predicates with sets instead of functions is more convenient. The predicate $P: D \rightarrow \{\text{TRUE}, \text{FALSE}\}$ may be written (D, S) , where $S = \{a \in D \mid P(a) = \text{TRUE}\}$, or simply S if the domain D is obvious from the context. Hence the relation *beats* may be written

$$\{(\text{SCISSORS}, \text{PAPER}), (\text{PAPER}, \text{STONE}), (\text{STONE}, \text{SCISSORS})\}.$$

A special type of binary relation, called an *equivalence relation*, captures the notion of two objects being equal in some feature. A binary relation R is an equivalence relation if R satisfies three conditions:

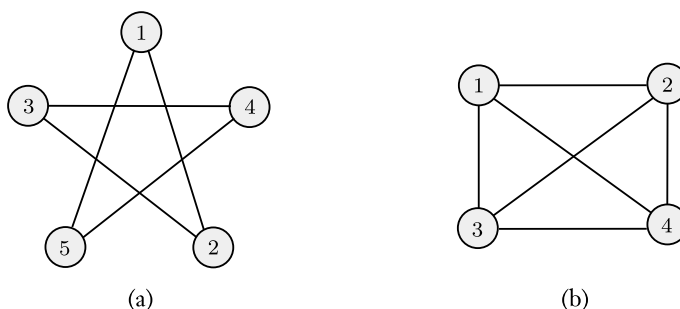
1. R is *reflexive* if for every x , xRx ;
2. R is *symmetric* if for every x and y , xRy implies yRx ; and
3. R is *transitive* if for every x , y , and z , xRy and yRz implies xRz .

EXAMPLE 0.11

Define an equivalence relation on the natural numbers, written $\equiv_7$. For $i, j \in \mathcal{N}$, say that $i \equiv_7 j$, if $i - j$ is a multiple of 7. This is an equivalence relation because it satisfies the three conditions. First, it is reflexive, as $i - i = 0$, which is a multiple of 7. Second, it is symmetric, as $i - j$ is a multiple of 7 if $j - i$ is a multiple of 7. Third, it is transitive, as whenever $i - j$ is a multiple of 7 and $j - k$ is a multiple of 7, then $i - k = (i - j) + (j - k)$ is the sum of two multiples of 7 and hence a multiple of 7, too. ■

GRAPHS

An *undirected graph*, or simply a *graph*, is a set of points with lines connecting some of the points. The points are called **nodes** or **vertices**, and the lines are called **edges**, as shown in the following figure.

**FIGURE 0.12**

Examples of graphs

The number of edges at a particular node is the *degree* of that node. In Figure 0.12(a), all the nodes have degree 2. In Figure 0.12(b), all the nodes have degree 3. No more than one edge is allowed between any two nodes. We may allow an edge from a node to itself, called a *self-loop*, depending on the situation.

In a graph G that contains nodes i and j , the pair (i, j) represents the edge that connects i and j . The order of i and j doesn't matter in an undirected graph, so the pairs (i, j) and (j, i) represent the same edge. Sometimes we describe undirected edges with unordered pairs using set notation as in $\{i, j\}$. If V is the set of nodes of G and E is the set of edges, we say $G = (V, E)$. We can describe a graph with a diagram or more formally by specifying V and E . For example, a formal description of the graph in Figure 0.12(a) is

$$(\{1, 2, 3, 4, 5\}, \{(1, 2), (2, 3), (3, 4), (4, 5), (5, 1)\}),$$

and a formal description of the graph in Figure 0.12(b) is

$$(\{1, 2, 3, 4\}, \{(1, 2), (1, 3), (1, 4), (2, 3), (2, 4), (3, 4)\}).$$

Graphs frequently are used to represent data. Nodes might be cities and edges the connecting highways, or nodes might be people and edges the friendships between them. Sometimes, for convenience, we label the nodes and/or edges of a graph, which then is called a **labeled graph**. Figure 0.13 depicts a graph whose nodes are cities and whose edges are labeled with the dollar cost of the cheapest nonstop airfare for travel between those cities if flying nonstop between them is possible.

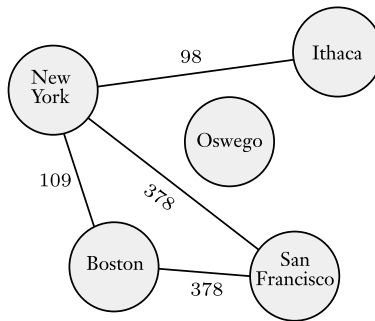


FIGURE 0.13
Cheapest nonstop airfares between various cities

We say that graph G is a **subgraph** of graph H if the nodes of G are a subset of the nodes of H , and the edges of G are the edges of H on the corresponding nodes. The following figure shows a graph H and a subgraph G .

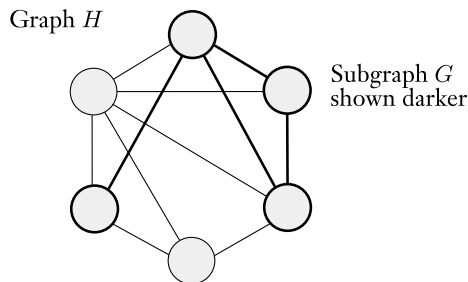


FIGURE 0.14
Graph G (shown darker) is a subgraph of H

A **path** in a graph is a sequence of nodes connected by edges. A **simple path** is a path that doesn't repeat any nodes. A graph is **connected** if every two nodes have a path between them. A path is a **cycle** if it starts and ends in the same node. A **simple cycle** is one that contains at least three nodes and repeats only the first and last nodes. A graph is a **tree** if it is connected and has no simple cycles, as shown in Figure 0.15. A tree may contain a specially designated node called the **root**. The nodes of degree 1 in a tree, other than the root, are called the **leaves** of the tree.

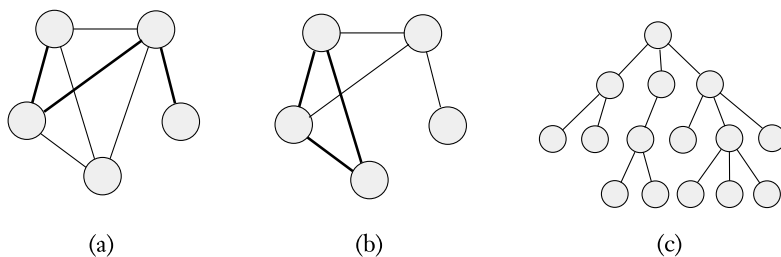


FIGURE 0.15

(a) A path in a graph, (b) a cycle in a graph, and (c) a tree

A **directed graph** has arrows instead of lines, as shown in the following figure. The number of arrows pointing from a particular node is the **outdegree** of that node, and the number of arrows pointing to a particular node is the **indegree**.

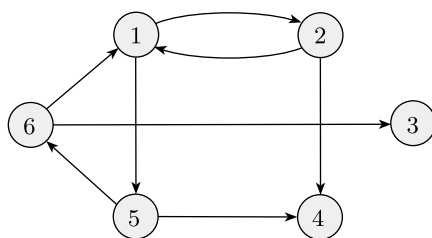


FIGURE 0.16

A directed graph

In a directed graph, we represent an edge from i to j as a pair (i, j) . The formal description of a directed graph G is (V, E) , where V is the set of nodes and E is the set of edges. The formal description of the graph in Figure 0.16 is

$$(\{1, 2, 3, 4, 5, 6\}, \{(1, 2), (1, 5), (2, 1), (2, 4), (5, 4), (5, 6), (6, 1), (6, 3)\}).$$

A path in which all the arrows point in the same direction as its steps is called a **directed path**. A directed graph is **strongly connected** if a directed path connects every two nodes. Directed graphs are a handy way of depicting binary relations. If R is a binary relation whose domain is $D \times D$, a labeled graph $G = (D, E)$ represents R , where $E = \{(x, y) \mid xRy\}$.

EXAMPLE 0.17

The directed graph shown here represents the relation given in Example 0.10.

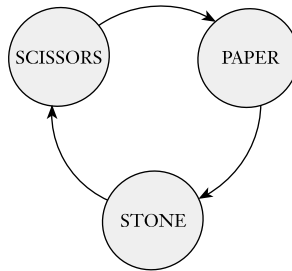


FIGURE 0.18

The graph of the relation *beats*

STRINGS AND LANGUAGES

Strings of characters are fundamental building blocks in computer science. The alphabet over which the strings are defined may vary with the application. For our purposes, we define an **alphabet** to be any **nonempty finite set**. The members of the alphabet are the **symbols** of the alphabet. We generally use capital Greek letters Σ and Γ to designate alphabets and a typewriter font for symbols from an alphabet. The following are a few examples of alphabets.

$$\Sigma_1 = \{0, 1\}$$

$$\Sigma_2 = \{a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z\}$$

$$\Gamma = \{0, 1, x, y, z\}$$

A **string over an alphabet** is a finite sequence of symbols from that alphabet, usually written next to one another and not separated by commas. If $\Sigma_1 = \{0,1\}$, then 01001 is a string over Σ_1 . If $\Sigma_2 = \{a, b, c, \dots, z\}$, then abracadabra is a string over Σ_2 . If w is a string over Σ , the **length** of w , written $|w|$, is the number of symbols that it contains. The string of length zero is called the **empty string** and is written ϵ . The empty string plays the role of 0 in a number system. If w has length n , we can write $w = w_1w_2 \cdots w_n$ where each $w_i \in \Sigma$. The **reverse** of w , written w^R , is the string obtained by writing w in the opposite order (i.e., $w_nw_{n-1} \cdots w_1$). String z is a **substring** of w if z appears consecutively within w . For example, cad is a substring of abracadabra.

If we have string x of length m and string y of length n , the **concatenation** of x and y , written xy , is the string obtained by appending y to the end of x , as in $x_1 \cdots x_my_1 \cdots y_n$. To concatenate a string with itself many times, we use the superscript notation x^k to mean

$$\overbrace{xx \cdots x}^k.$$

The **lexicographic order** of strings is the same as the familiar dictionary order. We'll occasionally use a modified lexicographic order, called **shortlex order** or simply **string order**, that is identical to lexicographic order, except that shorter strings precede longer strings. Thus the string ordering of all strings over the alphabet $\{0,1\}$ is

$$(\epsilon, 0, 1, 00, 01, 10, 11, 000, \dots).$$

Say that string x is a **prefix** of string y if a string z exists where $xz = y$, and that x is a **proper prefix** of y if in addition $x \neq y$. A **language** is a set of strings. A language is **prefix-free** if no member is a proper prefix of another member.

BOOLEAN LOGIC

Boolean logic is a mathematical system built around the two values TRUE and FALSE. Though originally conceived of as pure mathematics, this system is now considered to be the foundation of digital electronics and computer design. The values TRUE and FALSE are called the **Boolean values** and are often represented by the values 1 and 0. We use Boolean values in situations with two possibilities, such as a wire that may have a high or a low voltage, a proposition that may be true or false, or a question that may be answered yes or no.

We can manipulate Boolean values with the **Boolean operations**. The simplest Boolean operation is the **negation** or NOT operation, designated with the symbol $\neg$. The negation of a Boolean value is the opposite value. Thus $\neg 0 = 1$ and $\neg 1 = 0$. We designate the **conjunction** or AND operation with the symbol $\wedge$. The conjunction of two Boolean values is 1 if both of those values are 1. The **disjunction** or OR operation is designated with the symbol $\vee$. The disjunc-

tion of two Boolean values is 1 if either of those values is 1. We summarize this information as follows.

$$\begin{array}{lll}
 0 \wedge 0 = 0 & 0 \vee 0 = 0 & \neg 0 = 1 \\
 0 \wedge 1 = 0 & 0 \vee 1 = 1 & \neg 1 = 0 \\
 1 \wedge 0 = 0 & 1 \vee 0 = 1 & \\
 1 \wedge 1 = 1 & 1 \vee 1 = 1 &
 \end{array}$$

We use Boolean operations for combining simple statements into more complex Boolean expressions, just as we use the arithmetic operations $+$ and $\times$ to construct complex arithmetic expressions. For example, if P is the Boolean value representing the truth of the statement “the sun is shining” and Q represents the truth of the statement “today is Monday”, we may write $P \wedge Q$ to represent the truth value of the statement “the sun is shining *and* today is Monday” and similarly for $P \vee Q$ with *and* replaced by *or*. The values P and Q are called the **operands** of the operation.

Several other Boolean operations occasionally appear. The **exclusive or**, or **XOR**, operation is designated by the $\oplus$ symbol and is 1 if either but not both of its two operands is 1. The **equality** operation, written with the symbol $\leftrightarrow$, is 1 if both of its operands have the same value. Finally, the **implication** operation is designated by the symbol $\rightarrow$ and is 0 if its first operand is 1 and its second operand is 0; otherwise, $\rightarrow$ is 1. We summarize this information as follows.

$$\begin{array}{lll}
 0 \oplus 0 = 0 & 0 \leftrightarrow 0 = 1 & 0 \rightarrow 0 = 1 \\
 0 \oplus 1 = 1 & 0 \leftrightarrow 1 = 0 & 0 \rightarrow 1 = 1 \\
 1 \oplus 0 = 1 & 1 \leftrightarrow 0 = 0 & 1 \rightarrow 0 = 0 \\
 1 \oplus 1 = 0 & 1 \leftrightarrow 1 = 1 & 1 \rightarrow 1 = 1
 \end{array}$$

We can establish various relationships among these operations. In fact, we can express all Boolean operations in terms of the AND and NOT operations, as the following identities show. The two expressions in each row are equivalent. Each row expresses the operation in the left-hand column in terms of operations above it and AND and NOT.

$$\begin{array}{ll}
 P \vee Q & \neg(\neg P \wedge \neg Q) \\
 P \rightarrow Q & \neg P \vee Q \\
 P \leftrightarrow Q & (P \rightarrow Q) \wedge (Q \rightarrow P) \\
 P \oplus Q & \neg(P \leftrightarrow Q)
 \end{array}$$

The **distributive law** for AND and OR comes in handy when we manipulate Boolean expressions. It is similar to the distributive law for addition and multiplication, which states that $a \times (b + c) = (a \times b) + (a \times c)$. The Boolean version comes in two forms:

- $P \wedge (Q \vee R)$ equals $(P \wedge Q) \vee (P \wedge R)$, and its dual
- $P \vee (Q \wedge R)$ equals $(P \vee Q) \wedge (P \vee R)$.

SUMMARY OF MATHEMATICAL TERMS

Alphabet	A finite, nonempty set of objects called symbols
Argument	An input to a function
Binary relation	A relation whose domain is a set of pairs
Boolean operation	An operation on Boolean values
Boolean value	The values TRUE or FALSE, often represented by 1 or 0
Cartesian product	An operation on sets forming a set of all tuples of elements from respective sets
Complement	An operation on a set, forming the set of all elements not present
Concatenation	An operation that joins strings together
Conjunction	Boolean AND operation
Connected graph	A graph with paths connecting every two nodes
Cycle	A path that starts and ends in the same node
Directed graph	A collection of points and arrows connecting some pairs of points
Disjunction	Boolean OR operation
Domain	The set of possible inputs to a function
Edge	A line in a graph
Element	An object in a set
Empty set	The set with no members
Empty string	The string of length zero
Equivalence relation	A binary relation that is reflexive, symmetric, and transitive
Function	An operation that translates inputs into outputs
Graph	A collection of points and lines connecting some pairs of points
Intersection	An operation on sets forming the set of common elements
k -tuple	A list of k objects
Language	A set of strings
Member	An object in a set
Node	A point in a graph
Ordered pair	A list of two elements
Path	A sequence of nodes in a graph connected by edges
Predicate	A function whose range is {TRUE, FALSE}
Property	A predicate
Range	The set from which outputs of a function are drawn
Relation	A predicate, most typically when the domain is a set of k -tuples
Sequence	A list of objects
Set	A group of objects
Simple path	A path without repetition
Singleton set	A set with one member
String	A finite list of symbols from an alphabet
Symbol	A member of an alphabet
Tree	A connected graph without simple cycles
Union	An operation on sets combining all elements into a single set
Unordered pair	A set with two members
Vertex	A point in a graph

0.3

DEFINITIONS, THEOREMS, AND PROOFS

Theorems and proofs are the heart and soul of mathematics and definitions are its spirit. These three entities are central to every mathematical subject, including ours.

Definitions describe the objects and notions that we use. A definition may be simple, as in the definition of *set* given earlier in this chapter, or complex as in the definition of *security* in a cryptographic system. Precision is essential to any mathematical definition. When defining some object, we must make clear what constitutes that object and what does not.

After we have defined various objects and notions, we usually make **mathematical statements** about them. Typically, a statement expresses that some object has a certain property. The statement may or may not be true; but like a definition, it must be precise. No ambiguity about its meaning is allowed.

A **proof** is a convincing logical argument that a statement is true. In mathematics, an argument must be airtight; that is, convincing in an absolute sense. In everyday life or in the law, the standard of proof is lower. A murder trial demands proof “beyond any reasonable doubt.” The weight of evidence may compel the jury to accept the innocence or guilt of the suspect. However, evidence plays no role in a mathematical proof. A mathematician demands proof beyond *any* doubt.

A **theorem** is a mathematical statement proved true. Generally we reserve the use of that word for statements of special interest. Occasionally we prove statements that are interesting only because they assist in the proof of another, more significant statement. Such statements are called **lemmas**. Occasionally a theorem or its proof may allow us to conclude easily that other, related statements are true. These statements are called **corollaries** of the theorem.

FINDING PROOFS

The only way to determine the truth or falsity of a mathematical statement is with a mathematical proof. Unfortunately, finding proofs isn’t always easy. It can’t be reduced to a simple set of rules or processes. During this course, you will be asked to present proofs of various statements. Don’t despair at the prospect! Even though no one has a recipe for producing proofs, some helpful general strategies are available.

First, carefully read the statement you want to prove. Do you understand all the notation? Rewrite the statement in your own words. Break it down and consider each part separately.

Sometimes the parts of a multipart statement are not immediately evident. One frequently occurring type of multipart statement has the form “ P if and only if Q ”, often written “ P iff Q ”, where both P and Q are mathematical statements. This notation is shorthand for a two-part statement. The first part is “ P only if Q ,” which means: If P is true, then Q is true, written $P \Rightarrow Q$. The second is “ P if Q ,” which means: If Q is true, then P is true, written $P \Leftarrow Q$. The first of these parts is the **forward direction** of the original statement and the second is the **reverse direction**. We write “ P if and only if Q ” as $P \iff Q$. To prove a statement of this form, you must prove each of the two directions. Often, one of these directions is easier to prove than the other.

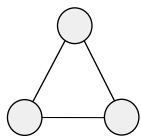
Another type of multipart statement states that two sets A and B are equal. The first part states that A is a subset of B , and the second part states that B is a subset of A . Thus one common way to prove that $A = B$ is to prove that every member of A also is a member of B , and that every member of B also is a member of A .

Next, when you want to prove a statement or part thereof, try to get an intuitive, “gut” feeling of why it should be true. Experimenting with examples is especially helpful. Thus if the statement says that all objects of a certain type have a particular property, pick a few objects of that type and observe that they actually do have that property. After doing so, try to find an object that fails to have the property, called a **counterexample**. If the statement actually is true, you will not be able to find a counterexample. Seeing where you run into difficulty when you attempt to find a counterexample can help you understand why the statement is true.

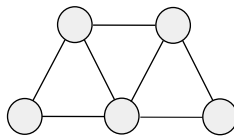
EXAMPLE 0.19

Suppose that you want to prove the statement *for every graph G , the sum of the degrees of all the nodes in G is an even number*.

First, pick a few graphs and observe this statement in action. Here are two examples.

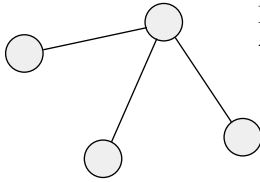


$$\begin{aligned}\text{sum} &= 2+2+2 \\ &= 6\end{aligned}$$



$$\begin{aligned}\text{sum} &= 2+3+4+3+2 \\ &= 14\end{aligned}$$

Next, try to find a counterexample; that is, a graph in which the sum is an odd number.



Every time an edge is added,
the sum increases by 2.

Can you now begin to see why the statement is true and how to prove it? ■

If you are still stuck trying to prove a statement, try something easier. Attempt to prove a special case of the statement. For example, if you are trying to prove that some property is true for every $k > 0$, first try to prove it for $k = 1$. If you succeed, try it for $k = 2$, and so on until you can understand the more general case. If a special case is hard to prove, try a different special case or perhaps a special case of the special case.

Finally, when you believe that you have found the proof, you must write it up properly. A well-written proof is a sequence of statements, wherein each one follows by simple reasoning from previous statements in the sequence. Carefully writing a proof is important, both to enable a reader to understand it, and for you to be sure that it is free from errors.

The following are a few tips for producing a proof.

- *Be patient.* Finding proofs takes time. If you don't see how to do it right away, don't worry. Researchers sometimes work for weeks or even years to find a single proof.
- *Come back to it.* Look over the statement you want to prove, think about it a bit, leave it, and then return a few minutes or hours later. Let the unconscious, intuitive part of your mind have a chance to work.
- *Be neat.* When you are building your intuition for the statement you are trying to prove, use simple, clear pictures and/or text. You are trying to develop your insight into the statement, and sloppiness gets in the way of insight. Furthermore, when you are writing a solution for another person to read, neatness will help that person understand it.
- *Be concise.* Brevity helps you express high-level ideas without getting lost in details. Good mathematical notation is useful for expressing ideas concisely. But be sure to include enough of your reasoning when writing up a proof so that the reader can easily understand what you are trying to say.

For practice, let's prove one of DeMorgan's laws.

THEOREM 0.20

For any two sets A and B , $\overline{A \cup B} = \overline{A} \cap \overline{B}$.

First, is the meaning of this theorem clear? If you don't understand the meaning of the symbols $\cup$ or $\cap$ or the overbar, review the discussion on page 4.

To prove this theorem, we must show that the two sets $\overline{A \cup B}$ and $\overline{A} \cap \overline{B}$ are equal. Recall that we may prove that two sets are equal by showing that every member of one set also is a member of the other and vice versa. Before looking at the following proof, consider a few examples and then try to prove it yourself.

PROOF This theorem states that two sets, $\overline{A \cup B}$ and $\overline{A} \cap \overline{B}$, are equal. We prove this assertion by showing that every element of one also is an element of the other and vice versa.

Suppose that x is an element of $\overline{A \cup B}$. Then x is not in $A \cup B$ from the definition of the complement of a set. Therefore, x is not in A and x is not in B , from the definition of the union of two sets. In other words, x is in $\overline{A}$ and x is in $\overline{B}$. Hence the definition of the intersection of two sets shows that x is in $\overline{A} \cap \overline{B}$.

For the other direction, suppose that x is in $\overline{A} \cap \overline{B}$. Then x is in both $\overline{A}$ and $\overline{B}$. Therefore, x is not in A and x is not in B , and thus not in the union of these two sets. Hence x is in the complement of the union of these sets; in other words, x is in $\overline{A \cup B}$, which completes the proof of the theorem.

Let's now prove the statement in Example 0.19.

THEOREM 0.21

For every graph G , the sum of the degrees of all the nodes in G is an even number.

PROOF Every edge in G is connected to two nodes. Each edge contributes 1 to the degree of each node to which it is connected. Therefore, each edge contributes 2 to the sum of the degrees of all the nodes. Hence, if G contains e edges, then the sum of the degrees of all the nodes of G is $2e$, which is an even number.

0.4

TYPES OF PROOF

Several types of arguments arise frequently in mathematical proofs. Here, we describe a few that often occur in the theory of computation. Note that a proof may contain more than one type of argument because the proof may contain within it several different subproofs.

PROOF BY CONSTRUCTION

Many theorems state that a particular type of object exists. One way to prove such a theorem is by demonstrating how to construct the object. This technique is a *proof by construction*.

Let's use a proof by construction to prove the following theorem. We define a graph to be *k-regular* if every node in the graph has degree *k*.

THEOREM 0.22

For each even number *n* greater than 2, there exists a 3-regular graph with *n* nodes.

PROOF Let *n* be an even number greater than 2. Construct graph $G = (V, E)$ with *n* nodes as follows. The set of nodes of *G* is $V = \{0, 1, \dots, n-1\}$, and the set of edges of *G* is the set

$$E = \{ \{i, i+1\} \mid \text{for } 0 \leq i \leq n-2 \} \cup \{ \{n-1, 0\} \} \\ \cup \{ \{i, i+n/2\} \mid \text{for } 0 \leq i \leq n/2-1 \}.$$

Picture the nodes of this graph written consecutively around the circumference of a circle. In that case, the edges described in the top line of *E* go between adjacent pairs around the circle. The edges described in the bottom line of *E* go between nodes on opposite sides of the circle. This mental picture clearly shows that every node in *G* has degree 3.

PROOF BY CONTRADICTION

In one common form of argument for proving a theorem, we assume that the theorem is false and then show that this assumption leads to an obviously false consequence, called a contradiction. We use this type of reasoning frequently in everyday life, as in the following example.

EXAMPLE 0.23

Jack sees Jill, who has just come in from outdoors. On observing that she is completely dry, he knows that it is not raining. His “proof” that it is not raining is that *if it were raining* (the assumption that the statement is false), *Jill would be wet* (the obviously false consequence). Therefore, it must not be raining. ■

Next, let’s prove by contradiction that the square root of 2 is an irrational number. A number is **rational** if it is a fraction $\frac{m}{n}$, where m and n are integers; in other words, a rational number is the *ratio* of integers m and n . For example, $\frac{2}{3}$ obviously is a rational number. A number is **irrational** if it is not rational.

THEOREM 0.24

$\sqrt{2}$ is irrational.

PROOF First, we assume for the purpose of later obtaining a contradiction that $\sqrt{2}$ is rational. Thus

$$\sqrt{2} = \frac{m}{n},$$

where m and n are integers. If both m and n are divisible by the same integer greater than 1, divide both by the largest such integer. Doing so doesn’t change the value of the fraction. Now, at least one of m and n must be an odd number.

We multiply both sides of the equation by n and obtain

$$n\sqrt{2} = m.$$

We square both sides and obtain

$$2n^2 = m^2.$$

Because m^2 is 2 times the integer n^2 , we know that m^2 is even. Therefore, m , too, is even, as the square of an odd number always is odd. So we can write $m = 2k$ for some integer k . Then, substituting $2k$ for m , we get

$$\begin{aligned} 2n^2 &= (2k)^2 \\ &= 4k^2. \end{aligned}$$

Dividing both sides by 2, we obtain

$$n^2 = 2k^2.$$

But this result shows that n^2 is even and hence that n is even. Thus we have established that both m and n are even. But we had earlier reduced m and n so that they were *not* both even—a contradiction.

PROOF BY INDUCTION

Proof by induction is an advanced method used to show that all elements of an infinite set have a specified property. For example, we may use a proof by induction to show that an arithmetic expression computes a desired quantity for

every assignment to its variables, or that a program works correctly at all steps or for all inputs.

To illustrate how proof by induction works, let's take the infinite set to be the natural numbers, $\mathcal{N} = \{1, 2, 3, \dots\}$, and say that the property is called $\mathcal{P}$. Our goal is to prove that $\mathcal{P}(k)$ is true for each natural number k . In other words, we want to prove that $\mathcal{P}(1)$ is true, as well as $\mathcal{P}(2)$, $\mathcal{P}(3)$, $\mathcal{P}(4)$, and so on.

Every proof by induction consists of two parts, the **basis** and the **induction step**. Each part is an individual proof on its own. The basis proves that $\mathcal{P}(1)$ is true. The induction step proves that for each $i \geq 1$, if $\mathcal{P}(i)$ is true, then so is $\mathcal{P}(i + 1)$.

When we have proven both of these parts, the desired result follows—namely, that $\mathcal{P}(i)$ is true for each i . Why? First, we know that $\mathcal{P}(1)$ is true because the basis alone proves it. Second, we know that $\mathcal{P}(2)$ is true because the induction step proves that if $\mathcal{P}(1)$ is true then $\mathcal{P}(2)$ is true, and we already know that $\mathcal{P}(1)$ is true. Third, we know that $\mathcal{P}(3)$ is true because the induction step proves that if $\mathcal{P}(2)$ is true then $\mathcal{P}(3)$ is true, and we already know that $\mathcal{P}(2)$ is true. This process continues for all natural numbers, showing that $\mathcal{P}(4)$ is true, $\mathcal{P}(5)$ is true, and so on.

Once you understand the preceding paragraph, you can easily understand variations and generalizations of the same idea. For example, the basis doesn't necessarily need to start with 1; it may start with any value b . In that case, the induction proof shows that $\mathcal{P}(k)$ is true for every k that is at least b .

In the induction step, the assumption that $\mathcal{P}(i)$ is true is called the **induction hypothesis**. Sometimes having the stronger induction hypothesis that $\mathcal{P}(j)$ is true for every $j \leq i$ is useful. The induction proof still works because when we want to prove that $\mathcal{P}(i + 1)$ is true, we have already proved that $\mathcal{P}(j)$ is true for every $j \leq i$.

The format for writing down a proof by induction is as follows.

Basis: Prove that $\mathcal{P}(1)$ is true.

⋮

Induction step: For each $i \geq 1$, assume that $\mathcal{P}(i)$ is true and use this assumption to show that $\mathcal{P}(i + 1)$ is true.

⋮

Now, let's prove by induction the correctness of the formula used to calculate the size of monthly payments of home mortgages. When buying a home, many people borrow some of the money needed for the purchase and repay this loan over a certain number of years. Typically, the terms of such repayments stipulate that a fixed amount of money is paid each month to cover the interest, as well as part of the original sum, so that the total is repaid in 30 years. The formula for calculating the size of the monthly payments is shrouded in mystery, but actually is quite simple. It touches many people's lives, so you should find it interesting. We use induction to prove that it works, making it a good illustration of that technique.

First, we set up the names and meanings of several variables. Let P be the *principal*, the amount of the original loan. Let $I > 0$ be the yearly *interest rate* of the loan, where $I = 0.06$ indicates a 6% rate of interest. Let Y be the monthly payment. For convenience, we use I to define another variable M , the monthly multiplier. It is the rate at which the loan changes each month because of the interest on it. Following standard banking practice, the monthly interest rate is one-twelfth of the annual rate so $M = 1 + I/12$, and interest is paid monthly (monthly compounding).

Two things happen each month. First, the amount of the loan tends to increase because of the monthly multiplier. Second, the amount tends to decrease because of the monthly payment. Let P_t be the amount of the loan outstanding after the t th month. Then $P_0 = P$ is the amount of the original loan, $P_1 = MP_0 - Y$ is the amount of the loan after one month, $P_2 = MP_1 - Y$ is the amount of the loan after two months, and so on. Now we are ready to state and prove a theorem by induction on t that gives a formula for the value of P_t .

THEOREM 0.25

For each $t \geq 0$,

$$P_t = PM^t - Y \left(\frac{M^t - 1}{M - 1} \right).$$

PROOF

Basis: Prove that the formula is true for $t = 0$. If $t = 0$, then the formula states that

$$P_0 = PM^0 - Y \left(\frac{M^0 - 1}{M - 1} \right).$$

We can simplify the right-hand side by observing that $M^0 = 1$. Thus we get

$$P_0 = P,$$

which holds because we have defined P_0 to be P . Therefore, we have proved that the basis of the induction is true.

Induction step: For each $k \geq 0$, assume that the formula is true for $t = k$ and show that it is true for $t = k + 1$. The induction hypothesis states that

$$P_k = PM^k - Y \left(\frac{M^k - 1}{M - 1} \right).$$

Our objective is to prove that

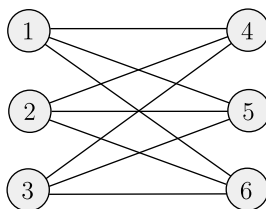
$$P_{k+1} = PM^{k+1} - Y \left(\frac{M^{k+1} - 1}{M - 1} \right).$$

- 0.3 Let A be the set $\{x, y, z\}$ and B be the set $\{x, y\}$.
- a. Is A a subset of B ?
 - b. Is B a subset of A ?
 - c. What is $A \cup B$?
 - d. What is $A \cap B$?
 - e. What is $A \times B$?
 - f. What is the power set of B ?
- 0.4 If A has a elements and B has b elements, how many elements are in $A \times B$? Explain your answer.
- 0.5 If C is a set with c elements, how many elements are in the power set of C ? Explain your answer.
- 0.6 Let X be the set $\{1, 2, 3, 4, 5\}$ and Y be the set $\{6, 7, 8, 9, 10\}$. The unary function $f: X \rightarrow Y$ and the binary function $g: X \times Y \rightarrow Y$ are described in the following tables.

n	$f(n)$	g	6	7	8	9	10
1	6	1	10	10	10	10	10
2	7	2	7	8	9	10	6
3	6	3	7	7	8	8	9
4	7	4	9	8	7	6	10
5	6	5	6	6	6	6	6

- a. What is the value of $f(2)$?
 - b. What are the range and domain of f ?
 - c. What is the value of $g(2, 10)$?
 - d. What are the range and domain of g ?
 - e. What is the value of $g(4, f(4))$?
- 0.7 For each part, give a relation that satisfies the condition.
- a. Reflexive and symmetric but not transitive
 - b. Reflexive and transitive but not symmetric
 - c. Symmetric and transitive but not reflexive
- 0.8 Consider the undirected graph $G = (V, E)$ where V , the set of nodes, is $\{1, 2, 3, 4\}$ and E , the set of edges, is $\{\{1, 2\}, \{2, 3\}, \{1, 3\}, \{2, 4\}, \{1, 4\}\}$. Draw the graph G . What are the degrees of each node? Indicate a path from node 3 to node 4 on your drawing of G .

0.9 Write a formal description of the following graph.



PROBLEMS

0.10 Find the error in the following proof that $2 = 1$.

Consider the equation $a = b$. Multiply both sides by a to obtain $a^2 = ab$. Subtract b^2 from both sides to get $a^2 - b^2 = ab - b^2$. Now factor each side, $(a + b)(a - b) = b(a - b)$, and divide each side by $(a - b)$ to get $a + b = b$. Finally, let a and b equal 1, which shows that $2 = 1$.

0.11 Let $S(n) = 1 + 2 + \cdots + n$ be the sum of the first n natural numbers and let $C(n) = 1^3 + 2^3 + \cdots + n^3$ be the sum of the first n cubes. Prove the following equalities by induction on n , to arrive at the curious conclusion that $C(n) = S^2(n)$ for every n .

- a. $S(n) = \frac{1}{2}n(n + 1)$.
- b. $C(n) = \frac{1}{4}(n^4 + 2n^3 + n^2) = \frac{1}{4}n^2(n + 1)^2$.

0.12 Find the error in the following proof that all horses are the same color.

CLAIM: In any set of h horses, all horses are the same color.

PROOF: By induction on h .

Basis: For $h = 1$. In any set containing just one horse, all horses clearly are the same color.

Induction step: For $k \geq 1$, assume that the claim is true for $h = k$ and prove that it is true for $h = k + 1$. Take any set H of $k + 1$ horses. We show that all the horses in this set are the same color. Remove one horse from this set to obtain the set H_1 with just k horses. By the induction hypothesis, all the horses in H_1 are the same color. Now replace the removed horse and remove a different one to obtain the set H_2 . By the same argument, all the horses in H_2 are the same color. Therefore, all the horses in H must be the same color, and the proof is complete.

0.13 Show that every graph with two or more nodes contains two nodes that have equal degrees.

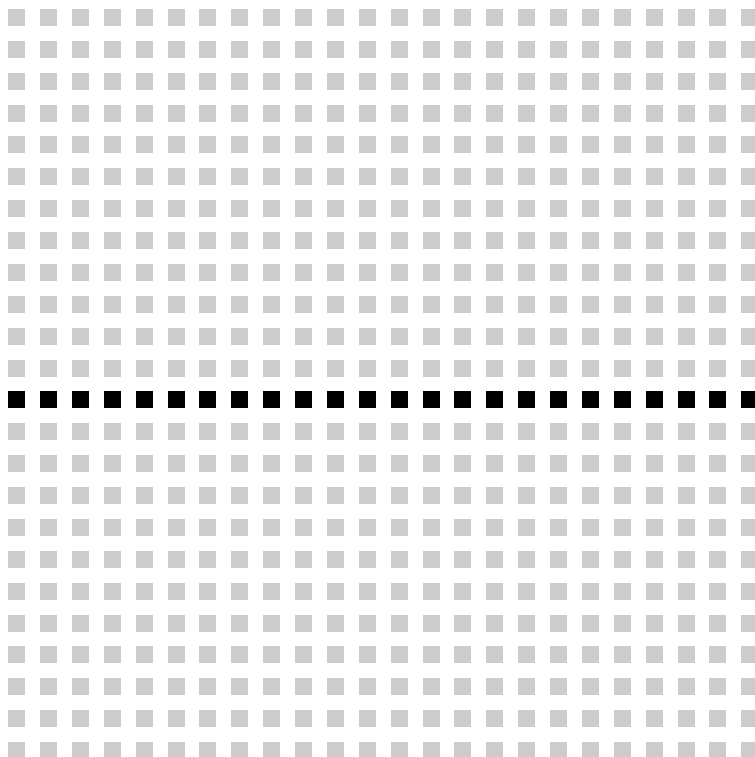
- A*0.14 Ramsey's theorem.** Let G be a graph. A **clique** in G is a subgraph in which every two nodes are connected by an edge. An **anti-clique**, also called an **independent set**, is a subgraph in which every two nodes are not connected by an edge. Show that every graph with n nodes contains either a clique or an anti-clique with at least $\frac{1}{2} \log_2 n$ nodes.
- A0.15** Use Theorem 0.25 to derive a formula for calculating the size of the monthly payment for a mortgage in terms of the principal P , the interest rate I , and the number of payments t . Assume that after t payments have been made, the loan amount is reduced to 0. Use the formula to calculate the dollar amount of each monthly payment for a 30-year mortgage with 360 monthly payments on an initial loan amount of \$100,000 with a 5% annual interest rate.



SELECTED SOLUTIONS

- 0.14** Make space for two piles of nodes: A and B . Then, starting with the entire graph, repeatedly add each remaining node x to A if its degree is greater than one half the number of remaining nodes and to B otherwise, and discard all nodes to which x isn't (is) connected if it was added to A (B). Continue until no nodes are left. At most half of the nodes are discarded at each of these steps, so at least $\log_2 n$ steps will occur before the process terminates. Each step adds a node to one of the piles, so one of the piles ends up with at least $\frac{1}{2} \log_2 n$ nodes. The A pile contains the nodes of a clique and the B pile contains the nodes of an anti-clique.
- 0.15** We let $P_t = 0$ and solve for Y to get the formula: $Y = PM^t(M - 1)/(M^t - 1)$. For $P = \$100,000$, $I = 0.05$, and $t = 360$, we have $M = 1 + (0.05)/12$. We use a calculator to find that $Y \approx \$536.82$ is the monthly payment.

PART ONE



A U T O M A T A A N D L A N G U A G E S

The theory of computation begins with a question: What is a computer? It is perhaps a silly question, as everyone knows that this thing I type on is a computer. But these real computers are quite complicated—too much so to allow us to set up a manageable mathematical theory of them directly. Instead, we use an idealized computer called a *computational model*. As with any model in science, a computational model may be accurate in some ways but perhaps not in others. Thus we will use several different computational models, depending on the features we want to focus on. We begin with the simplest model, called the *finite state machine* or *finite automaton*.

FINITE AUTOMATA

Finite automata are good models for computers with an extremely limited amount of memory. What can a computer do with such a small memory? Many useful things! In fact, we interact with such computers all the time, as they lie at the heart of various electromechanical devices.

The controller for an automatic door is one example of such a device. Often found at supermarket entrances and exits, automatic doors swing open when the controller senses that a person is approaching. An automatic door has a pad

in front to detect the presence of a person about to walk through the doorway. Another pad is located to the rear of the doorway so that the controller can hold the door open long enough for the person to pass all the way through and also so that the door does not strike someone standing behind it as it opens. This configuration is shown in the following figure.

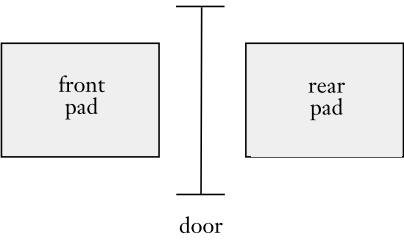


FIGURE 1.1
Top view of an automatic door

The controller is in either of two states: “OPEN” or “CLOSED,” representing the corresponding condition of the door. As shown in the following figures, there are four possible input conditions: “FRONT” (meaning that a person is standing on the pad in front of the doorway), “REAR” (meaning that a person is standing on the pad to the rear of the doorway), “BOTH” (meaning that people are standing on both pads), and “NEITHER” (meaning that no one is standing on either pad).

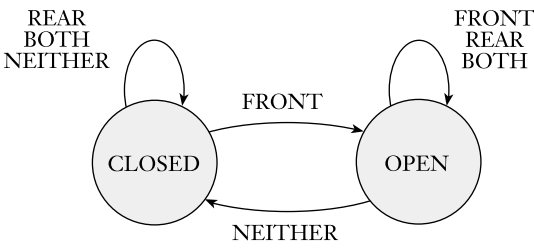


FIGURE 1.2
State diagram for an automatic door controller

		input signal			
state		NEITHER	FRONT	REAR	BOTH
	CLOSED	CLOSED	OPEN	CLOSED	CLOSED
	OPEN	CLOSED	OPEN	OPEN	OPEN

FIGURE 1.3
State transition table for an automatic door controller

The controller moves from state to state, depending on the input it receives. When in the CLOSED state and receiving input NEITHER or REAR, it remains in the CLOSED state. In addition, if the input BOTH is received, it stays CLOSED because opening the door risks knocking someone over on the rear pad. But if the input FRONT arrives, it moves to the OPEN state. In the OPEN state, if input FRONT, REAR, or BOTH is received, it remains in OPEN. If input NEITHER arrives, it returns to CLOSED.

For example, a controller might start in state CLOSED and receive the series of input signals FRONT, REAR, NEITHER, FRONT, BOTH, NEITHER, REAR, and NEITHER. It then would go through the series of states CLOSED (starting), OPEN, OPEN, CLOSED, OPEN, OPEN, CLOSED, CLOSED, and CLOSED.

Thinking of an automatic door controller as a finite automaton is useful because that suggests standard ways of representation as in Figures 1.2 and 1.3. This controller is a computer that has just a single bit of memory, capable of recording which of the two states the controller is in. Other common devices have controllers with somewhat larger memories. In an elevator controller, a state may represent the floor the elevator is on and the inputs might be the signals received from the buttons. This computer might need several bits to keep track of this information. Controllers for various household appliances such as dishwashers and electronic thermostats, as well as parts of digital watches and calculators, are additional examples of computers with limited memories. The design of such devices requires keeping the methodology and terminology of finite automata in mind.

Finite automata and their probabilistic counterpart **Markov chains** are useful tools when we are attempting to recognize patterns in data. These devices are used in speech processing and in optical character recognition. Markov chains have even been used to **model and predict price changes in financial markets**.

We will now take a closer look at finite automata from a mathematical perspective. We will develop a precise definition of a finite automaton, terminology for describing and manipulating finite automata, and theoretical results that describe their power and limitations. Besides giving you a clearer understanding of what finite automata are and what they can and cannot do, this theoretical development will allow you to practice and become more comfortable with mathematical definitions, theorems, and proofs in a relatively simple setting.

In beginning to describe the mathematical theory of finite automata, we do so in the abstract, without reference to any particular application. The following figure depicts a finite automaton called M_1 .

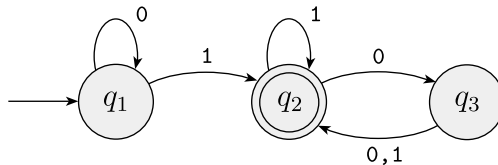


FIGURE 1.4

A finite automaton called M_1 that has three states

Figure 1.4 is called the **state diagram** of M_1 . It has three **states**, labeled q_1 , q_2 , and q_3 . The **start state**, q_1 , is indicated by the arrow pointing at it from nowhere. The **accept state**, q_2 , is the one with a double circle. The arrows going from one state to another are called **transitions**.

When this automaton receives an input string such as 1101, it processes that string and produces an output. The output is either **accept** or **reject**. We will consider only this yes/no type of output for now to keep things simple. The processing begins in M_1 's start state. The automaton receives the symbols from the input string one by one from left to right. After reading each symbol, M_1 moves from one state to another along the transition that has that symbol as its label. When it reads the last symbol, M_1 produces its output. The output is *accept* if M_1 is now in an accept state and *reject* if it is not.

For example, when we feed the input string 1101 into the machine M_1 in Figure 1.4, the processing proceeds as follows:

1. Start in state q_1 .
2. Read 1, follow transition from q_1 to q_2 .
3. Read 1, follow transition from q_2 to q_2 .
4. Read 0, follow transition from q_2 to q_3 .
5. Read 1, follow transition from q_3 to q_2 .
6. *Accept* because M_1 is in an accept state q_2 at the end of the input.

Experimenting with this machine on a variety of input strings reveals that it accepts the strings 1, 01, 11, and 0101010101. In fact, M_1 accepts any string that ends with a 1, as it goes to its accept state q_2 whenever it reads the symbol 1. In addition, it accepts strings 100, 0100, 110000, and 0101000000, and any string that ends with an even number of 0s following the last 1. It rejects other strings, such as 0, 10, 101000. Can you describe the language consisting of all strings that M_1 accepts? We will do so shortly.

FORMAL DEFINITION OF A FINITE AUTOMATON

In the preceding section, we used state diagrams to introduce finite automata. Now we define finite automata formally. Although state diagrams are easier to grasp intuitively, we need the formal definition, too, for two specific reasons.

First, a formal definition is precise. It resolves any uncertainties about what is allowed in a finite automaton. If you were uncertain about whether finite automata were allowed to have 0 accept states or whether they must have exactly one transition exiting every state for each possible input symbol, you could consult the formal definition and verify that the answer is yes in both cases. Second, a formal definition provides notation. Good notation helps you think and express your thoughts clearly.

The language of a formal definition is somewhat arcane, having some similarity to the language of a legal document. Both need to be precise, and every detail must be spelled out.

A finite automaton has several parts. It has a set of states and rules for going from one state to another, depending on the input symbol. It has an input alphabet that indicates the allowed input symbols. It has a start state and a set of accept states. The formal definition says that a finite automaton is a list of those five objects: set of states, input alphabet, rules for moving, start state, and accept states. In mathematical language, a list of five elements is often called a 5-tuple. Hence we define a finite automaton to be a 5-tuple consisting of these five parts.

We use something called a **transition function**, frequently denoted δ , to define the rules for moving. If the finite automaton has an arrow from a state x to a state y labeled with the input symbol 1, that means that if the automaton is in state x when it reads a 1, it then moves to state y . We can indicate the same thing with the transition function by saying that $\delta(x, 1) = y$. This notation is a kind of mathematical shorthand. Putting it all together, we arrive at the formal definition of finite automata.

DEFINITION 1.5

A **finite automaton** is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where

1. Q is a finite set called the **states**,
2. Σ is a finite set called the **alphabet**,
3. $\delta: Q \times \Sigma \rightarrow Q$ is the **transition function**,¹
4. $q_0 \in Q$ is the **start state**, and
5. $F \subseteq Q$ is the **set of accept states**.²

¹Refer back to page 7 if you are uncertain about the meaning of $\delta: Q \times \Sigma \rightarrow Q$.

²Accept states sometimes are called **final states**.

The formal definition precisely describes what we mean by a finite automaton. For example, returning to the earlier question of whether 0 accept states is allowable, you can see that setting F to be the empty set $\emptyset$ yields 0 accept states, which is allowable. Furthermore, the transition function δ specifies exactly one next state for each possible combination of a state and an input symbol. That answers our other question affirmatively, showing that exactly one transition arrow exits every state for each possible input symbol.

We can use the notation of the formal definition to describe individual finite automata by specifying each of the five parts listed in Definition 1.5. For example, let's return to the finite automaton M_1 we discussed earlier, redrawn here for convenience.

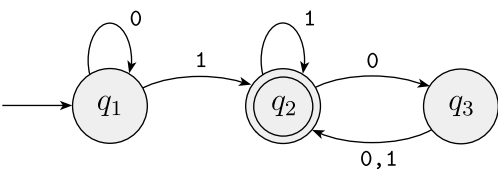


FIGURE 1.6
The finite automaton M_1

We can describe M_1 formally by writing $M_1 = (Q, \Sigma, \delta, q_1, F)$, where

- 1. $Q = \{q_1, q_2, q_3\}$,
- 2. $\Sigma = \{0, 1\}$,
- 3. δ is described as

	0	1
q_1	q_1	q_2
q_2	q_3	q_2
q_3	q_2	q_2

- 4. q_1 is the start state, and
- 5. $F = \{q_2\}$.

If A is the set of all strings that machine M accepts, we say that A is the **language of machine M** and write $L(M) = A$. We say that **M recognizes A** or that **M accepts A** . Because the term *accept* has different meanings when we refer to machines accepting strings and machines accepting languages, we prefer the term *recognize* for languages in order to avoid confusion.

A machine may accept several strings, but it always recognizes only one language. If the machine accepts no strings, it still recognizes one language—namely, the empty language $\emptyset$.

In our example, let

$$A = \{w \mid w \text{ contains at least one 1 and} \\ \text{an even number of 0s follow the last 1}\}.$$

Then $L(M_1) = A$, or equivalently, M_1 recognizes A .

EXAMPLES OF FINITE AUTOMATA

EXAMPLE 1.7

Here is the state diagram of finite automaton M_2 .

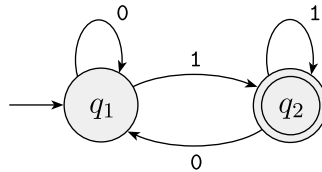


FIGURE 1.8

State diagram of the two-state finite automaton M_2

In the formal description, M_2 is $(\{q_1, q_2\}, \{0, 1\}, \delta, q_1, \{q_2\})$. The transition function δ is

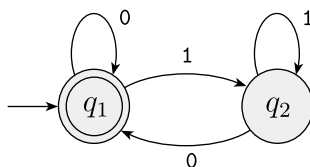
	0	1
q_1	q_1	q_2
q_2	q_1	q_2

Remember that the state diagram of M_2 and the formal description of M_2 contain the same information, only in different forms. You can always go from one to the other if necessary.

A good way to begin understanding any machine is to try it on some sample input strings. When you do these “experiments” to see how the machine is working, its method of functioning often becomes apparent. On the sample string 1101, the machine M_2 starts in its start state q_1 and proceeds first to state q_2 after reading the first 1, and then to states q_2 , q_1 , and q_2 after reading 1, 0, and 1. The string is accepted because q_2 is an accept state. But string 110 leaves M_2 in state q_1 , so it is rejected. After trying a few more examples, you would see that M_2 accepts all strings that end in a 1. Thus $L(M_2) = \{w \mid w \text{ ends in a 1}\}$.

EXAMPLE 1.9

Consider the finite automaton M_3 .

**FIGURE 1.10**

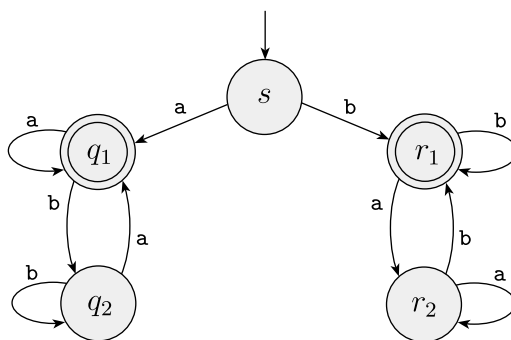
State diagram of the two-state finite automaton M_3

Machine M_3 is similar to M_2 except for the location of the accept state. As usual, the machine accepts all strings that leave it in an accept state when it has finished reading. Note that because the start state is also an accept state, M_3 accepts the empty string ε . As soon as a machine begins reading the empty string, it is at the end; so if the start state is an accept state, ε is accepted. In addition to the empty string, this machine accepts any string ending with a 0. Here,

$$L(M_3) = \{w \mid w \text{ is the empty string } \varepsilon \text{ or ends in a } 0\}.$$

EXAMPLE 1.11

The following figure shows a five-state machine M_4 .

**FIGURE 1.12**

Finite automaton M_4

Machine M_4 has two accept states, q_1 and r_1 , and operates over the alphabet $\Sigma = \{a, b\}$. Some experimentation shows that it accepts strings a , b , aa , bb , and bab , but not strings ab , ba , or $bbba$. This machine begins in state s , and after it reads the first symbol in the input, it goes either left into the q states or right into the r states. In both cases, it can never return to the start state (in contrast to the previous examples), as it has no way to get from any other state back to s . If the first symbol in the input string is a , then it goes left and accepts when the string ends with an a . Similarly, if the first symbol is a b , the machine goes right and accepts when the string ends in b . So M_4 accepts all strings that start and end with a or that start and end with b . In other words, M_4 accepts strings that start and end with the same symbol. ■

EXAMPLE 1.13

Figure 1.14 shows the three-state machine M_5 , which has a four-symbol input alphabet, $\Sigma = \{\langle \text{RESET} \rangle, 0, 1, 2\}$. We treat $\langle \text{RESET} \rangle$ as a single symbol.

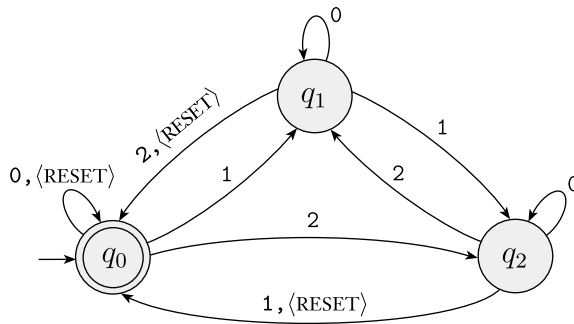


FIGURE 1.14

Finite automaton M_5

Machine M_5 keeps a running count of the sum of the numerical input symbols it reads, modulo 3. Every time it receives the $\langle \text{RESET} \rangle$ symbol, it resets the count to 0. It accepts if the sum is 0 modulo 3, or in other words, if the sum is a multiple of 3. ■

Describing a finite automaton by state diagram is not possible in some cases. That may occur when the diagram would be too big to draw or if, as in the next example, the description depends on some unspecified parameter. In these cases, we resort to a formal description to specify the machine.

EXAMPLE 1.15

Consider a generalization of Example 1.13, using the same four-symbol alphabet Σ . For each $i \geq 1$ let A_i be the language of all strings where the sum of the numbers is a multiple of i , except that the sum is reset to 0 whenever the symbol $\langle \text{RESET} \rangle$ appears. For each A_i we give a finite automaton B_i , recognizing A_i . We describe the machine B_i formally as follows: $B_i = (Q_i, \Sigma, \delta_i, q_0, \{q_0\})$, where Q_i is the set of i states $\{q_0, q_1, q_2, \dots, q_{i-1}\}$, and we design the transition function δ_i so that for each j , if B_i is in q_j , the running sum is j , modulo i . For each q_j let

$$\begin{aligned}\delta_i(q_j, 0) &= q_j, \\ \delta_i(q_j, 1) &= q_k, \text{ where } k = j + 1 \text{ modulo } i, \\ \delta_i(q_j, 2) &= q_k, \text{ where } k = j + 2 \text{ modulo } i, \text{ and} \\ \delta_i(q_j, \langle \text{RESET} \rangle) &= q_0.\end{aligned}$$

FORMAL DEFINITION OF COMPUTATION

So far we have described finite automata informally, using state diagrams, and with a formal definition, as a 5-tuple. The informal description is easier to grasp at first, but the formal definition is useful for making the notion precise, resolving any ambiguities that may have occurred in the informal description. Next we do the same for a finite automaton's computation. We already have an informal idea of the way it computes, and we now formalize it mathematically.

Let $M = (Q, \Sigma, \delta, q_0, F)$ be a finite automaton and let $w = w_1 w_2 \dots w_n$ be a string where each w_i is a member of the alphabet Σ . Then M **accepts** w if a sequence of states $r_0, r_1, \dots, r_n$ in Q exists with three conditions:

1. $r_0 = q_0$,
2. $\delta(r_i, w_{i+1}) = r_{i+1}$, for $i = 0, \dots, n-1$, and
3. $r_n \in F$.

Condition 1 says that the machine starts in the start state. Condition 2 says that the machine goes from state to state according to the transition function. Condition 3 says that the machine accepts its input if it ends up in an accept state. We say that M **recognizes language** A if $A = \{w \mid M \text{ accepts } w\}$.

DEFINITION 1.16

A language is called a **regular language** if some finite automaton recognizes it.

EXAMPLE 1.17

Take machine M_5 from Example 1.13. Let w be the string

$$10\langle\text{RESET}\rangle 22\langle\text{RESET}\rangle 012.$$

Then M_5 accepts w according to the formal definition of computation because the sequence of states it enters when computing on w is

$$q_0, q_1, q_1, q_0, q_2, q_1, q_0, q_0, q_1, q_0,$$

which satisfies the three conditions. The language of M_5 is

$$L(M_5) = \{w \mid \text{the sum of the symbols in } w \text{ is } 0 \text{ modulo } 3, \\ \text{except that } \langle\text{RESET}\rangle \text{ resets the count to } 0\}.$$

As M_5 recognizes this language, it is a regular language. ■

DESIGNING FINITE AUTOMATA

Whether it be of automaton or artwork, **design is a creative process**. As such, it cannot be reduced to a simple recipe or formula. However, you might find a particular approach helpful when designing various types of automata. That is, put *yourself* in the place of the machine you are trying to design and then see how you would go about performing the machine's task. Pretending that you are the machine is a psychological trick that helps engage your whole mind in the design process.

Let's design a finite automaton using the "reader as automaton" method just described. Suppose that you are given some language and want to design a finite automaton that recognizes it. Pretending to be the automaton, you receive an input string and must determine whether it is a member of the language the automaton is supposed to recognize. You get to see the symbols in the string one by one. After each symbol, you must decide whether the string seen so far is in the language. The reason is that you, like the machine, don't know when the end of the string is coming, so you must always be ready with the answer.

First, in order to make these decisions, you have to figure out what you need to remember about the string as you are reading it. Why not simply remember all you have seen? Bear in mind that you are pretending to be a finite automaton and that this type of machine has only a finite number of states, which means a finite memory. Imagine that the input is extremely long—say, from here to the moon—so that you could not possibly remember the entire thing. You have a finite memory—say, a single sheet of paper—which has a limited storage capacity. Fortunately, for many languages you don't need to remember the entire input. You need to remember only certain crucial information. Exactly which information is crucial depends on the particular language considered.

For example, suppose that the alphabet is $\{0,1\}$ and that the language consists of all strings with an odd number of 1s. You want to construct a finite automaton E_1 to recognize this language. Pretending to be the automaton, you start getting

an input string of 0s and 1s symbol by symbol. Do you need to remember the entire string seen so far in order to determine whether the number of 1s is odd? Of course not. Simply remember whether the number of 1s seen so far is even or odd and keep track of this information as you read new symbols. If you read a 1, flip the answer; but if you read a 0, leave the answer as is.

But how does this help you design E_1 ? Once you have determined the necessary information to remember about the string as it is being read, you represent this information as a finite list of possibilities. In this instance, the possibilities would be

1. even so far, and
2. odd so far.

Then you assign a state to each of the possibilities. These are the states of E_1 , as shown here.

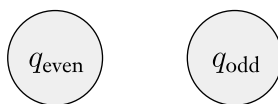


FIGURE 1.18

The two states q_{even} and q_{odd}

Next, you assign the transitions by seeing how to go from one possibility to another upon reading a symbol. So, if state q_{even} represents the even possibility and state q_{odd} represents the odd possibility, you would set the transitions to flip state on a 1 and stay put on a 0, as shown here.

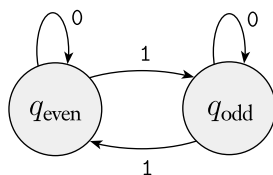


FIGURE 1.19

Transitions telling how the possibilities rearrange

Next, you set the start state to be the state corresponding to the possibility associated with having seen 0 symbols so far (the empty string ε). In this case, the start state corresponds to state q_{even} because 0 is an even number. Last, set the accept states to be those corresponding to possibilities where you want to accept the input string. Set q_{odd} to be an accept state because you want to accept

when you have seen an odd number of 1s. These additions are shown in the following figure.

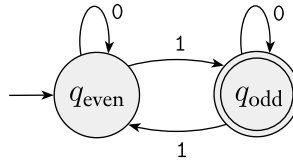


FIGURE 1.20
Adding the start and accept states

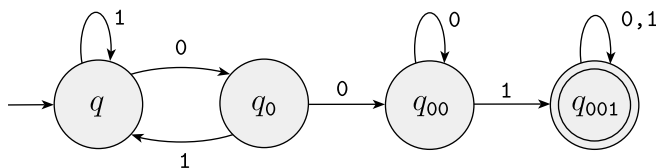
EXAMPLE 1.21

This example shows how to design a finite automaton E_2 to recognize the regular language of all strings that contain the string 001 as a substring. For example, 0010, 1001, 001, and 1111110011111 are all in the language, but 11 and 0000 are not. How would you recognize this language if you were pretending to be E_2 ? As symbols come in, you would initially skip over all 1s. If you come to a 0, then you note that you may have just seen the first of the three symbols in the pattern 001 you are seeking. If at this point you see a 1, there were too few 0s, so you go back to skipping over 1s. But if you see a 0 at that point, you should remember that you have just seen two symbols of the pattern. Now you simply need to continue scanning until you see a 1. If you find it, remember that you succeeded in finding the pattern and continue reading the input string until you get to the end.

So there are four possibilities: You

1. haven't just seen any symbols of the pattern,
2. have just seen a 0,
3. have just seen 00, or
4. have seen the entire pattern 001.

Assign the states q , q_0 , q_{00} , and q_{001} to these possibilities. You can assign the transitions by observing that from q reading a 1 you stay in q , but reading a 0 you move to q_0 . In q_0 reading a 1 you return to q , but reading a 0 you move to q_{00} . In q_{00} reading a 1 you move to q_{001} , but reading a 0 leaves you in q_{00} . Finally, in q_{001} reading a 0 or a 1 leaves you in q_{001} . The start state is q , and the only accept state is q_{001} , as shown in Figure 1.22.

**FIGURE 1.22**

Accepts strings containing 001

THE REGULAR OPERATIONS

In the preceding two sections, we introduced and defined finite automata and regular languages. We now begin to investigate their properties. Doing so will help develop a toolbox of techniques for designing automata to recognize particular languages. The toolbox also will include ways of proving that certain other languages are nonregular (i.e., beyond the capability of finite automata).

In arithmetic, the basic objects are numbers and the tools are operations for manipulating them, such as $+$ and $\times$. In the theory of computation, the objects are languages and the tools include operations specifically designed for manipulating them. We define three operations on languages, called the **regular operations**, and use them to study properties of the regular languages.

DEFINITION 1.23

Let A and B be languages. We define the regular operations **union**, **concatenation**, and **star** as follows:

- **Union:** $A \cup B = \{x \mid x \in A \text{ or } x \in B\}$.
- **Concatenation:** $A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$.
- **Star:** $A^* = \{x_1x_2 \dots x_k \mid k \geq 0 \text{ and each } x_i \in A\}$.

You are already familiar with the union operation. It simply takes all the strings in both A and B and lumps them together into one language.

The concatenation operation is a little trickier. It attaches a string from A in front of a string from B in all possible ways to get the strings in the new language.

The star operation is a bit different from the other two because it applies to a single language rather than to two different languages. That is, the star operation is a **unary operation** instead of a **binary operation**. It works by attaching any number of strings in A together to get a string in the new language. Because

“any number” includes 0 as a possibility, the empty string ε is always a member of A^* , no matter what A is.

EXAMPLE 1.24

Let the alphabet Σ be the standard 26 letters $\{a, b, \dots, z\}$. If $A = \{\text{good}, \text{bad}\}$ and $B = \{\text{boy}, \text{girl}\}$, then

$$A \cup B = \{\text{good}, \text{bad}, \text{boy}, \text{girl}\},$$

$$A \circ B = \{\text{goodboy}, \text{goodgirl}, \text{badboy}, \text{badgirl}\}, \text{ and}$$

$$A^* = \{\varepsilon, \text{good}, \text{bad}, \text{goodgood}, \text{goodbad}, \text{badgood}, \text{badbad}, \\ \text{goodgoodgood}, \text{goodgoodbad}, \text{goodbadgood}, \text{goodbadbad}, \dots\}.$$

Let $\mathcal{N} = \{1, 2, 3, \dots\}$ be the set of natural numbers. When we say that $\mathcal{N}$ is *closed under multiplication*, we mean that for any x and y in $\mathcal{N}$, the product $x \times y$ also is in $\mathcal{N}$. In contrast, $\mathcal{N}$ is not closed under division, as 1 and 2 are in $\mathcal{N}$ but $1/2$ is not. Generally speaking, a collection of objects is **closed** under some operation if applying that operation to members of the collection returns an object still in the collection. We show that the collection of regular languages is closed under all three of the regular operations. In Section 1.3, we show that these are useful tools for manipulating regular languages and understanding the power of finite automata. We begin with the union operation.

THEOREM 1.25

The class of regular languages is closed under the union operation.

In other words, if A_1 and A_2 are regular languages, so is $A_1 \cup A_2$.

PROOF IDEA We have regular languages A_1 and A_2 and want to show that $A_1 \cup A_2$ also is regular. Because A_1 and A_2 are regular, we know that some finite automaton M_1 recognizes A_1 and some finite automaton M_2 recognizes A_2 . To prove that $A_1 \cup A_2$ is regular, we demonstrate a finite automaton, call it M , that recognizes $A_1 \cup A_2$.

This is a proof by construction. We construct M from M_1 and M_2 . Machine M must accept its input exactly when either M_1 or M_2 would accept it in order to recognize the union language. It works by *simulating* both M_1 and M_2 and accepting if either of the simulations accept.

How can we make machine M simulate M_1 and M_2 ? Perhaps it first simulates M_1 on the input and then simulates M_2 on the input. But we must be careful here! Once the symbols of the input have been read and used to simulate M_1 , we can't “rewind the input tape” to try the simulation on M_2 . We need another approach.

Pretend that you are M . As the input symbols arrive one by one, you simulate both M_1 and M_2 simultaneously. That way, only one pass through the input is necessary. But can you keep track of both simulations with finite memory? All you need to remember is the state that each machine would be in if it had read up to this point in the input. Therefore, you need to remember a pair of states. How many possible pairs are there? If M_1 has k_1 states and M_2 has k_2 states, the number of pairs of states, one from M_1 and the other from M_2 , is the product $k_1 \times k_2$. This product will be the number of states in M , one for each pair. The transitions of M go from pair to pair, updating the current state for both M_1 and M_2 . The accept states of M are those pairs wherein either M_1 or M_2 is in an accept state.

PROOF

Let M_1 recognize A_1 , where $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$, and
 M_2 recognize A_2 , where $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$.

Construct M to recognize $A_1 \cup A_2$, where $M = (Q, \Sigma, \delta, q_0, F)$.

1. $Q = \{(r_1, r_2) \mid r_1 \in Q_1 \text{ and } r_2 \in Q_2\}$.
 This set is the **Cartesian product** of sets Q_1 and Q_2 and is written $Q_1 \times Q_2$. It is the set of all pairs of states, the first from Q_1 and the second from Q_2 .
2. Σ , the alphabet, is the same as in M_1 and M_2 . In this theorem and in all subsequent similar theorems, we assume for simplicity that both M_1 and M_2 have the same input alphabet Σ . The theorem remains true if they have different alphabets, Σ_1 and Σ_2 . We would then modify the proof to let $\Sigma = \Sigma_1 \cup \Sigma_2$.
3. δ , the transition function, is defined as follows. For each $(r_1, r_2) \in Q$ and each $a \in \Sigma$, let

$$\delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a)).$$

Hence δ gets a state of M (which actually is a pair of states from M_1 and M_2), together with an input symbol, and returns M 's next state.

4. q_0 is the pair (q_1, q_2) .
5. F is the set of pairs in which either member is an accept state of M_1 or M_2 . We can write it as

$$F = \{(r_1, r_2) \mid r_1 \in F_1 \text{ or } r_2 \in F_2\}.$$

This expression is the same as $F = (F_1 \times Q_2) \cup (Q_1 \times F_2)$. (Note that it is *not* the same as $F = F_1 \times F_2$. What would that give us instead?³)

³ This expression would define M 's accept states to be those for which *both* members of the pair are accept states. In this case, M would accept a string only if both M_1 and M_2 accept it, so the resulting language would be the *intersection* and not the union. In fact, this result proves that the class of regular languages is closed under intersection.

This concludes the construction of the finite automaton M that recognizes the union of A_1 and A_2 . This construction is fairly simple, and thus its correctness is evident from the strategy described in the proof idea. More complicated constructions require additional discussion to prove correctness. A formal correctness proof for a construction of this type usually proceeds by induction. For an example of a construction proved correct, see the proof of Theorem 1.54. Most of the constructions that you will encounter in this course are fairly simple and so do not require a formal correctness proof.

We have just shown that the union of two regular languages is regular, thereby proving that the class of regular languages is closed under the union operation. We now turn to the concatenation operation and attempt to show that the class of regular languages is closed under that operation, too.

THEOREM 1.26 ---

The class of regular languages is closed under the concatenation operation.

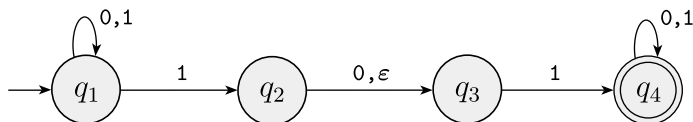
In other words, if A_1 and A_2 are regular languages then so is $A_1 \circ A_2$.

To prove this theorem, let's try something along the lines of the proof of the union case. As before, we can start with finite automata M_1 and M_2 recognizing the regular languages A_1 and A_2 . But now, instead of constructing automaton M to accept its input if either M_1 or M_2 accept, it must accept if its input can be broken into two pieces, where M_1 accepts the first piece and M_2 accepts the second piece. The problem is that M doesn't know where to break its input (i.e., where the first part ends and the second begins). To solve this problem, we introduce a new technique called nondeterminism.

1.2 --- NONDETERMINISM

Nondeterminism is a useful concept that has had great impact on the theory of computation. So far in our discussion, every step of a computation follows in a unique way from the preceding step. When the machine is in a given state and reads the next input symbol, we know what the next state will be—it is determined. We call this *deterministic* computation. In a *nondeterministic* machine, several choices may exist for the next state at any point.

Nondeterminism is a generalization of determinism, so every deterministic finite automaton is automatically a nondeterministic finite automaton. As Figure 1.27 shows, nondeterministic finite automata may have additional features.

**FIGURE 1.27**

The nondeterministic finite automaton N_1

The difference between a deterministic finite automaton, abbreviated DFA, and a nondeterministic finite automaton, abbreviated NFA, is immediately apparent. First, every state of a DFA always has exactly one exiting transition arrow for each symbol in the alphabet. The NFA shown in Figure 1.27 violates that rule. State q_1 has one exiting arrow for 0, but it has two for 1; q_2 has one arrow for 0, but it has none for 1. In an NFA, a state may have zero, one, or many exiting arrows for each alphabet symbol.

Second, in a DFA, labels on the transition arrows are symbols from the alphabet. This NFA has an arrow with the label ϵ . In general, an NFA may have arrows labeled with members of the alphabet or ϵ . Zero, one, or many arrows may exit from each state with the label ϵ .

How does an NFA compute? Suppose that we are running an NFA on an input string and come to a state with multiple ways to proceed. For example, say that we are in state q_1 in NFA N_1 and that the next input symbol is a 1. After reading that symbol, the machine splits into multiple copies of itself and follows *all* the possibilities in parallel. Each copy of the machine takes one of the possible ways to proceed and continues as before. If there are subsequent choices, the machine splits again. If the next input symbol doesn't appear on any of the arrows exiting the state occupied by a copy of the machine, that copy of the machine dies, along with the branch of the computation associated with it. Finally, if *any one* of these copies of the machine is in an accept state at the end of the input, the NFA accepts the input string.

If a state with an ϵ symbol on an exiting arrow is encountered, something similar happens. Without reading any input, the machine splits into multiple copies, one following each of the exiting ϵ -labeled arrows and one staying at the current state. Then the machine proceeds nondeterministically as before.

Nondeterminism may be viewed as a kind of parallel computation wherein multiple independent “processes” or “threads” can be running concurrently. When the NFA splits to follow several choices, that corresponds to a process “forking” into several children, each proceeding separately. If at least one of these processes accepts, then the entire computation accepts.

Another way to think of a nondeterministic computation is as a tree of possibilities. The root of the tree corresponds to the start of the computation. Every branching point in the tree corresponds to a point in the computation at which the machine has multiple choices. The machine accepts if at least one of the computation branches ends in an accept state, as shown in Figure 1.28.

On input 010110, start in the start state q_1 and read the first symbol 0. From q_1 there is only one place to go on a 0—namely, back to q_1 —so remain there. Next, read the second symbol 1. In q_1 on a 1 there are two choices: either stay in q_1 or move to q_2 . Nondeterministically, the machine splits in two to follow each choice. Keep track of the possibilities by placing a finger on each state where a machine could be. So you now have fingers on states q_1 and q_2 . An ϵ arrow exits state q_2 so the machine splits again; keep one finger on q_2 , and move the other to q_3 . You now have fingers on q_1 , q_2 , and q_3 .

When the third symbol 0 is read, take each finger in turn. Keep the finger on q_1 in place, move the finger on q_2 to q_3 , and remove the finger that has been on q_3 . That last finger had no 0 arrow to follow and corresponds to a process that simply “dies.” At this point, you have fingers on states q_1 and q_3 .

When the fourth symbol 1 is read, split the finger on q_1 into fingers on states q_1 and q_2 , then further split the finger on q_2 to follow the ϵ arrow to q_3 , and move the finger that was on q_3 to q_4 . You now have a finger on each of the four states.

When the fifth symbol 1 is read, the fingers on q_1 and q_3 result in fingers on states q_1 , q_2 , q_3 , and q_4 , as you saw with the fourth symbol. The finger on state q_2 is removed. The finger that was on q_4 stays on q_4 . Now you have two fingers on q_4 , so remove one because you only need to remember that q_4 is a possible state at this point, not that it is possible for multiple reasons.

When the sixth and final symbol 0 is read, keep the finger on q_1 in place, move the one on q_2 to q_3 , remove the one that was on q_3 , and leave the one on q_4 in place. You are now at the end of the string, and you accept if some finger is on an accept state. You have fingers on states q_1 , q_3 , and q_4 ; and as q_4 is an accept state, N_1 accepts this string.

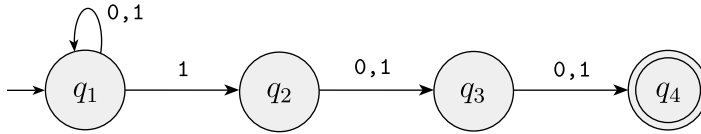
What does N_1 do on input 010? Start with a finger on q_1 . After reading the 0, you still have a finger only on q_1 ; but after the 1 there are fingers on q_1 , q_2 , and q_3 (don’t forget the ϵ arrow). After the third symbol 0, remove the finger on q_3 , move the finger on q_2 to q_3 , and leave the finger on q_1 where it is. At this point you are at the end of the input; and as no finger is on an accept state, N_1 rejects this input.

By continuing to experiment in this way, you will see that N_1 accepts all strings that contain either 101 or 11 as a substring.

Nondeterministic finite automata are useful in several respects. As we will show, every NFA can be converted into an equivalent DFA, and constructing NFAs is sometimes easier than directly constructing DFAs. An NFA may be much smaller than its deterministic counterpart, or its functioning may be easier to understand. Nondeterminism in finite automata is also a good introduction to nondeterminism in more powerful computational models because finite automata are especially easy to understand. Now we turn to several examples of NFAs.

EXAMPLE 1.30

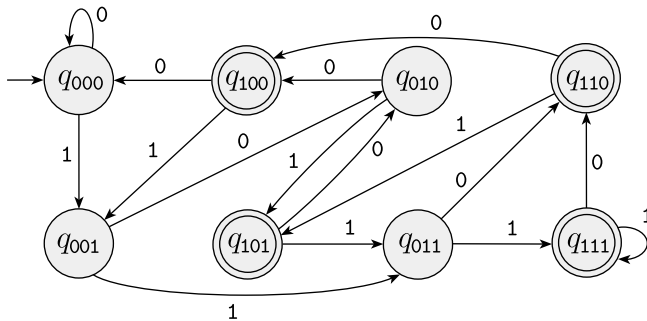
Let A be the language consisting of all strings over $\{0,1\}$ containing a 1 in the third position from the end (e.g., 000100 is in A but 0011 is not). The following four-state NFA N_2 recognizes A .

**FIGURE 1.31**

The NFA N_2 recognizing A

One good way to view the computation of this NFA is to say that it stays in the start state q_1 until it “guesses” that it is three places from the end. At that point, if the input symbol is a 1, it branches to state q_2 and uses q_3 and q_4 to “check” on whether its guess was correct.

As mentioned, every NFA can be converted into an equivalent DFA; but sometimes that DFA may have many more states. The smallest DFA for A contains eight states. Furthermore, understanding the functioning of the NFA is much easier, as you may see by examining the following figure for the DFA.

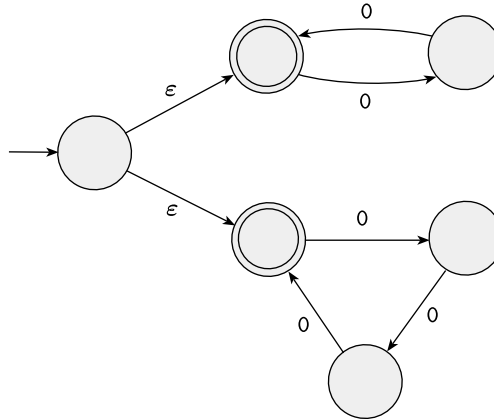
**FIGURE 1.32**

A DFA recognizing A

Suppose that we added ϵ to the labels on the arrows going from q_2 to q_3 and from q_3 to q_4 in machine N_2 in Figure 1.31. So both arrows would then have the label $0, 1, \epsilon$ instead of just $0, 1$. What language would N_2 recognize with this modification? Try modifying the DFA in Figure 1.32 to recognize that language.

EXAMPLE 1.33

The following NFA N_3 has an input alphabet $\{0\}$ consisting of a single symbol. An alphabet containing only one symbol is called a *unary alphabet*.

**FIGURE 1.34**

The NFA N_3

This machine demonstrates the convenience of having ε arrows. It accepts all strings of the form 0^k where k is a multiple of 2 or 3. (Remember that the superscript denotes repetition, not numerical exponentiation.) For example, N_3 accepts the strings ε , 00, 000, 0000, and 000000, but not 0 or 00000.

Think of the machine operating by initially guessing whether to test for a multiple of 2 or a multiple of 3 by branching into either the top loop or the bottom loop and then checking whether its guess was correct. Of course, we could replace this machine by one that doesn't have ε arrows or even any nondeterminism at all, but the machine shown is the easiest one to understand for this language. ■

EXAMPLE 1.35

We give another example of an NFA in Figure 1.36. Practice with it to satisfy yourself that it accepts the strings ε , a, baba, and baa, but that it doesn't accept the strings b, bb, and babba. Later we use this machine to illustrate the procedure for converting NFAs to DFAs.

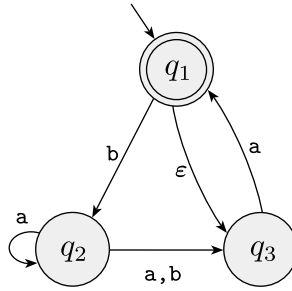


FIGURE 1.36
The NFA N_4

FORMAL DEFINITION OF A NONDETERMINISTIC FINITE AUTOMATON

The formal definition of a nondeterministic finite automaton is similar to that of a deterministic finite automaton. Both have states, an input alphabet, a transition function, a start state, and a collection of accept states. However, they differ in one essential way: in the type of transition function. In a DFA, the transition function takes a state and an input symbol and produces the next state. In an NFA, the transition function takes a state and an input symbol *or the empty string* and produces *the set of possible next states*. In order to write the formal definition, we need to set up some additional notation. For any set Q we write $\mathcal{P}(Q)$ to be the collection of all subsets of Q . Here $\mathcal{P}(Q)$ is called the **power set** of Q . For any alphabet Σ we write Σ_ϵ to be $\Sigma \cup \{\epsilon\}$. Now we can write the formal description of the type of the transition function in an NFA as $\delta: Q \times \Sigma_\epsilon \rightarrow \mathcal{P}(Q)$.

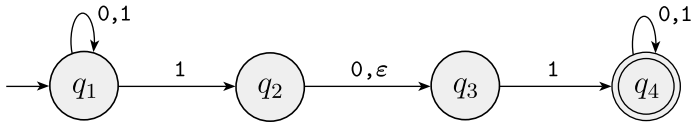
DEFINITION 1.37

A **nondeterministic finite automaton** is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where

1. Q is a finite set of states,
2. Σ is a finite alphabet,
3. $\delta: Q \times \Sigma_\epsilon \rightarrow \mathcal{P}(Q)$ is the transition function,
4. $q_0 \in Q$ is the start state, and
5. $F \subseteq Q$ is the set of accept states.

EXAMPLE 1.38

Recall the NFA N_1 :



The formal description of N_1 is $(Q, \Sigma, \delta, q_1, F)$, where

1. $Q = \{q_1, q_2, q_3, q_4\}$,
2. $\Sigma = \{0,1\}$,
3. δ is given as

	0	1	ϵ
q_1	$\{q_1\}$	$\{q_1, q_2\}$	$\emptyset$
q_2	$\{q_3\}$	$\emptyset$	$\{q_3\}$
q_3	$\emptyset$	$\{q_4\}$	$\emptyset$
q_4	$\{q_4\}$	$\{q_4\}$	$\emptyset$

4. q_1 is the start state, and
5. $F = \{q_4\}$.

The formal definition of computation for an NFA is similar to that for a DFA. Let $N = (Q, \Sigma, \delta, q_0, F)$ be an NFA and w a string over the alphabet Σ . Then we say that N **accepts** w if we can write w as $w = y_1 y_2 \cdots y_m$, where each y_i is a member of Σ_ϵ and a sequence of states $r_0, r_1, \dots, r_m$ exists in Q with three conditions:

1. $r_0 = q_0$,
2. $r_{i+1} \in \delta(r_i, y_{i+1})$, for $i = 0, \dots, m-1$, and
3. $r_m \in F$.

Condition 1 says that the machine starts out in the start state. Condition 2 says that state r_{i+1} is one of the allowable next states when N is in state r_i and reading y_{i+1} . Observe that $\delta(r_i, y_{i+1})$ is the *set* of allowable next states and so we say that r_{i+1} is a member of that set. Finally, condition 3 says that the machine accepts its input if the last state is an accept state.

EQUIVALENCE OF NFAS AND DFAS

Deterministic and nondeterministic finite automata recognize the same class of languages. Such equivalence is both surprising and useful. It is surprising because NFAs appear to have more power than DFAs, so we might expect that NFAs recognize more languages. It is useful because describing an NFA for a given language sometimes is much easier than describing a DFA for that language.

Say that two machines are **equivalent** if they recognize the same language.

THEOREM 1.39

Every nondeterministic finite automaton has an equivalent deterministic finite automaton.

PROOF IDEA If a language is recognized by an NFA, then we must show the existence of a DFA that also recognizes it. The idea is to convert the NFA into an equivalent DFA that simulates the NFA.

Recall the “reader as automaton” strategy for designing finite automata. How would you simulate the NFA if you were pretending to be a DFA? What do you need to keep track of as the input string is processed? In the examples of NFAs, you kept track of the various branches of the computation by placing a finger on each state that could be active at given points in the input. You updated the simulation by moving, adding, and removing fingers according to the way the NFA operates. All you needed to keep track of was the set of states having fingers on them.

If k is the number of states of the NFA, it has 2^k subsets of states. Each subset corresponds to one of the possibilities that the DFA must remember, so the DFA simulating the NFA will have 2^k states. Now we need to figure out which will be the start state and accept states of the DFA, and what will be its transition function. We can discuss this more easily after setting up some formal notation.

PROOF Let $N = (Q, \Sigma, \delta, q_0, F)$ be the NFA recognizing some language A . We construct a DFA $M = (Q', \Sigma, \delta', q_0', F')$ recognizing A . Before doing the full construction, let's first consider the easier case wherein N has no ϵ arrows. Later we take the ϵ arrows into account.

1. $Q' = \mathcal{P}(Q)$.
Every state of M is a set of states of N . Recall that $\mathcal{P}(Q)$ is the set of subsets of Q .
2. For $R \in Q'$ and $a \in \Sigma$, let $\delta'(R, a) = \{q \in Q \mid q \in \delta(r, a) \text{ for some } r \in R\}$. If R is a state of M , it is also a set of states of N . When M reads a symbol a in state R , it shows where a takes each state in R . Because each state may go to a set of states, we take the union of all these sets. Another way to write this expression is

$$\delta'(R, a) = \bigcup_{r \in R} \delta(r, a).^4$$

3. $q_0' = \{q_0\}$.
 M starts in the state corresponding to the collection containing just the start state of N .
4. $F' = \{R \in Q' \mid R \text{ contains an accept state of } N\}$.
The machine M accepts if one of the possible states that N could be in at this point is an accept state.

⁴The notation $\bigcup_{r \in R} \delta(r, a)$ means: the union of the sets $\delta(r, a)$ for each possible r in R .

Now we need to consider the ε arrows. To do so, we set up an extra bit of notation. For any state R of M , we define $E(R)$ to be the collection of states that can be reached from members of R by going only along ε arrows, including the members of R themselves. Formally, for $R \subseteq Q$ let

$$E(R) = \{q \mid q \text{ can be reached from } R \text{ by traveling along 0 or more } \varepsilon \text{ arrows}\}.$$

Then we modify the transition function of M to place additional fingers on all states that can be reached by going along ε arrows after every step. Replacing $\delta(r, a)$ by $E(\delta(r, a))$ achieves this effect. Thus

$$\delta'(R, a) = \{q \in Q \mid q \in E(\delta(r, a)) \text{ for some } r \in R\}.$$

Additionally, we need to modify the start state of M to move the fingers initially to all possible states that can be reached from the start state of N along the ε arrows. Changing q_0' to be $E(\{q_0\})$ achieves this effect. We have now completed the construction of the DFA M that simulates the NFA N .

The construction of M obviously works correctly. At every step in the computation of M on an input, it clearly enters a state that corresponds to the subset of states that N could be in at that point. Thus our proof is complete.

Theorem 1.39 states that every NFA can be converted into an equivalent DFA. Thus nondeterministic finite automata give an alternative way of characterizing the regular languages. We state this fact as a corollary of Theorem 1.39.

COROLLARY 1.40

A language is regular if and only if some nondeterministic finite automaton recognizes it.

One direction of the “if and only if” condition states that a language is regular if some NFA recognizes it. Theorem 1.39 shows that any NFA can be converted into an equivalent DFA. Consequently, if an NFA recognizes some language, so does some DFA, and hence the language is regular. The other direction of the “if and only if” condition states that a language is regular only if some NFA recognizes it. That is, if a language is regular, some NFA must be recognizing it. Obviously, this condition is true because a regular language has a DFA recognizing it and any DFA is also an NFA.

EXAMPLE 1.41

Let's illustrate the procedure we gave in the proof of Theorem 1.39 for converting an NFA to a DFA by using the machine N_4 that appears in Example 1.35. For clarity, we have relabeled the states of N_4 to be $\{1, 2, 3\}$. Thus in the formal description of $N_4 = (Q, \{a, b\}, \delta, 1, \{1\})$, the set of states Q is $\{1, 2, 3\}$ as shown in Figure 1.42.

To construct a DFA D that is equivalent to N_4 , we first determine D 's states. N_4 has three states, $\{1, 2, 3\}$, so we construct D with eight states, one for each subset of N_4 's states. We label each of D 's states with the corresponding subset. Thus D 's state set is

$$\{\emptyset, \{1\}, \{2\}, \{3\}, \{1,2\}, \{1,3\}, \{2,3\}, \{1,2,3\}\}.$$

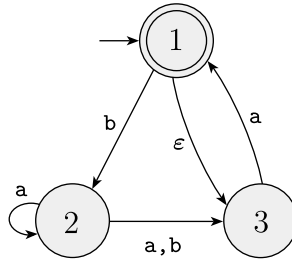


FIGURE 1.42
The NFA N_4

Next, we determine the start and accept states of D . The start state is $E(\{1\})$, the set of states that are reachable from 1 by traveling along ϵ arrows, plus 1 itself. An ϵ arrow goes from 1 to 3, so $E(\{1\}) = \{1, 3\}$. The new accept states are those containing N_4 's accept state; thus $\{\{1\}, \{1,2\}, \{1,3\}, \{1,2,3\}\}$.

Finally, we determine D 's transition function. Each of D 's states goes to one place on input a and one place on input b . We illustrate the process of determining the placement of D 's transition arrows with a few examples.

In D , state $\{2\}$ goes to $\{2,3\}$ on input a because in N_4 , state 2 goes to both 2 and 3 on input a and we can't go farther from 2 or 3 along ϵ arrows. State $\{2\}$ goes to state $\{3\}$ on input b because in N_4 , state 2 goes only to state 3 on input b and we can't go farther from 3 along ϵ arrows.

State $\{1\}$ goes to $\emptyset$ on a because no a arrows exit it. It goes to $\{2\}$ on b . Note that the procedure in Theorem 1.39 specifies that we follow the ϵ arrows *after* each input symbol is read. An alternative procedure based on following the ϵ arrows before reading each input symbol works equally well, but that method is not illustrated in this example.

State $\{3\}$ goes to $\{1,3\}$ on a because in N_4 , state 3 goes to 1 on a and 1 in turn goes to 3 with an ϵ arrow. State $\{3\}$ on b goes to $\emptyset$.

State $\{1,2\}$ on a goes to $\{2,3\}$ because 1 points at no states with a arrows, 2 points at both 2 and 3 with a arrows, and neither points anywhere with ϵ arrows. State $\{1,2\}$ on b goes to $\{2,3\}$. Continuing in this way, we obtain the diagram for D in Figure 1.43.

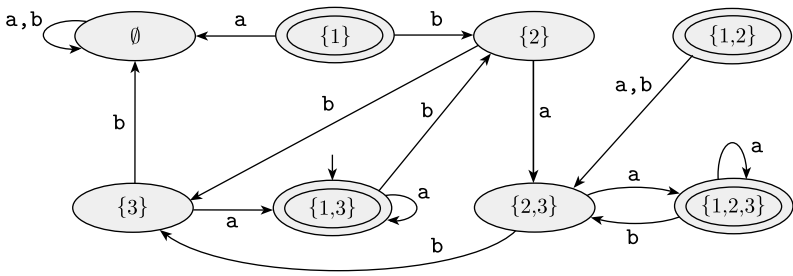


FIGURE 1.43
A DFA D that is equivalent to the NFA N_4

We may simplify this machine by observing that no arrows point at states $\{1\}$ and $\{1,2\}$, so they may be removed without affecting the performance of the machine. Doing so yields the following figure.

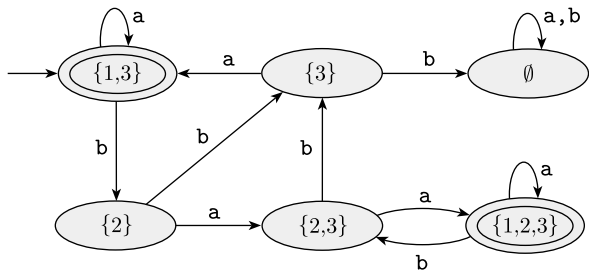


FIGURE 1.44
DFA D after removing unnecessary states

CLOSURE UNDER THE REGULAR OPERATIONS

Now we return to the closure of the class of regular languages under the regular operations that we began in Section 1.1. Our aim is to prove that the union, concatenation, and star of regular languages are still regular. We abandoned the original attempt to do so when dealing with the concatenation operation was too complicated. The use of nondeterminism makes the proofs much easier.

First, let's consider again closure under union. Earlier we proved closure under union by simulating deterministically both machines simultaneously via a Cartesian product construction. We now give a new proof to illustrate the

technique of nondeterminism. Reviewing the first proof, appearing on page 45, may be worthwhile to see how much easier and more intuitive the new proof is.

THEOREM 1.45

The class of regular languages is closed under the union operation.

PROOF IDEA We have regular languages A_1 and A_2 and want to prove that $A_1 \cup A_2$ is regular. The idea is to take two NFAs, N_1 and N_2 for A_1 and A_2 , and combine them into one new NFA, N .

Machine N must accept its input if either N_1 or N_2 accepts this input. The new machine has a new start state that branches to the start states of the old machines with ϵ arrows. In this way, the new machine nondeterministically guesses which of the two machines accepts the input. If one of them accepts the input, N will accept it, too.

We represent this construction in the following figure. On the left, we indicate the start and accept states of machines N_1 and N_2 with large circles and some additional states with small circles. On the right, we show how to combine N_1 and N_2 into N by adding additional transition arrows.

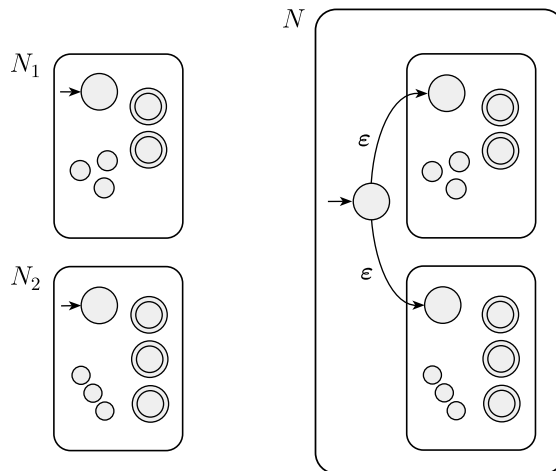


FIGURE 1.46

Construction of an NFA N to recognize $A_1 \cup A_2$

PROOF

Let $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ recognize A_1 , and
 $N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ recognize A_2 .

Construct $N = (Q, \Sigma, \delta, q_0, F)$ to recognize $A_1 \cup A_2$.

1. $Q = \{q_0\} \cup Q_1 \cup Q_2$.

The states of N are all the states of N_1 and N_2 , with the addition of a new start state q_0 .

2. The state q_0 is the start state of N .

3. The set of accept states $F = F_1 \cup F_2$.

The accept states of N are all the accept states of N_1 and N_2 . That way, N accepts if either N_1 accepts or N_2 accepts.

4. Define δ so that for any $q \in Q$ and any $a \in \Sigma_\varepsilon$,

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \\ \delta_2(q, a) & q \in Q_2 \\ \{q_1, q_2\} & q = q_0 \text{ and } a = \varepsilon \\ \emptyset & q = q_0 \text{ and } a \neq \varepsilon. \end{cases}$$

Now we can prove closure under concatenation. Recall that earlier, without nondeterminism, completing the proof would have been difficult.

THEOREM 1.47

The class of regular languages is closed under the concatenation operation.

PROOF IDEA We have regular languages A_1 and A_2 and want to prove that $A_1 \circ A_2$ is regular. The idea is to take two NFAs, N_1 and N_2 for A_1 and A_2 , and combine them into a new NFA N as we did for the case of union, but this time in a different way, as shown in Figure 1.48.

Assign N 's start state to be the start state of N_1 . The accept states of N_1 have additional ε arrows that nondeterministically allow branching to N_2 whenever N_1 is in an accept state, signifying that it has found an initial piece of the input that constitutes a string in A_1 . The accept states of N are the accept states of N_2 only. Therefore, it accepts when the input can be split into two parts, the first accepted by N_1 and the second by N_2 . We can think of N as nondeterministically guessing where to make the split.

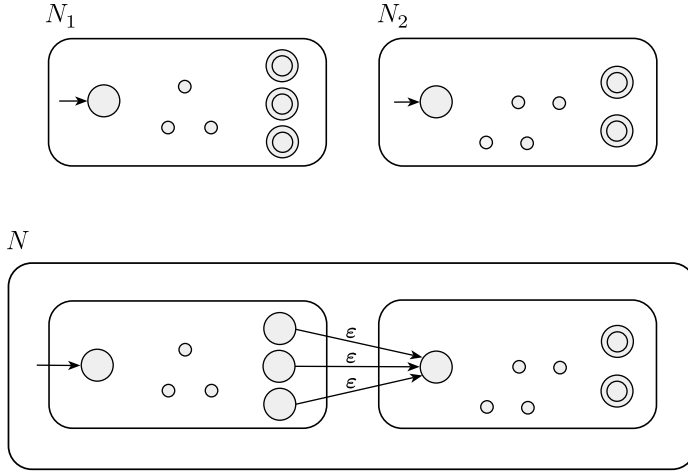


FIGURE 1.48
Construction of N to recognize $A_1 \circ A_2$

PROOF

Let $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ recognize A_1 , and
 $N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ recognize A_2 .

Construct $N = (Q, \Sigma, \delta, q_1, F_2)$ to recognize $A_1 \circ A_2$.

1. $Q = Q_1 \cup Q_2$.
 The states of N are all the states of N_1 and N_2 .
2. The state q_1 is the same as the start state of N_1 .
3. The accept states F_2 are the same as the accept states of N_2 .
4. Define δ so that for any $q \in Q$ and any $a \in \Sigma_\epsilon$,

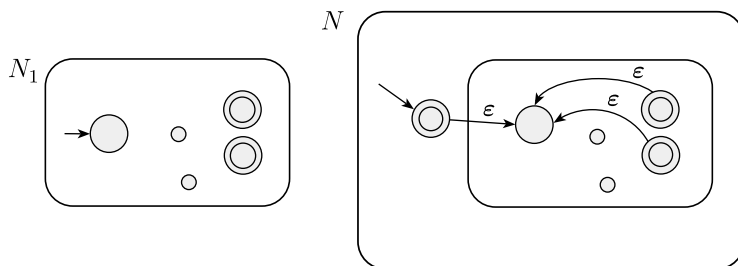
$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \text{ and } q \notin F_1 \\ \delta_1(q, a) & q \in F_1 \text{ and } a \neq \epsilon \\ \delta_1(q, a) \cup \{q_2\} & q \in F_1 \text{ and } a = \epsilon \\ \delta_2(q, a) & q \in Q_2. \end{cases}$$

THEOREM 1.49

The class of regular languages is closed under the star operation.

PROOF IDEA We have a regular language A_1 and want to prove that A_1^* also is regular. We take an NFA N_1 for A_1 and modify it to recognize A_1^* , as shown in the following figure. The resulting NFA N will accept its input whenever it can be broken into several pieces and N_1 accepts each piece.

We can construct N like N_1 with additional ϵ arrows returning to the start state from the accept states. This way, when processing gets to the end of a piece that N_1 accepts, the machine N has the option of jumping back to the start state to try to read another piece that N_1 accepts. In addition, we must modify N so that it accepts ϵ , which always is a member of A_1^* . One (slightly bad) idea is simply to add the start state to the set of accept states. This approach certainly adds ϵ to the recognized language, but it may also add other, undesired strings. Exercise 1.15 asks for an example of the failure of this idea. The way to fix it is to add a new start state, which also is an accept state, and which has an ϵ arrow to the old start state. This solution has the desired effect of adding ϵ to the language without adding anything else.

**FIGURE 1.50**

Construction of N to recognize A^*

PROOF Let $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ recognize A_1 . Construct $N = (Q, \Sigma, \delta, q_0, F)$ to recognize A_1^* .

1. $Q = \{q_0\} \cup Q_1$.

The states of N are the states of N_1 plus a new start state.

2. The state q_0 is the new start state.

3. $F = \{q_0\} \cup F_1$.

The accept states are the old accept states plus the new start state.

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \text{ and } q \notin F_1 \\ \delta_1(q, a) & q \in F_1 \text{ and } a \neq \varepsilon \\ \delta_1(q, a) \cup \{q_1\} & q \in F_1 \text{ and } a = \varepsilon \\ \{q_1\} & q = q_0 \text{ and } a = \varepsilon \\ \emptyset & q = q_0 \text{ and } a \neq \varepsilon. \end{cases}$$

REGULAR EXPRESSIONS

$$(5 + 3) \times 4.$$
$$(0 \cup 1)0^*.$$

Regular expressions have an important role in computer science applications. In applications involving text, users may want to search for strings that satisfy certain patterns. Regular expressions provide a powerful method for describing such patterns. Utilities such as `awk` and `grep` in UNIX, modern programming languages such as Perl, and text editors all provide mechanisms for the description of patterns by using regular expressions.

EXAMPLE 1.51

Another example of a regular expression is

$$(0 \cup 1)^*.$$

It starts with the language $(0 \cup 1)$ and applies the $*$ operation. The value of this expression is the language consisting of all possible strings of 0s and 1s. If $\Sigma = \{0,1\}$, we can write Σ as shorthand for the regular expression $(0 \cup 1)$. More generally, if Σ is any alphabet, the regular expression Σ describes the language consisting of all strings of length 1 over this alphabet, and Σ^* describes the language consisting of all strings over that alphabet. Similarly, Σ^*1 is the language that contains all strings that end in a 1. The language $(0\Sigma^*) \cup (\Sigma^*1)$ consists of all strings that start with a 0 or end with a 1. ■

In arithmetic, we say that $\times$ has precedence over $+$ to mean that when there is a choice, we do the $\times$ operation first. Thus in $2 + 3 \times 4$, the 3×4 is done before the addition. To have the addition done first, we must add parentheses to obtain $(2 + 3) \times 4$. In regular expressions, the star operation is done first, followed by concatenation, and finally union, unless parentheses change the usual order.

FORMAL DEFINITION OF A REGULAR EXPRESSION**DEFINITION 1.52**

Say that R is a *regular expression* if R is

1. a for some a in the alphabet Σ ,
2. ϵ ,
3. $\emptyset$,
4. $(R_1 \cup R_2)$, where R_1 and R_2 are regular expressions,
5. $(R_1 \circ R_2)$, where R_1 and R_2 are regular expressions, or
6. (R_1^*) , where R_1 is a regular expression.

In items 1 and 2, the regular expressions a and ϵ represent the languages $\{a\}$ and $\{\epsilon\}$, respectively. In item 3, the regular expression $\emptyset$ represents the empty language. In items 4, 5, and 6, the expressions represent the languages obtained by taking the union or concatenation of the languages R_1 and R_2 , or the star of the language R_1 , respectively.

Don't confuse the regular expressions ϵ and $\emptyset$. The expression ϵ represents the language containing a single string—namely, the empty string—whereas $\emptyset$ represents the language that doesn't contain any strings.

Seemingly, we are in danger of defining the notion of a regular expression in terms of itself. If true, we would have a *circular definition*, which would be invalid. However, R_1 and R_2 always are smaller than R . Thus we actually are defining regular expressions in terms of smaller regular expressions and thereby avoiding circularity. A definition of this type is called an *inductive definition*.

Parentheses in an expression may be omitted. If they are, evaluation is done in the precedence order: star, then concatenation, then union.

For convenience, we let R^+ be shorthand for RR^* . In other words, whereas R^* has all strings that are 0 or more concatenations of strings from R , the language R^+ has all strings that are 1 or more concatenations of strings from R . So $R^+ \cup \varepsilon = R^*$. In addition, we let R^k be shorthand for the concatenation of k R 's with each other.

When we want to distinguish between a regular expression R and the language that it describes, we write $L(R)$ to be the language of R .

EXAMPLE 1.53

In the following instances, we assume that the alphabet Σ is $\{0,1\}$.

1. $0^*10^* = \{w \mid w \text{ contains a single } 1\}$.
2. $\Sigma^*1\Sigma^* = \{w \mid w \text{ has at least one } 1\}$.
3. $\Sigma^*001\Sigma^* = \{w \mid w \text{ contains the string } 001 \text{ as a substring}\}$.
4. $1^*(01^+)^* = \{w \mid \text{every } 0 \text{ in } w \text{ is followed by at least one } 1\}$.
5. $(\Sigma\Sigma)^* = \{w \mid w \text{ is a string of even length}\}$.⁵
6. $(\Sigma\Sigma\Sigma)^* = \{w \mid \text{the length of } w \text{ is a multiple of } 3\}$.
7. $01 \cup 10 = \{01, 10\}$.
8. $0\Sigma^*0 \cup 1\Sigma^*1 \cup 0 \cup 1 = \{w \mid w \text{ starts and ends with the same symbol}\}$.
9. $(0 \cup \varepsilon)1^* = 01^* \cup 1^*$.

The expression $0 \cup \varepsilon$ describes the language $\{0, \varepsilon\}$, so the concatenation operation adds either 0 or ε before every string in 1^* .

10. $(0 \cup \varepsilon)(1 \cup \varepsilon) = \{\varepsilon, 0, 1, 01\}$.

11. $1^*\emptyset = \emptyset$.

Concatenating the empty set to any set yields the empty set.

12. $\emptyset^* = \{\varepsilon\}$.

The star operation puts together any number of strings from the language to get a string in the result. If the language is empty, the star operation can put together 0 strings, giving only the empty string.

⁵The *length* of a string is the number of symbols that it contains.

If we let R be any regular expression, we have the following identities. They are good tests of whether you understand the definition.

$$R \cup \emptyset = R.$$

Adding the empty language to any other language will not change it.

$$R \circ \varepsilon = R.$$

Joining the empty string to any string will not change it.

However, exchanging $\emptyset$ and ε in the preceding identities may cause the equalities to fail.

$$R \cup \varepsilon \text{ may not equal } R.$$

For example, if $R = 0$, then $L(R) = \{0\}$ but $L(R \cup \varepsilon) = \{0, \varepsilon\}$.

$$R \circ \emptyset \text{ may not equal } R.$$

For example, if $R = 0$, then $L(R) = \{0\}$ but $L(R \circ \emptyset) = \emptyset$.

Regular expressions are useful tools in the design of compilers for programming languages. Elemental objects in a programming language, called *tokens*, such as the variable names and constants, may be described with regular expressions. For example, a numerical constant that may include a fractional part and/or a sign may be described as a member of the language

$$(+ \cup - \cup \varepsilon) (D^+ \cup D^+ . D^* \cup D^* . D^+)$$

where $D = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ is the alphabet of decimal digits. Examples of generated strings are: 72, 3.14159, +7., and -.01.

Once the syntax of a programming language has been described with a regular expression in terms of its tokens, automatic systems can generate the *lexical analyzer*, the part of a compiler that initially processes the input program.

EQUIVALENCE WITH FINITE AUTOMATA

Regular expressions and finite automata are equivalent in their descriptive power. This fact is surprising because finite automata and regular expressions superficially appear to be rather different. However, any regular expression can be converted into a finite automaton that recognizes the language it describes, and vice versa. Recall that a regular language is one that is recognized by some finite automaton.

THEOREM 1.54

A language is regular if and only if some regular expression describes it.

This theorem has two directions. We state and prove each direction as a separate lemma.

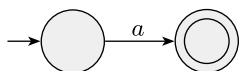
LEMMA 1.55

If a language is described by a regular expression, then it is regular.

PROOF IDEA Say that we have a regular expression R describing some language A . We show how to convert R into an NFA recognizing A . By Corollary 1.40, if an NFA recognizes A then A is regular.

PROOF Let's convert R into an NFA N . We consider the six cases in the formal definition of regular expressions.

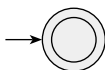
1. $R = a$ for some $a \in \Sigma$. Then $L(R) = \{a\}$, and the following NFA recognizes $L(R)$.



Note that this machine fits the definition of an NFA but not that of a DFA because it has some states with no exiting arrow for each possible input symbol. Of course, we could have presented an equivalent DFA here; but an NFA is all we need for now, and it is easier to describe.

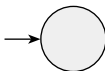
Formally, $N = (\{q_1, q_2\}, \Sigma, \delta, q_1, \{q_2\})$, where we describe δ by saying that $\delta(q_1, a) = \{q_2\}$ and that $\delta(r, b) = \emptyset$ for $r \neq q_1$ or $b \neq a$.

2. $R = \epsilon$. Then $L(R) = \{\epsilon\}$, and the following NFA recognizes $L(R)$.



Formally, $N = (\{q_1\}, \Sigma, \delta, q_1, \{q_1\})$, where $\delta(r, b) = \emptyset$ for any r and b .

3. $R = \emptyset$. Then $L(R) = \emptyset$, and the following NFA recognizes $L(R)$.



Formally, $N = (\{q\}, \Sigma, \delta, q, \emptyset)$, where $\delta(r, b) = \emptyset$ for any r and b .

4. $R = R_1 \cup R_2$.
5. $R = R_1 \circ R_2$.
6. $R = R_1^*$.

For the last three cases, we use the constructions given in the proofs that the class of regular languages is closed under the regular operations. In other words, we construct the NFA for R from the NFAs for R_1 and R_2 (or just R_1 in case 6) and the appropriate closure construction.

That ends the first part of the proof of Theorem 1.54, giving the easier direction of the if and only if condition. Before going on to the other direction, let's consider some examples whereby we use this procedure to convert a regular expression to an NFA.

EXAMPLE 1.56

We convert the regular expression $(ab \cup a)^*$ to an NFA in a sequence of stages. We build up from the smallest subexpressions to larger subexpressions until we have an NFA for the original expression, as shown in the following diagram. Note that this procedure generally doesn't give the NFA with the fewest states. In this example, the procedure gives an NFA with eight states, but the smallest equivalent NFA has only two states. Can you find it?

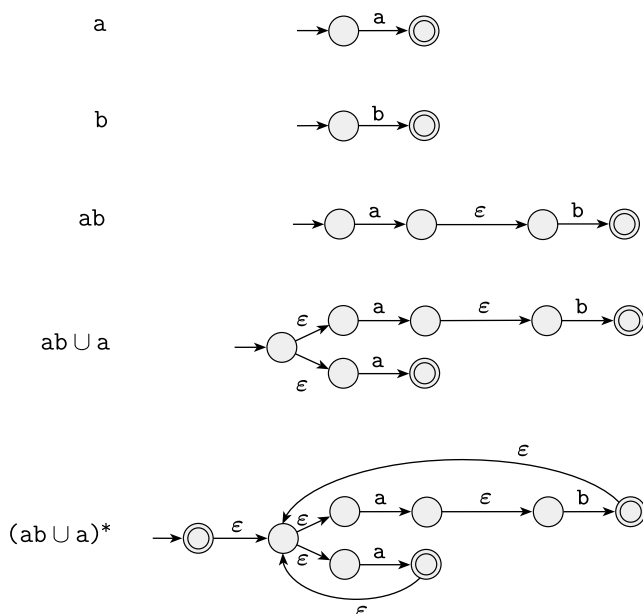


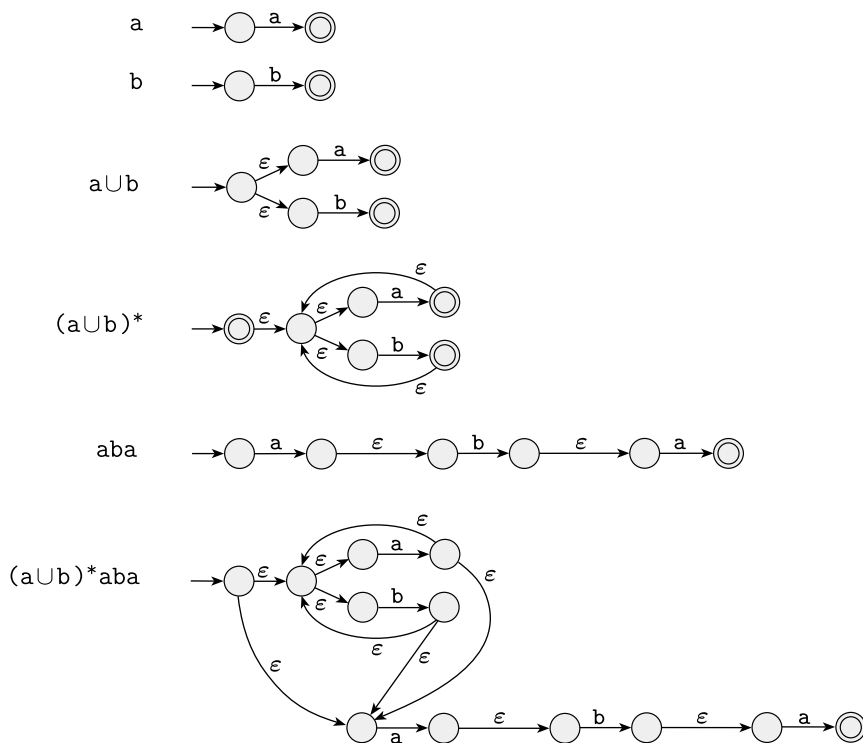
FIGURE 1.57

Building an NFA from the regular expression $(ab \cup a)^*$

■

EXAMPLE 1.58

In Figure 1.59, we convert the regular expression $(a \cup b)^*aba$ to an NFA. A few of the minor steps are not shown.

**FIGURE 1.59**

Building an NFA from the regular expression $(a \cup b)^*aba$

Now let's turn to the other direction of the proof of Theorem 1.54.

LEMMA 1.60

If a language is regular, then it is described by a regular expression.

PROOF IDEA We need to show that if a language A is regular, a regular expression describes it. Because A is regular, it is accepted by a DFA. We describe a procedure for converting DFAs into equivalent regular expressions.

We break this procedure into two parts, using a new type of finite automaton called a *generalized nondeterministic finite automaton*, GNFA. First we show how to convert DFAs into GNFAs, and then GNFAs into regular expressions.

Generalized nondeterministic finite automata are simply nondeterministic finite automata wherein the transition arrows may have any regular expressions as labels, instead of only members of the alphabet or ϵ . The GNFA reads blocks of symbols from the input, not necessarily just one symbol at a time as in an ordinary NFA. The GNFA moves along a transition arrow connecting two states by reading a block of symbols from the input, which themselves constitute a string described by the regular expression on that arrow. A GNFA is nondeterministic and so may have several different ways to process the same input string. It accepts its input if its processing can cause the GNFA to be in an accept state at the end of the input. The following figure presents an example of a GNFA.

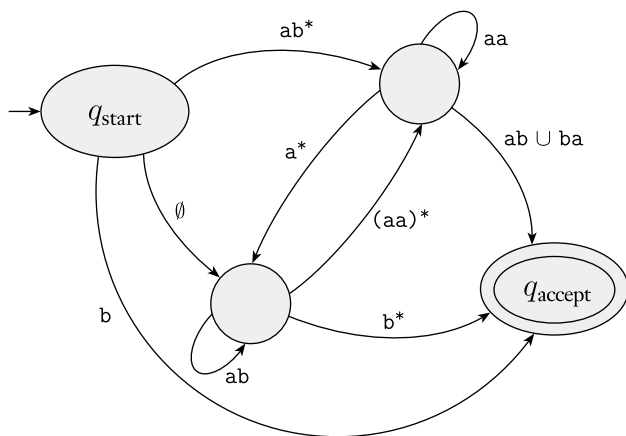


FIGURE 1.61
A generalized nondeterministic finite automaton

For convenience, we require that GNFA's always have a special form that meets the following conditions.

- The start state has transition arrows going to every other state but no arrows coming in from any other state.
- There is only a single accept state, and it has arrows coming in from every other state but no arrows going to any other state. Furthermore, the accept state is not the same as the start state.
- Except for the start and accept states, one arrow goes from every state to every other state and also from each state to itself.

We can easily convert a DFA into a GNFA in the special form. We simply add a new start state with an ϵ arrow to the old start state and a new accept state with ϵ arrows from the old accept states. If any arrows have multiple labels (or if there are multiple arrows going between the same two states in the same direction), we replace each with a single arrow whose label is the union of the previous labels. Finally, we add arrows labeled $\emptyset$ between states that had no arrows. This last step won't change the language recognized because a transition labeled with $\emptyset$ can never be used. From here on we assume that all GNFAs are in the special form.

Now we show how to convert a GNFA into a regular expression. Say that the GNFA has k states. Then, because a GNFA must have a start and an accept state and they must be different from each other, we know that $k \geq 2$. If $k > 2$, we construct an equivalent GNFA with $k - 1$ states. This step can be repeated on the new GNFA until it is reduced to two states. If $k = 2$, the GNFA has a single arrow that goes from the start state to the accept state. The label of this arrow is the equivalent regular expression. For example, the stages in converting a DFA with three states to an equivalent regular expression are shown in the following figure.

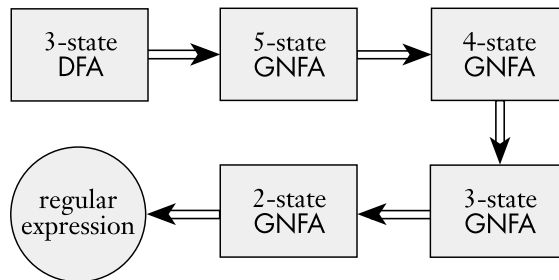


FIGURE 1.62
Typical stages in converting a DFA to a regular expression

The crucial step is constructing an equivalent GNFA with one fewer state when $k > 2$. We do so by selecting a state, ripping it out of the machine, and repairing the remainder so that the same language is still recognized. Any state will do, provided that it is not the start or accept state. We are guaranteed that such a state will exist because $k > 2$. Let's call the removed state q_{rip} .

After removing q_{rip} we repair the machine by altering the regular expressions that label each of the remaining arrows. The new labels compensate for the absence of q_{rip} by adding back the lost computations. The new label going from a state q_i to a state q_j is a regular expression that describes all strings that would

take the machine from q_i to q_j either directly or via q_{rip} . We illustrate this approach in Figure 1.63.

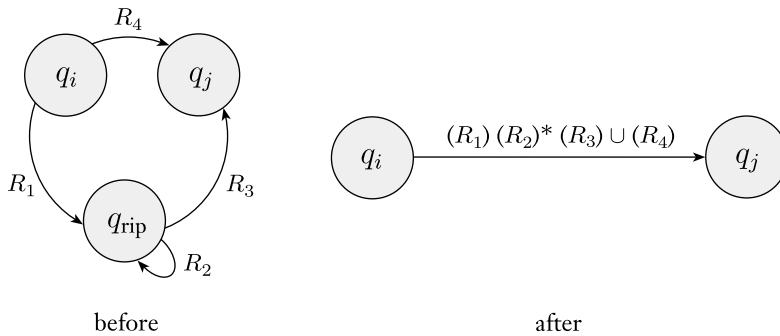


FIGURE 1.63

Constructing an equivalent GNFA with one fewer state

In the old machine, if

1. q_i goes to q_{rip} with an arrow labeled R_1 ,
2. q_{rip} goes to itself with an arrow labeled R_2 ,
3. q_{rip} goes to q_j with an arrow labeled R_3 , and
4. q_i goes to q_j with an arrow labeled R_4 ,

then in the new machine, the arrow from q_i to q_j gets the label

$$(R_1)(R_2)^*(R_3) \cup (R_4).$$

We make this change for each arrow going from any state q_i to any state q_j , including the case where $q_i = q_j$. The new machine recognizes the original language.

PROOF Let's now carry out this idea formally. First, to facilitate the proof, we formally define the new type of automaton introduced. A GNFA is similar to a nondeterministic finite automaton except for the transition function, which has the form

$$\delta: (Q - \{q_{accept}\}) \times (Q - \{q_{start}\}) \longrightarrow \mathcal{R}.$$

The symbol $\mathcal{R}$ is the collection of all regular expressions over the alphabet Σ , and q_{start} and q_{accept} are the start and accept states. If $\delta(q_i, q_j) = R$, the arrow from state q_i to state q_j has the regular expression R as its label. The domain of the transition function is $(Q - \{q_{accept}\}) \times (Q - \{q_{start}\})$ because an arrow connects every state to every other state, except that no arrows are coming from q_{accept} or going to q_{start} .

DEFINITION 1.64

A *generalized nondeterministic finite automaton* is a 5-tuple, $(Q, \Sigma, \delta, q_{\text{start}}, q_{\text{accept}})$, where

1. Q is the finite set of states,
2. Σ is the input alphabet,
3. $\delta: (Q - \{q_{\text{accept}}\}) \times (Q - \{q_{\text{start}}\}) \rightarrow \mathcal{R}$ is the transition function,
4. q_{start} is the start state, and
5. q_{accept} is the accept state.

A GNFA accepts a string w in Σ^* if $w = w_1 w_2 \cdots w_k$, where each w_i is in Σ^* and a sequence of states $q_0, q_1, \dots, q_k$ exists such that

1. $q_0 = q_{\text{start}}$ is the start state,
2. $q_k = q_{\text{accept}}$ is the accept state, and
3. for each i , we have $w_i \in L(R_i)$, where $R_i = \delta(q_{i-1}, q_i)$; in other words, R_i is the expression on the arrow from q_{i-1} to q_i .

Returning to the proof of Lemma 1.60, we let M be the DFA for language A . Then we convert M to a GNFA G by adding a new start state and a new accept state and additional transition arrows as necessary. We use the procedure $\text{CONVERT}(G)$, which takes a GNFA and returns an equivalent regular expression. This procedure uses *recursion*, which means that it calls itself. An infinite loop is avoided because the procedure calls itself only to process a GNFA that has one fewer state. The case where the GNFA has two states is handled without recursion.

$\text{CONVERT}(G)$:

1. Let k be the number of states of G .
2. If $k = 2$, then G must consist of a start state, an accept state, and a single arrow connecting them and labeled with a regular expression R .
Return the expression R .
3. If $k > 2$, we select any state $q_{\text{rip}} \in Q$ different from q_{start} and q_{accept} and let G' be the GNFA $(Q', \Sigma, \delta', q_{\text{start}}, q_{\text{accept}})$, where

$$Q' = Q - \{q_{\text{rip}}\},$$

and for any $q_i \in Q' - \{q_{\text{accept}}\}$ and any $q_j \in Q' - \{q_{\text{start}}\}$, let

$$\delta'(q_i, q_j) = (R_1)(R_2)^*(R_3) \cup (R_4),$$

for $R_1 = \delta(q_i, q_{\text{rip}})$, $R_2 = \delta(q_{\text{rip}}, q_{\text{rip}})$, $R_3 = \delta(q_{\text{rip}}, q_j)$, and $R_4 = \delta(q_i, q_j)$.

4. Compute $\text{CONVERT}(G')$ and return this value.

Next we prove that CONVERT returns a correct value.

CLAIM 1.65

For any GNFA G , $\text{CONVERT}(G)$ is equivalent to G .

We prove this claim by induction on k , the number of states of the GNFA.

Basis: Prove the claim true for $k = 2$ states. If G has only two states, it can have only a single arrow, which goes from the start state to the accept state. The regular expression label on this arrow describes all the strings that allow G to get to the accept state. Hence this expression is equivalent to G .

Induction step: Assume that the claim is true for $k - 1$ states and use this assumption to prove that the claim is true for k states. First we show that G and G' recognize the same language. Suppose that G accepts an input w . Then in an accepting branch of the computation, G enters a sequence of states:

$$q_{\text{start}}, q_1, q_2, q_3, \dots, q_{\text{accept}}.$$

If none of them is the removed state q_{rip} , clearly G' also accepts w . The reason is that each of the new regular expressions labeling the arrows of G' contains the old regular expression as part of a union.

If q_{rip} does appear, removing each run of consecutive q_{rip} states forms an accepting computation for G' . The states q_i and q_j bracketing a run have a new regular expression on the arrow between them that describes all strings taking q_i to q_j via q_{rip} on G . So G' accepts w .

Conversely, suppose that G' accepts an input w . As each arrow between any two states q_i and q_j in G' describes the collection of strings taking q_i to q_j in G , either directly or via q_{rip} , G must also accept w . Thus G and G' are equivalent.

The induction hypothesis states that when the algorithm calls itself recursively on input G' , the result is a regular expression that is equivalent to G' because G' has $k - 1$ states. Hence this regular expression also is equivalent to G , and the algorithm is proved correct.

This concludes the proof of Claim 1.65, Lemma 1.60, and Theorem 1.54.

EXAMPLE 1.66

In this example, we use the preceding algorithm to convert a DFA into a regular expression. We begin with the two-state DFA in Figure 1.67(a).

In Figure 1.67(b), we make a four-state GNFA by adding a new start state and a new accept state, called s and a instead of q_{start} and q_{accept} so that we can draw them conveniently. To avoid cluttering up the figure, we do not draw the arrows

labeled $\emptyset$, even though they are present. Note that we replace the label a, b on the self-loop at state 2 on the DFA with the label $a \cup b$ at the corresponding point on the GNFA. We do so because the DFA's label represents two transitions, one for a and the other for b , whereas the GNFA may have only a single transition going from 2 to itself.

In Figure 1.67(c), we remove state 2 and update the remaining arrow labels. In this case, the only label that changes is the one from 1 to a . In part (b) it was $\emptyset$, but in part (c) it is $b(a \cup b)^*$. We obtain this result by following step 3 of the CONVERT procedure. State q_i is state 1, state q_j is a , and q_{rip} is 2, so $R_1 = b$, $R_2 = a \cup b$, $R_3 = \epsilon$, and $R_4 = \emptyset$. Therefore, the new label on the arrow from 1 to a is $(b)(a \cup b)^*(\epsilon) \cup \emptyset$. We simplify this regular expression to $b(a \cup b)^*$.

In Figure 1.67(d), we remove state 1 from part (c) and follow the same procedure. Because only the start and accept states remain, the label on the arrow joining them is the regular expression that is equivalent to the original DFA.

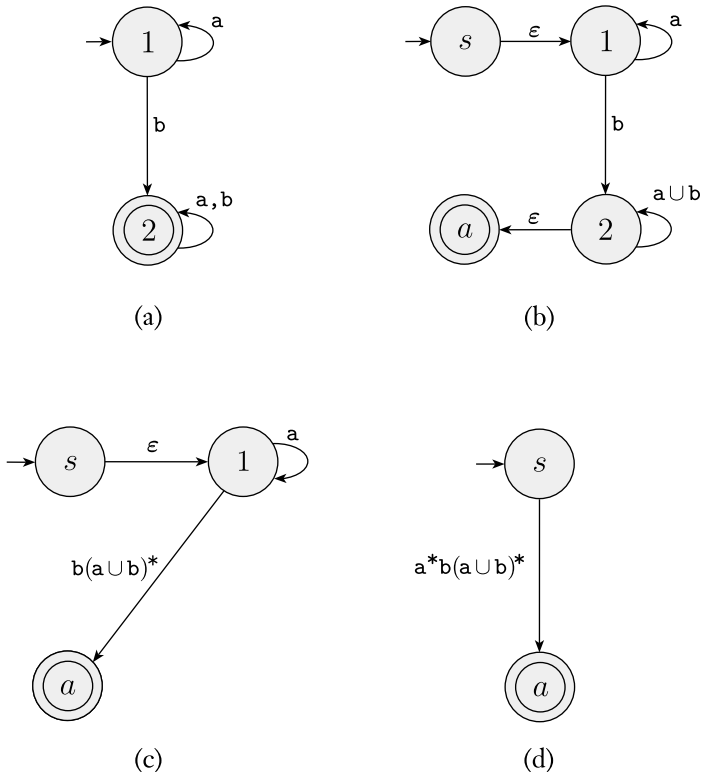


FIGURE 1.67
Converting a two-state DFA to an equivalent regular expression

EXAMPLE 1.68

In this example, we begin with a three-state DFA. The steps in the conversion are shown in the following figure.

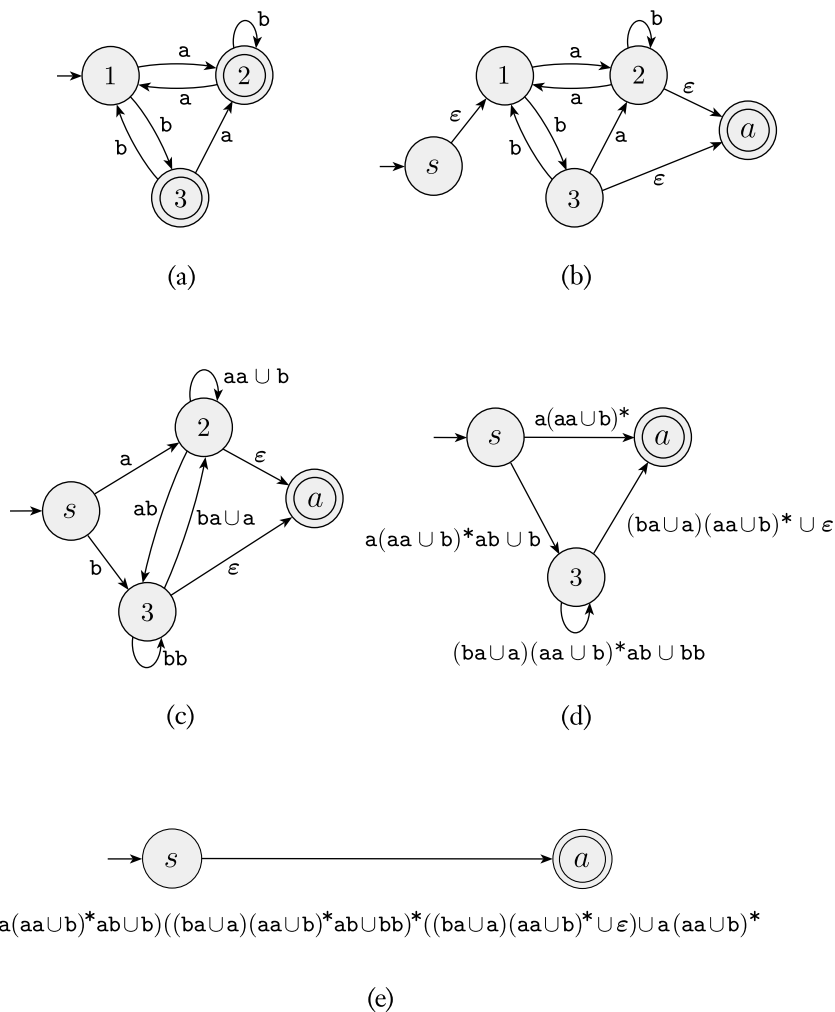


FIGURE 1.69

Converting a three-state DFA to an equivalent regular expression



1.4

NONREGULAR LANGUAGES

To understand the power of finite automata, you must also understand their limitations. In this section, we show how to prove that certain languages cannot be recognized by any finite automaton.

Let's take the language $B = \{0^n 1^n \mid n \geq 0\}$. If we attempt to find a DFA that recognizes B , we discover that the machine seems to need to remember how many 0s have been seen so far as it reads the input. Because the number of 0s isn't limited, the machine will have to keep track of an unlimited number of possibilities. But it cannot do so with any finite number of states.

Next, we present a method for proving that languages such as B are not regular. Doesn't the argument already given prove nonregularity because the number of 0s is unlimited? It does not. Just because the language appears to require unbounded memory doesn't mean that it is necessarily so. It does happen to be true for the language B ; but other languages seem to require an unlimited number of possibilities, yet actually they are regular. For example, consider two languages over the alphabet $\Sigma = \{0, 1\}$:

$C = \{w \mid w \text{ has an equal number of 0s and 1s}\}$, and

$D = \{w \mid w \text{ has an equal number of occurrences of 01 and 10 as substrings}\}$.

At first glance, a recognizing machine appears to need to count in each case, and therefore neither language appears to be regular. As expected, C is not regular, but surprisingly D is regular!⁶ Thus our intuition can sometimes lead us astray, which is why we need mathematical proofs for certainty. In this section, we show how to prove that certain languages are not regular.

THE PUMPING LEMMA FOR REGULAR LANGUAGES

Our technique for proving nonregularity stems from a theorem about regular languages, traditionally called the *pumping lemma*. This theorem states that all regular languages have a special property. If we can show that a language does not have this property, we are guaranteed that it is not regular. The property states that all strings in the language can be "pumped" if they are at least as long as a certain special value, called the *pumping length*. That means each such string contains a section that can be repeated any number of times with the resulting string remaining in the language.

⁶See Problem 1.48.

THEOREM 1.70

Pumping lemma If A is a regular language, then there is a number p (the pumping length) where if s is any string in A of length at least p , then s may be divided into three pieces, $s = xyz$, satisfying the following conditions:

1. for each $i \geq 0$, $xy^iz \in A$,
2. $|y| > 0$, and
3. $|xy| \leq p$.

Recall the notation where $|s|$ represents the length of string s , y^i means that i copies of y are concatenated together, and y^0 equals ϵ .

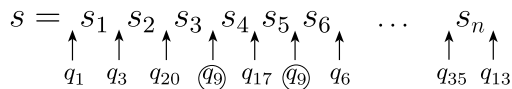
When s is divided into xyz , either x or z may be ϵ , but condition 2 says that $y \neq \epsilon$. Observe that without condition 2 the theorem would be trivially true. Condition 3 states that the pieces x and y together have length at most p . It is an extra technical condition that we occasionally find useful when proving certain languages to be nonregular. See Example 1.74 for an application of condition 3.

PROOF IDEA Let $M = (Q, \Sigma, \delta, q_1, F)$ be a DFA that recognizes A . We assign the pumping length p to be the number of states of M . We show that any string s in A of length at least p may be broken into the three pieces xyz , satisfying our three conditions. What if no strings in A are of length at least p ? Then our task is even easier because the theorem becomes *vacuously* true: Obviously the three conditions hold for all strings of length at least p if there aren't any such strings.

If s in A has length at least p , consider the sequence of states that M goes through when computing with input s . It starts with q_1 the start state, then goes to, say, q_3 , then, say, q_{20} , then q_9 , and so on, until it reaches the end of s in state q_{13} . With s in A , we know that M accepts s , so q_{13} is an accept state.

If we let n be the length of s , the sequence of states $q_1, q_3, q_{20}, q_9, \dots, q_{13}$ has length $n + 1$. Because n is at least p , we know that $n + 1$ is greater than p , the number of states of M . Therefore, the sequence must contain a repeated state. This result is an example of the **pigeonhole principle**, a fancy name for the rather obvious fact that if p pigeons are placed into fewer than p holes, some hole has to have more than one pigeon in it.

The following figure shows the string s and the sequence of states that M goes through when processing s . State q_9 is the one that repeats.

**FIGURE 1.71**

Example showing state q_9 repeating when M reads s

We now divide s into the three pieces x , y , and z . Piece x is the part of s appearing before q_9 , piece y is the part between the two appearances of q_9 , and

piece z is the remaining part of s , coming after the second occurrence of q_9 . So x takes M from the state q_1 to q_9 , y takes M from q_9 back to q_9 , and z takes M from q_9 to the accept state q_{13} , as shown in the following figure.

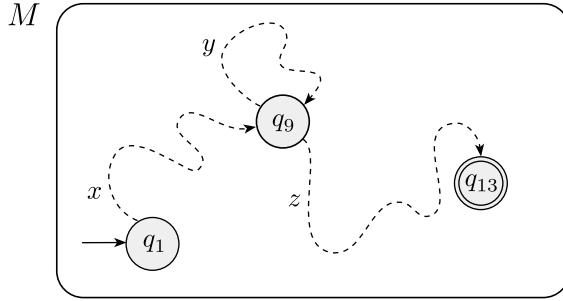


FIGURE 1.72
Example showing how the strings x , y , and z affect M

Let's see why this division of s satisfies the three conditions. Suppose that we run M on input $xyyz$. We know that x takes M from q_1 to q_9 , and then the first y takes it from q_9 back to q_9 , as does the second y , and then z takes it to q_{13} . With q_{13} being an accept state, M accepts input $xyyz$. Similarly, it will accept $xy^i z$ for any $i > 0$. For the case $i = 0$, $xy^i z = xz$, which is accepted for similar reasons. That establishes condition 1.

Checking condition 2, we see that $|y| > 0$, as it was the part of s that occurred between two different occurrences of state q_9 .

In order to get condition 3, we make sure that q_9 is the first repetition in the sequence. By the pigeonhole principle, the first $p + 1$ states in the sequence must contain a repetition. Therefore, $|xy| \leq p$.

PROOF Let $M = (Q, \Sigma, \delta, q_1, F)$ be a DFA recognizing A and p be the number of states of M .

Let $s = s_1 s_2 \cdots s_n$ be a string in A of length n , where $n \geq p$. Let $r_1, \dots, r_{n+1}$ be the sequence of states that M enters while processing s , so $r_{i+1} = \delta(r_i, s_i)$ for $1 \leq i \leq n$. This sequence has length $n + 1$, which is at least $p + 1$. Among the first $p + 1$ elements in the sequence, two must be the same state, by the pigeonhole principle. We call the first of these r_j and the second r_l . Because r_l occurs among the first $p + 1$ places in a sequence starting at r_1 , we have $l \leq p + 1$. Now let $x = s_1 \cdots s_{j-1}$, $y = s_j \cdots s_{l-1}$, and $z = s_l \cdots s_n$.

As x takes M from r_1 to r_j , y takes M from r_j to r_j , and z takes M from r_j to r_{n+1} , which is an accept state, M must accept $xy^i z$ for $i \geq 0$. We know that $j \neq l$, so $|y| > 0$; and $l \leq p + 1$, so $|xy| \leq p$. Thus we have satisfied all conditions of the pumping lemma.

To use the pumping lemma to prove that a language B is not regular, first assume that B is regular in order to obtain a contradiction. Then use the pumping lemma to guarantee the existence of a pumping length p such that all strings of length p or greater in B can be pumped. Next, find a string s in B that has length p or greater but that cannot be pumped. Finally, demonstrate that s cannot be pumped by considering all ways of dividing s into x , y , and z (taking condition 3 of the pumping lemma into account if convenient) and, for each such division, finding a value i where $xy^iz \notin B$. This final step often involves grouping the various ways of dividing s into several cases and analyzing them individually. The existence of s contradicts the pumping lemma if B were regular. Hence B cannot be regular.

Finding s sometimes takes a bit of creative thinking. You may need to hunt through several candidates for s before you discover one that works. Try members of B that seem to exhibit the “essence” of B ’s nonregularity. We further discuss the task of finding s in some of the following examples.

EXAMPLE 1.73

Let B be the language $\{0^n 1^n \mid n \geq 0\}$. We use the pumping lemma to prove that B is not regular. The proof is by contradiction.

Assume to the contrary that B is regular. Let p be the pumping length given by the pumping lemma. Choose s to be the string $0^p 1^p$. Because s is a member of B and s has length more than p , the pumping lemma guarantees that s can be split into three pieces, $s = xyz$, where for any $i \geq 0$ the string xy^iz is in B . We consider three cases to show that this result is impossible.

1. The string y consists only of 0s. In this case, the string $xyyz$ has more 0s than 1s and so is not a member of B , violating condition 1 of the pumping lemma. This case is a contradiction.
2. The string y consists only of 1s. This case also gives a contradiction.
3. The string y consists of both 0s and 1s. In this case, the string $xyyz$ may have the same number of 0s and 1s, but they will be out of order with some 1s before 0s. Hence it is not a member of B , which is a contradiction.

Thus a contradiction is unavoidable if we make the assumption that B is regular, so B is not regular. Note that we can simplify this argument by applying condition 3 of the pumping lemma to eliminate cases 2 and 3.

In this example, finding the string s was easy because any string in B of length p or more would work. In the next two examples, some choices for s do not work so additional care is required. ■

EXAMPLE 1.74

Let $C = \{w \mid w \text{ has an equal number of 0s and 1s}\}$. We use the pumping lemma to prove that C is not regular. The proof is by contradiction.

Assume to the contrary that C is regular. Let p be the pumping length given by the pumping lemma. As in Example 1.73, let s be the string $0^p 1^p$. With s being a member of C and having length more than p , the pumping lemma guarantees that s can be split into three pieces, $s = xyz$, where for any $i \geq 0$ the string $xy^i z$ is in C . We would like to show that this outcome is impossible. But wait, it *is* possible! If we let x and z be the empty string and y be the string $0^p 1^p$, then $xy^i z$ always has an equal number of 0s and 1s and hence is in C . So it *seems* that s can be pumped.

Here condition 3 in the pumping lemma is useful. It stipulates that when pumping s , it must be divided so that $|xy| \leq p$. That restriction on the way that s may be divided makes it easier to show that the string $s = 0^p 1^p$ we selected cannot be pumped. If $|xy| \leq p$, then y must consist only of 0s, so $xyyz \notin C$. Therefore, s cannot be pumped. That gives us the desired contradiction.

Selecting the string s in this example required more care than in Example 1.73. If we had chosen $s = (01)^p$ instead, we would have run into trouble because we need a string that *cannot* be pumped and that string *can* be pumped, even taking condition 3 into account. Can you see how to pump it? One way to do so sets $x = \varepsilon$, $y = 01$, and $z = (01)^{p-1}$. Then $xy^i z \in C$ for every value of i . If you fail on your first attempt to find a string that cannot be pumped, don't despair. Try another one!

An alternative method of proving that C is nonregular follows from our knowledge that B is nonregular. If C were regular, $C \cap 0^* 1^*$ also would be regular. The reasons are that the language $0^* 1^*$ is regular and that the class of regular languages is closed under intersection, which we proved in footnote 3 (page 46). But $C \cap 0^* 1^*$ equals B , and we know that B is nonregular from Example 1.73. ■

EXAMPLE 1.75

Let $F = \{ww \mid w \in \{0,1\}^*\}$. We show that F is nonregular, using the pumping lemma.

Assume to the contrary that F is regular. Let p be the pumping length given by the pumping lemma. Let s be the string $0^p 10^p 1$. Because s is a member of F and s has length more than p , the pumping lemma guarantees that s can be split into three pieces, $s = xyz$, satisfying the three conditions of the lemma. We show that this outcome is impossible.

Condition 3 is once again crucial because without it we could pump s if we let x and z be the empty string. With condition 3 the proof follows because y must consist only of 0s, so $xyyz \notin F$.

Observe that we chose $s = 0^p 10^p 1$ to be a string that exhibits the “essence” of the nonregularity of F , as opposed to, say, the string $0^p 0^p$. Even though $0^p 0^p$ is a member of F , it fails to demonstrate a contradiction because it can be pumped. ■

EXAMPLE 1.76

Here we demonstrate a nonregular unary language. Let $D = \{1^{n^2} \mid n \geq 0\}$. In other words, D contains all strings of 1s whose length is a perfect square. We use the pumping lemma to prove that D is not regular. The proof is by contradiction.

Assume to the contrary that D is regular. Let p be the pumping length given by the pumping lemma. Let s be the string 1^{p^2} . Because s is a member of D and s has length at least p , the pumping lemma guarantees that s can be split into three pieces, $s = xyz$, where for any $i \geq 0$ the string xy^iz is in D . As in the preceding examples, we show that this outcome is impossible. Doing so in this case requires a little thought about the sequence of perfect squares:

$$0, 1, 4, 9, 16, 25, 36, 49, \dots$$

Note the growing gap between successive members of this sequence. Large members of this sequence cannot be near each other.

Now consider the two strings xyz and xy^2z . These strings differ from each other by a single repetition of y , and consequently their lengths differ by the length of y . By condition 3 of the pumping lemma, $|xy| \leq p$ and thus $|y| \leq p$. We have $|xyz| = p^2$ and so $|xy^2z| \leq p^2 + p$. But $p^2 + p < p^2 + 2p + 1 = (p+1)^2$. Moreover, condition 2 implies that y is not the empty string and so $|xy^2z| > p^2$. Therefore, the length of xy^2z lies strictly between the consecutive perfect squares p^2 and $(p+1)^2$. Hence this length cannot be a perfect square itself. So we arrive at the contradiction $xy^2z \notin D$ and conclude that D is not regular. ■

EXAMPLE 1.77

Sometimes “pumping down” is useful when we apply the pumping lemma. We use the pumping lemma to show that $E = \{0^i1^j \mid i > j\}$ is not regular. The proof is by contradiction.

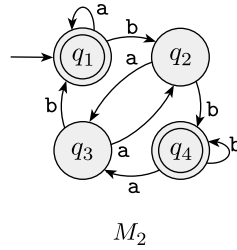
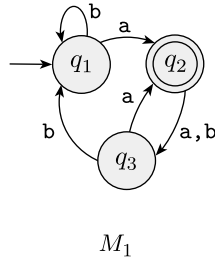
Assume that E is regular. Let p be the pumping length for E given by the pumping lemma. Let $s = 0^{p+1}1^p$. Then s can be split into xyz , satisfying the conditions of the pumping lemma. By condition 3, y consists only of 0s. Let's examine the string $xyyz$ to see whether it can be in E . Adding an extra copy of y increases the number of 0s. But, E contains all strings in 0^*1^* that have more 0s than 1s, so increasing the number of 0s will still give a string in E . No contradiction occurs. We need to try something else.

The pumping lemma states that $xy^iz \in E$ even when $i = 0$, so let's consider the string $xy^0z = xz$. Removing string y decreases the number of 0s in s . Recall that s has just one more 0 than 1. Therefore, xz cannot have more 0s than 1s, so it cannot be a member of E . Thus we obtain a contradiction. ■



EXERCISES

- ^A1.1 The following are the state diagrams of two DFAs, M_1 and M_2 . Answer the following questions about each of these machines.



- a. What is the start state?
 - b. What is the set of accept states?
 - c. What sequence of states does the machine go through on input aabb?
 - d. Does the machine accept the string aabb?
 - e. Does the machine accept the string ϵ ?
- ^A1.2 Give the formal description of the machines M_1 and M_2 pictured in Exercise 1.1.
- 1.3 The formal description of a DFA M is $(\{q_1, q_2, q_3, q_4, q_5\}, \{u, d\}, \delta, q_3, \{q_3\})$, where δ is given by the following table. Give the state diagram of this machine.

	u	d
q_1	q_1	q_2
q_2	q_1	q_3
q_3	q_2	q_4
q_4	q_3	q_5
q_5	q_4	q_5

- 1.4 Each of the following languages is the intersection of two simpler languages. In each part, construct DFAs for the simpler languages, then combine them using the construction discussed in footnote 3 (page 46) to give the state diagram of a DFA for the language given. In all parts, $\Sigma = \{a, b\}$.
- a. $\{w \mid w \text{ has at least three a's and at least two b's}\}$
 - ^Ab. $\{w \mid w \text{ has exactly two a's and at least two b's}\}$
 - c. $\{w \mid w \text{ has an even number of a's and one or two b's}\}$
 - ^Ad. $\{w \mid w \text{ has an even number of a's and each a is followed by at least one b}\}$
 - e. $\{w \mid w \text{ starts with an a and has at most one b}\}$
 - f. $\{w \mid w \text{ has an odd number of a's and ends with a b}\}$
 - g. $\{w \mid w \text{ has even length and an odd number of a's}\}$

1.5 Each of the following languages is the complement of a simpler language. In each part, construct a DFA for the simpler language, then use it to give the state diagram of a DFA for the language given. In all parts, $\Sigma = \{a, b\}$.

- ^Aa. $\{w \mid w \text{ does not contain the substring } ab\}$
- ^Ab. $\{w \mid w \text{ does not contain the substring } baba\}$
 - c. $\{w \mid w \text{ contains neither the substrings } ab \text{ nor } ba\}$
 - d. $\{w \mid w \text{ is any string not in } a^*b^*\}$
 - e. $\{w \mid w \text{ is any string not in } (ab^+)^*\}$
 - f. $\{w \mid w \text{ is any string not in } a^* \cup b^*\}$
 - g. $\{w \mid w \text{ is any string that doesn't contain exactly two a's}\}$
 - h. $\{w \mid w \text{ is any string except } a \text{ and } b\}$

1.6 Give state diagrams of DFAs recognizing the following languages. In all parts, the alphabet is $\{0, 1\}$.

- a. $\{w \mid w \text{ begins with a 1 and ends with a 0}\}$
- b. $\{w \mid w \text{ contains at least three 1s}\}$
- c. $\{w \mid w \text{ contains the substring } 0101 \text{ (i.e., } w = x0101y \text{ for some } x \text{ and } y)\}$
- d. $\{w \mid w \text{ has length at least 3 and its third symbol is a 0}\}$
- e. $\{w \mid w \text{ starts with 0 and has odd length, or starts with 1 and has even length}\}$
- f. $\{w \mid w \text{ doesn't contain the substring } 110\}$
- g. $\{w \mid \text{the length of } w \text{ is at most 5}\}$
- h. $\{w \mid w \text{ is any string except } 11 \text{ and } 111\}$
- i. $\{w \mid \text{every odd position of } w \text{ is a 1}\}$
- j. $\{w \mid w \text{ contains at least two 0s and at most one 1}\}$
- k. $\{\epsilon, 0\}$
 - l. $\{w \mid w \text{ contains an even number of 0s, or contains exactly two 1s}\}$
- m. The empty set
- n. All strings except the empty string

1.7 Give state diagrams of NFAs with the specified number of states recognizing each of the following languages. In all parts, the alphabet is $\{0, 1\}$.

- ^Aa. The language $\{w \mid w \text{ ends with } 00\}$ with three states
- b. The language of Exercise 1.6c with five states
- c. The language of Exercise 1.6l with six states
- d. The language $\{0\}$ with two states
- e. The language $0^*1^*0^+$ with three states
- ^Af. The language $1^*(001^+)^*$ with three states
- g. The language $\{\epsilon\}$ with one state
- h. The language 0^* with one state

1.8 Use the construction in the proof of Theorem 1.45 to give the state diagrams of NFAs recognizing the union of the languages described in

- a. Exercises 1.6a and 1.6b.
- b. Exercises 1.6c and 1.6f.

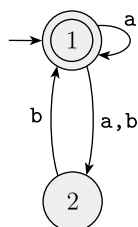
- 1.9 Use the construction in the proof of Theorem 1.47 to give the state diagrams of NFAs recognizing the concatenation of the languages described in
- Exercises 1.6g and 1.6i.
 - Exercises 1.6b and 1.6m.
- 1.10 Use the construction in the proof of Theorem 1.49 to give the state diagrams of NFAs recognizing the star of the languages described in
- Exercise 1.6b.
 - Exercise 1.6j.
 - Exercise 1.6m.
- ^A1.11 Prove that every NFA can be converted to an equivalent one that has a single accept state.
- 1.12 Let $D = \{w \mid w \text{ contains an even number of a's and an odd number of b's and does not contain the substring } ab\}$. Give a DFA with five states that recognizes D and a regular expression that generates D . (Suggestion: Describe D more simply.)
- 1.13 Let F be the language of all strings over $\{0,1\}$ that do not contain a pair of 1s that are separated by an odd number of symbols. Give the state diagram of a DFA with five states that recognizes F . (You may find it helpful first to find a 4-state NFA for the complement of F .)
- 1.14
- Show that if M is a DFA that recognizes language B , swapping the accept and nonaccept states in M yields a new DFA recognizing the complement of B . Conclude that the class of regular languages is closed under complement.
 - Show by giving an example that if M is an NFA that recognizes language C , swapping the accept and nonaccept states in M doesn't necessarily yield a new NFA that recognizes the complement of C . Is the class of languages recognized by NFAs closed under complement? Explain your answer.
- 1.15 Give a counterexample to show that the following construction fails to prove Theorem 1.49, the closure of the class of regular languages under the star operation.⁷ Let $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ recognize A_1 . Construct $N = (Q_1, \Sigma, \delta, q_1, F)$ as follows. N is supposed to recognize A_1^* .
- The states of N are the states of N_1 .
 - The start state of N is the same as the start state of N_1 .
 - $F = \{q_1\} \cup F_1$.
The accept states F are the old accept states plus its start state.
 - Define δ so that for any $q \in Q_1$ and any $a \in \Sigma_\epsilon$,

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \notin F_1 \text{ or } a \neq \epsilon \\ \delta_1(q, a) \cup \{q_1\} & q \in F_1 \text{ and } a = \epsilon. \end{cases}$$

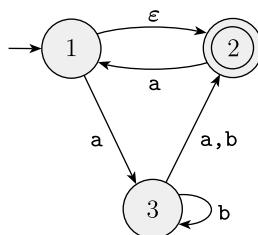
(Suggestion: Show this construction graphically, as in Figure 1.50.)

⁷In other words, you must present a finite automaton, N_1 , for which the constructed automaton N does not recognize the star of N_1 's language.

- 1.16 Use the construction given in Theorem 1.39 to convert the following two nondeterministic finite automata to equivalent deterministic finite automata.

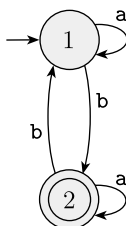


(a)

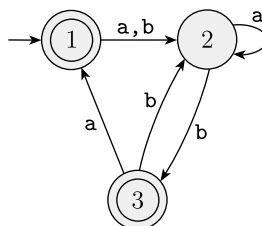


(b)

- 1.17 a. Give an NFA recognizing the language $(01 \cup 001 \cup 010)^*$.
 b. Convert this NFA to an equivalent DFA. Give only the portion of the DFA that is reachable from the start state.
- 1.18 Give regular expressions generating the languages of Exercise 1.6.
- 1.19 Use the procedure described in Lemma 1.55 to convert the following regular expressions to nondeterministic finite automata.
- $(0 \cup 1)^* 000(0 \cup 1)^*$
 - $((00)^*(11) \cup 01)^*$
 - $\emptyset^*$
- 1.20 For each of the following languages, give two strings that are members and two strings that are *not* members—a total of four strings for each part. Assume the alphabet $\Sigma = \{a, b\}$ in all parts.
- a^*b^*
 - $a(ba)^*b$
 - $a^* \cup b^*$
 - $(aaa)^*$
 - $\Sigma^*a\Sigma^*b\Sigma^*a\Sigma^*$
 - $aba \cup bab$
 - $(\varepsilon \cup a)b$
 - $(a \cup ba \cup bb)\Sigma^*$
- 1.21 Use the procedure described in Lemma 1.60 to convert the following finite automata to regular expressions.

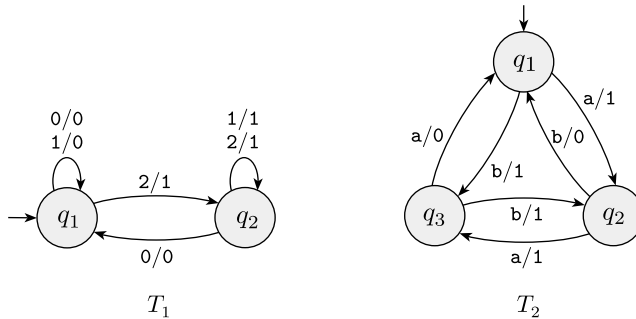


(a)



(b)

- 1.22 In certain programming languages, comments appear between delimiters such as `/#` and `#/`. Let C be the language of all valid delimited comment strings. A member of C must begin with `/#` and end with `#/` but have no intervening `#/`. For simplicity, assume that the alphabet for C is $\Sigma = \{a, b, /, \#\}$.
- Give a DFA that recognizes C .
 - Give a regular expression that generates C .
- ^A1.23 Let B be any language over the alphabet Σ . Prove that $B = B^*$ iff $BB \subseteq B$.
- 1.24 A **finite state transducer** (FST) is a type of deterministic finite automaton whose output is a string and not just *accept* or *reject*. The following are state diagrams of finite state transducers T_1 and T_2 .



Each transition of an FST is labeled with two symbols, one designating the input symbol for that transition and the other designating the output symbol. The two symbols are written with a slash, $/$, separating them. In T_1 , the transition from q_1 to q_2 has input symbol 2 and output symbol 1. Some transitions may have multiple input–output pairs, such as the transition in T_1 from q_1 to itself. When an FST computes on an input string w , it takes the input symbols $w_1 \cdots w_n$ one by one and, starting at the start state, follows the transitions by matching the input labels with the sequence of symbols $w_1 \cdots w_n = w$. Every time it goes along a transition, it outputs the corresponding output symbol. For example, on input 2212011, machine T_1 enters the sequence of states $q_1, q_2, q_2, q_2, q_2, q_1, q_1, q_1$ and produces output 1111000. On input abbb, T_2 outputs 1011. Give the sequence of states entered and the output produced in each of the following parts.

- T_1 on input 011
 - T_1 on input 211
 - T_1 on input 121
 - T_1 on input 0202
 - T_2 on input b
 - T_2 on input bbab
 - T_2 on input bbbbbb
 - T_2 on input ϵ
- 1.25 Read the informal definition of the finite state transducer given in Exercise 1.24. Give a formal definition of this model, following the pattern in Definition 1.5 (page 35). Assume that an FST has an input alphabet Σ and an output alphabet Γ but not a set of accept states. Include a formal definition of the computation of an FST. (Hint: An FST is a 5-tuple. Its transition function is of the form $\delta: Q \times \Sigma \rightarrow Q \times \Gamma$.)
- 1.26 Using the solution you gave to Exercise 1.25, give a formal description of the machines T_1 and T_2 depicted in Exercise 1.24.

- 1.27 Read the informal definition of the finite state transducer given in Exercise 1.24. Give the state diagram of an FST with the following behavior. Its input and output alphabets are $\{0,1\}$. Its output string is identical to the input string on the even positions but inverted on the odd positions. For example, on input 0000111 it should output 1010010.
- 1.28 Convert the following regular expressions to NFAs using the procedure given in Theorem 1.54. In all parts, $\Sigma = \{a, b\}$.
- $a(abb)^* \cup b$
 - $a^+ \cup (ab)^+$
 - $(a \cup b^+)a^+b^+$
- 1.29 Use the pumping lemma to show that the following languages are not regular.
- $A_1 = \{0^n 1^n 2^n \mid n \geq 0\}$
 - $A_2 = \{www \mid w \in \{a, b\}^*\}$
 - $A_3 = \{a^{2^n} \mid n \geq 0\}$ (Here, a^{2^n} means a string of 2^n a's.)
- 1.30 Describe the error in the following “proof” that 0^*1^* is not a regular language. (An error must exist because 0^*1^* is regular.) The proof is by contradiction. Assume that 0^*1^* is regular. Let p be the pumping length for 0^*1^* given by the pumping lemma. Choose s to be the string $0^p 1^p$. You know that s is a member of 0^*1^* , but Example 1.73 shows that s cannot be pumped. Thus you have a contradiction. So 0^*1^* is not regular.

PROBLEMS

- 1.31 For any string $w = w_1 w_2 \cdots w_n$, the **reverse** of w , written $w^{\mathcal{R}}$, is the string w in reverse order, $w_n \cdots w_2 w_1$. For any language A , let $A^{\mathcal{R}} = \{w^{\mathcal{R}} \mid w \in A\}$. Show that if A is regular, so is $A^{\mathcal{R}}$.
- 1.32 Let

$$\Sigma_3 = \left\{ \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \dots, \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \right\}.$$

Σ_3 contains all size 3 columns of 0s and 1s. A string of symbols in Σ_3 gives three rows of 0s and 1s. Consider each row to be a binary number and let

$$B = \{w \in \Sigma_3^* \mid \text{the bottom row of } w \text{ is the sum of the top two rows}\}.$$

For example,

$$\begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} \in B, \quad \text{but} \quad \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} \notin B.$$

Show that B is regular. (Hint: Working with $B^{\mathcal{R}}$ is easier. You may assume the result claimed in Problem 1.31.)

1.33 Let

$$\Sigma_2 = \left\{ \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right\}.$$

Here, Σ_2 contains all columns of 0s and 1s of height two. A string of symbols in Σ_2 gives two rows of 0s and 1s. Consider each row to be a binary number and let

$$C = \{w \in \Sigma_2^* \mid \text{the bottom row of } w \text{ is three times the top row}\}.$$

For example, $\begin{bmatrix} 0 \\ 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} \in C$, but $\begin{bmatrix} 0 \\ 1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} \notin C$. Show that C is regular. (You may assume the result claimed in Problem 1.31.)

1.34 Let Σ_2 be the same as in Problem 1.33. Consider each row to be a binary number and let

$$D = \{w \in \Sigma_2^* \mid \text{the top row of } w \text{ is a larger number than is the bottom row}\}.$$

For example, $\begin{bmatrix} 0 \\ 0 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} \in D$, but $\begin{bmatrix} 0 \\ 0 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} \notin D$. Show that D is regular.

1.35 Let Σ_2 be the same as in Problem 1.33. Consider the top and bottom rows to be strings of 0s and 1s, and let

$$E = \{w \in \Sigma_2^* \mid \text{the bottom row of } w \text{ is the reverse of the top row of } w\}.$$

Show that E is not regular.

1.36 Let $B_n = \{a^k \mid k \text{ is a multiple of } n\}$. Show that for each $n \geq 1$, the language B_n is regular.

1.37 Let $C_n = \{x \mid x \text{ is a binary number that is a multiple of } n\}$. Show that for each $n \geq 1$, the language C_n is regular.

1.38 An *all-NFA* M is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ that accepts $x \in \Sigma^*$ if *every* possible state that M could be in after reading input x is a state from F . Note, in contrast, that an ordinary NFA accepts a string if *some* state among these possible states is an accept state. Prove that all-NFAs recognize the class of regular languages.

1.39 The construction in Theorem 1.54 shows that every GNFA is equivalent to a GNFA with only two states. We can show that an opposite phenomenon occurs for DFAs. Prove that for every $k > 1$, a language $A_k \subseteq \{0,1\}^*$ exists that is recognized by a DFA with k states but not by one with only $k - 1$ states.

1.40 Recall that string x is a *prefix* of string y if a string z exists where $xz = y$, and that x is a *proper prefix* of y if in addition $x \neq y$. In each of the following parts, we define an operation on a language A . Show that the class of regular languages is closed under that operation.

a. $\text{NOPREFIX}(A) = \{w \in A \mid \text{no proper prefix of } w \text{ is a member of } A\}$.

b. $\text{NOEXTEND}(A) = \{w \in A \mid w \text{ is not the proper prefix of any string in } A\}$.

1.41 For languages A and B , let the *perfect shuffle* of A and B be the language

$$\{w \mid w = a_1 b_1 \cdots a_k b_k, \text{ where } a_1 \cdots a_k \in A \text{ and } b_1 \cdots b_k \in B, \text{ each } a_i, b_i \in \Sigma\}.$$

Show that the class of regular languages is closed under perfect shuffle.

1.42 For languages A and B , let the *shuffle* of A and B be the language

$$\{w \mid w = a_1 b_1 \cdots a_k b_k, \text{ where } a_1 \cdots a_k \in A \text{ and } b_1 \cdots b_k \in B, \text{ each } a_i, b_i \in \Sigma^*\}.$$

Show that the class of regular languages is closed under shuffle.

1.43 Let A be any language. Define $DROP-OUT(A)$ to be the language containing all strings that can be obtained by removing one symbol from a string in A . Thus, $DROP-OUT(A) = \{xz \mid xyz \in A \text{ where } x, z \in \Sigma^*, y \in \Sigma\}$. Show that the class of regular languages is closed under the $DROP-OUT$ operation. Give both a proof by picture and a more formal proof by construction as in Theorem 1.47.

^A**1.44** Let B and C be languages over $\Sigma = \{0, 1\}$. Define

$$B \stackrel{1}{\leftarrow} C = \{w \in B \mid \text{for some } y \in C, \text{ strings } w \text{ and } y \text{ contain equal numbers of 1s}\}.$$

Show that the class of regular languages is closed under the $\stackrel{1}{\leftarrow}$ operation.

***1.45** Let $A/B = \{w \mid wx \in A \text{ for some } x \in B\}$. Show that if A is regular and B is any language, then A/B is regular.

1.46 Prove that the following languages are not regular. You may use the pumping lemma and the closure of the class of regular languages under union, intersection, and complement.

- a. $\{0^n 1^m 0^n \mid m, n \geq 0\}$
- ^Ab. $\{0^m 1^n \mid m \neq n\}$
- c. $\{w \mid w \in \{0, 1\}^* \text{ is not a palindrome}\}^8$
- ^{*}d. $\{wtw \mid w, t \in \{0, 1\}^+\}$

1.47 Let $\Sigma = \{1, \#\}$ and let

$$Y = \{w \mid w = x_1 \# x_2 \# \cdots \# x_k \text{ for } k \geq 0, \text{ each } x_i \in 1^*, \text{ and } x_i \neq x_j \text{ for } i \neq j\}.$$

Prove that Y is not regular.

1.48 Let $\Sigma = \{0, 1\}$ and let

$$D = \{w \mid w \text{ contains an equal number of occurrences of the substrings } 01 \text{ and } 10\}.$$

Thus $101 \in D$ because 101 contains a single 01 and a single 10 , but $1010 \notin D$ because 1010 contains two 10 s and one 01 . Show that D is a regular language.

- 1.49**
- a. Let $B = \{1^k y \mid y \in \{0, 1\}^* \text{ and } y \text{ contains at least } k \text{ 1s, for } k \geq 1\}$. Show that B is a regular language.
 - b. Let $C = \{1^k y \mid y \in \{0, 1\}^* \text{ and } y \text{ contains at most } k \text{ 1s, for } k \geq 1\}$. Show that C isn't a regular language.

^A**1.50** Read the informal definition of the finite state transducer given in Exercise 1.24. Prove that no FST can output w^R for every input w if the input and output alphabets are $\{0, 1\}$.

1.51 Let x and y be strings and let L be any language. We say that x and y are **distinguishable by L** if some string z exists whereby exactly one of the strings xz and yz is a member of L ; otherwise, for every string z , we have $xz \in L$ whenever $yz \in L$ and we say that x and y are **indistinguishable by L** . If x and y are indistinguishable by L , we write $x \equiv_L y$. Show that $\equiv_L$ is an equivalence relation.

⁸A **palindrome** is a string that reads the same forward and backward.

^{A*}1.52 **Myhill–Nerode theorem.** Refer to Problem 1.51. Let L be a language and let X be a set of strings. Say that X is *pairwise distinguishable by L* if every two distinct strings in X are distinguishable by L . Define the *index of L* to be the maximum number of elements in any set that is pairwise distinguishable by L . The index of L may be finite or infinite.

- Show that if L is recognized by a DFA with k states, L has index at most k .
- Show that if the index of L is a finite number k , it is recognized by a DFA with k states.
- Conclude that L is regular iff it has finite index. Moreover, its index is the size of the smallest DFA recognizing it.

1.53 Let $\Sigma = \{0, 1, +, =\}$ and

$$ADD = \{x=y+z \mid x, y, z \text{ are binary integers, and } x \text{ is the sum of } y \text{ and } z\}.$$

Show that ADD is not regular.

1.54 Consider the language $F = \{a^i b^j c^k \mid i, j, k \geq 0 \text{ and if } i = 1 \text{ then } j = k\}$.

- Show that F is not regular.
- Show that F acts like a regular language in the pumping lemma. In other words, give a pumping length p and demonstrate that F satisfies the three conditions of the pumping lemma for this value of p .
- Explain why parts (a) and (b) do not contradict the pumping lemma.

1.55 The pumping lemma says that every regular language has a pumping length p , such that every string in the language can be pumped if it has length p or more. If p is a pumping length for language A , so is any length $p' \geq p$. The *minimum pumping length* for A is the smallest p that is a pumping length for A . For example, if $A = 01^*$, the minimum pumping length is 2. The reason is that the string $s = 0$ is in A and has length 1 yet s cannot be pumped; but any string in A of length 2 or more contains a 1 and hence can be pumped by dividing it so that $x = 0$, $y = 1$, and z is the rest. For each of the following languages, give the minimum pumping length and justify your answer.

- | | |
|---|-------------------|
| ^A a. 0001^* | f. ϵ |
| ^A b. 0^*1^* | g. $1^*01^*01^*$ |
| c. $001 \cup 0^*1^*$ | h. $10(11^*0)^*0$ |
| ^A d. $0^*1^*0^*1^* \cup 10^*1$ | i. 1011 |
| e. $(01)^*$ | j. Σ^* |

*1.56 If A is a set of natural numbers and k is a natural number greater than 1, let

$$B_k(A) = \{w \mid w \text{ is the representation in base } k \text{ of some number in } A\}.$$

Here, we do not allow leading 0s in the representation of a number. For example, $B_2(\{3, 5\}) = \{11, 101\}$ and $B_3(\{3, 5\}) = \{10, 12\}$. Give an example of a set A for which $B_2(A)$ is regular but $B_3(A)$ is not regular. Prove that your example works.

- *1.57 If A is any language, let $A_{\frac{1}{2}-}$ be the set of all first halves of strings in A so that

$$A_{\frac{1}{2}-} = \{x \mid \text{for some } y, |x| = |y| \text{ and } xy \in A\}.$$

Show that if A is regular, then so is $A_{\frac{1}{2}-}$.

- *1.58 If A is any language, let $A_{\frac{1}{3}-\frac{1}{3}}$ be the set of all strings in A with their middle thirds removed so that

$$A_{\frac{1}{3}-\frac{1}{3}} = \{xyz \mid \text{for some } y, |x| = |y| = |z| \text{ and } xyz \in A\}.$$

Show that if A is regular, then $A_{\frac{1}{3}-\frac{1}{3}}$ is not necessarily regular.

- *1.59 Let $M = (Q, \Sigma, \delta, q_0, F)$ be a DFA and let h be a state of M called its “home”. A **synchronizing sequence** for M and h is a string $s \in \Sigma^*$ where $\delta(q, s) = h$ for every $q \in Q$. (Here we have extended δ to strings, so that $\delta(q, s)$ equals the state where M ends up when M starts at state q and reads input s .) Say that M is **synchronizable** if it has a synchronizing sequence for some state h . Prove that if M is a k -synchronizable DFA, then it has a synchronizing sequence of length at most k^3 . Can you improve upon this bound?

- 1.60 Let $\Sigma = \{a, b\}$. For each $k \geq 1$, let C_k be the language consisting of all strings that contain an a exactly k places from the right-hand end. Thus $C_k = \Sigma^* a \Sigma^{k-1}$. Describe an NFA with $k + 1$ states that recognizes C_k in terms of both a state diagram and a formal description.

- 1.61 Consider the languages C_k defined in Problem 1.60. Prove that for each k , no DFA can recognize C_k with fewer than 2^k states.

- 1.62 Let $\Sigma = \{a, b\}$. For each $k \geq 1$, let D_k be the language consisting of all strings that have at least one a among the last k symbols. Thus $D_k = \Sigma^* a (\Sigma \cup \epsilon)^{k-1}$. Describe a DFA with at most $k + 1$ states that recognizes D_k in terms of both a state diagram and a formal description.

- *1.63 a. Let A be an infinite regular language. Prove that A can be split into two infinite disjoint regular subsets.
b. Let B and D be two languages. Write $B \Subset D$ if $B \subseteq D$ and D contains infinitely many strings that are not in B . Show that if B and D are two regular languages where $B \Subset D$, then we can find a regular language C where $B \Subset C \Subset D$.

- 1.64 Let N be an NFA with k states that recognizes some language A .

- a. Show that if A is nonempty, A contains some string of length at most k .
b. Show, by giving an example, that part (a) is not necessarily true if you replace both A 's by $\bar{A}$.
c. Show that if $\bar{A}$ is nonempty, $\bar{A}$ contains some string of length at most 2^k .
d. Show that the bound given in part (c) is nearly tight; that is, for each k , demonstrate an NFA recognizing a language A_k where $\bar{A}_k$ is nonempty and where $\bar{A}_k$'s shortest member strings are of length exponential in k . Come as close to the bound in (c) as you can.

- *1.65 Prove that for each $n > 0$, a language B_n exists where
- B_n is recognizable by an NFA that has n states, and
 - if $B_n = A_1 \cup \dots \cup A_k$, for regular languages A_i , then at least one of the A_i requires a DFA with exponentially many states.
- 1.66 A **homomorphism** is a function $f: \Sigma \rightarrow \Gamma^*$ from one alphabet to strings over another alphabet. We can extend f to operate on strings by defining $f(w) = f(w_1)f(w_2)\dots f(w_n)$, where $w = w_1w_2\dots w_n$ and each $w_i \in \Sigma$. We further extend f to operate on languages by defining $f(A) = \{f(w) \mid w \in A\}$, for any language A .
- Show, by giving a formal construction, that the class of regular languages is closed under homomorphism. In other words, given a DFA M that recognizes B and a homomorphism f , construct a finite automaton M' that recognizes $f(B)$. Consider the machine M' that you constructed. Is it a DFA in every case?
 - Show, by giving an example, that the class of non-regular languages is not closed under homomorphism.
- *1.67 Let the **rotational closure** of language A be $RC(A) = \{yx \mid xy \in A\}$.
- Show that for any language A , we have $RC(A) = RC(RC(A))$.
 - Show that the class of regular languages is closed under rotational closure.
- *1.68 In the traditional method for cutting a deck of playing cards, the deck is arbitrarily split two parts, which are exchanged before reassembling the deck. In a more complex cut, called Scarne's cut, the deck is broken into three parts and the middle part is placed first in the reassembly. We'll take Scarne's cut as the inspiration for an operation on languages. For a language A , let $CUT(A) = \{y x z \mid x y z \in A\}$.
- Exhibit a language B for which $CUT(B) \neq CUT(CUT(B))$.
 - Show that the class of regular languages is closed under CUT .
- 1.69 Let $\Sigma = \{0,1\}$. Let $WW_k = \{ww \mid w \in \Sigma^* \text{ and } w \text{ is of length } k\}$.
- Show that for each k , no DFA can recognize WW_k with fewer than 2^k states.
 - Describe a much smaller NFA for $\overline{WW_k}$, the complement of WW_k .
- 1.70 We define the **avoids** operation for languages A and B to be
- $$A \text{ avoids } B = \{w \mid w \in A \text{ and } w \text{ doesn't contain any string in } B \text{ as a substring}\}.$$
- Prove that the class of regular languages is closed under the *avoids* operation.
- 1.71 Let $\Sigma = \{0,1\}$.
- Let $A = \{0^k u 0^k \mid k \geq 1 \text{ and } u \in \Sigma^*\}$. Show that A is regular.
 - Let $B = \{0^k 1 u 0^k \mid k \geq 1 \text{ and } u \in \Sigma^*\}$. Show that B is not regular.
- 1.72 Let M_1 and M_2 be DFAs that have k_1 and k_2 states, respectively, and then let $U = L(M_1) \cup L(M_2)$.
- Show that if $U \neq \emptyset$, then U contains some string s , where $|s| < \max(k_1, k_2)$.
 - Show that if $U \neq \Sigma^*$, then U excludes some string s , where $|s| < k_1 k_2$.
- 1.73 Let $\Sigma = \{0,1,\#\}$. Let $C = \{x\#\#x \mid x \in \{0,1\}^*\}$. Show that $\overline{C}$ is a CFL.

SELECTED SOLUTIONS

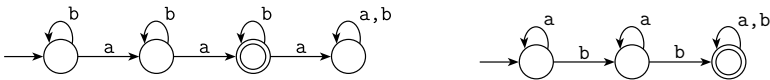
1.1 For M_1 : (a) q_1 ; (b) $\{q_2\}$; (c) q_1, q_2, q_3, q_1, q_1 ; (d) No; (e) No
For M_2 : (a) q_1 ; (b) $\{q_1, q_4\}$; (c) q_1, q_1, q_1, q_2, q_4 ; (d) Yes; (e) Yes

1.2 $M_1 = (\{q_1, q_2, q_3\}, \{a, b\}, \delta_1, q_1, \{q_2\})$.
 $M_2 = (\{q_1, q_2, q_3, q_4\}, \{a, b\}, \delta_2, q_1, \{q_1, q_4\})$.
The transition functions are

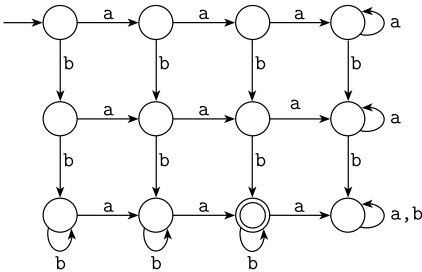
δ_1	a	b
q_1	q_2	q_1
q_2	q_3	q_3
q_3	q_2	q_1

δ_2	a	b
q_1	q_1	q_2
q_2	q_3	q_4
q_3	q_2	q_1
q_4	q_3	q_4

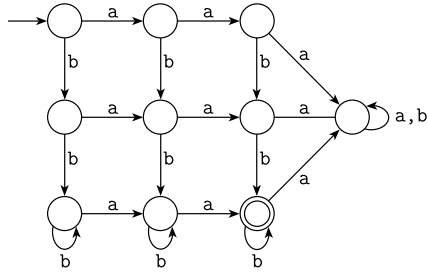
1.4 (b) The following are DFAs for the two languages $\{w \mid w \text{ has exactly two a's}\}$ and $\{w \mid w \text{ has at least two b's}\}$.



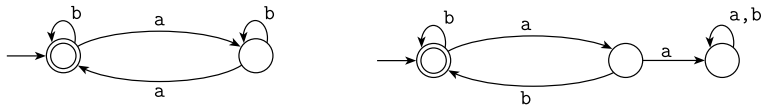
Combining them using the intersection construction gives the following DFA.



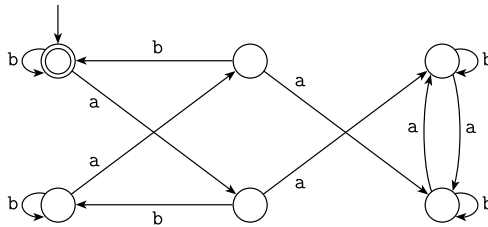
Though the problem doesn't request you to simplify the DFA, certain states can be combined to give the following DFA.



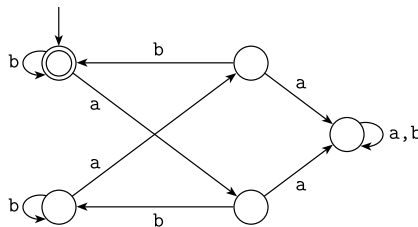
(d) These are DFAs for the two languages $\{w \mid w \text{ has an even number of a's}\}$ and $\{w \mid \text{each a in } w \text{ is followed by at least one b}\}$.



Combining them using the intersection construction gives the following DFA.



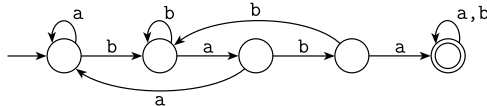
Though the problem doesn't request you to simplify the DFA, certain states can be combined to give the following DFA.



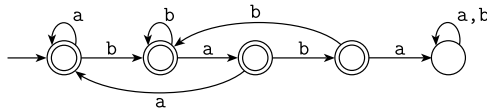
- 1.5 (a) The left-hand DFA recognizes $\{w \mid w \text{ contains } ab\}$. The right-hand DFA recognizes its complement, $\{w \mid w \text{ doesn't contain } ab\}$.



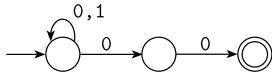
- (b) This DFA recognizes $\{w \mid w \text{ contains } baba\}$.



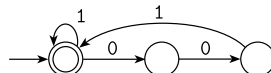
This DFA recognizes $\{w \mid w \text{ does not contain } baba\}$.



- 1.7 (a)



- (f)



- 1.11 Let $N = (Q, \Sigma, \delta, q_0, F)$ be any NFA. Construct an NFA N' with a single accept state that recognizes the same language as N . Informally, N' is exactly like N except it has ϵ -transitions from the states corresponding to the accept states of N , to a new accept state, q_{accept} . State q_{accept} has no emerging transitions. More formally, $N' = (Q \cup \{q_{\text{accept}}\}, \Sigma, \delta', q_0, \{q_{\text{accept}}\})$, where for each $q \in Q$ and $a \in \Sigma_\epsilon$

$$\delta'(q, a) = \begin{cases} \delta(q, a) & \text{if } a \neq \epsilon \text{ or } q \notin F \\ \delta(q, a) \cup \{q_{\text{accept}}\} & \text{if } a = \epsilon \text{ and } q \in F \end{cases}$$

and $\delta'(q_{\text{accept}}, a) = \emptyset$ for each $a \in \Sigma_\epsilon$.

- 1.23 We prove both directions of the “iff.”

($\rightarrow$) Assume that $B = B^+$ and show that $BB \subseteq B$.

For every language $BB \subseteq B^+$ holds, so if $B = B^+$, then $BB \subseteq B$.

($\leftarrow$) Assume that $BB \subseteq B$ and show that $B = B^+$.

For every language $B \subseteq B^+$, so we need to show only $B^+ \subseteq B$. If $w \in B^+$, then $w = x_1x_2 \cdots x_k$ where each $x_i \in B$ and $k \geq 1$. Because $x_1, x_2 \in B$ and $BB \subseteq B$, we have $x_1x_2 \in B$. Similarly, because x_1x_2 is in B and x_3 is in B , we have $x_1x_2x_3 \in B$. Continuing in this way, $x_1 \cdots x_k \in B$. Hence $w \in B$, and so we may conclude that $B^+ \subseteq B$.

The latter argument may be written formally as the following proof by induction. Assume that $BB \subseteq B$.

Claim: For each $k \geq 1$, if $x_1, \dots, x_k \in B$, then $x_1 \cdots x_k \in B$.

Basis: Prove for $k = 1$. This statement is obviously true.

Induction step: For each $k \geq 1$, assume that the claim is true for k and prove it to be true for $k + 1$.

If $x_1, \dots, x_k, x_{k+1} \in B$, then by the induction assumption, $x_1 \cdots x_k \in B$. Therefore, $x_1 \cdots x_k x_{k+1} \in BB$, but $BB \subseteq B$, so $x_1 \cdots x_{k+1} \in B$. That proves the induction step and the claim. The claim implies that if $BB \subseteq B$, then $B^+ \subseteq B$.

- 1.29 (a)** Assume that $A_1 = \{0^n 1^n 2^n \mid n \geq 0\}$ is regular. Let p be the pumping length given by the pumping lemma. Choose s to be the string $0^p 1^p 2^p$. Because s is a member of A_1 and s is longer than p , the pumping lemma guarantees that s can be split into three pieces, $s = xyz$, where for any $i \geq 0$ the string $xy^i z$ is in A_1 . Consider two possibilities:

1. The string y consists only of 0s, only of 1s, or only of 2s. In these cases, the string $xyyz$ will not have equal numbers of 0s, 1s, and 2s. Hence $xyyz$ is not a member of A_1 , a contradiction.
2. The string y consists of more than one kind of symbol. In this case, $xyyz$ will have the 0s, 1s, or 2s out of order. Hence $xyyz$ is not a member of A_1 , a contradiction.

Either way we arrive at a contradiction. Therefore, A_1 is not regular.

(c) Assume that $A_3 = \{a^{2^n} \mid n \geq 0\}$ is regular. Let p be the pumping length given by the pumping lemma. Choose s to be the string a^{2^p} . Because s is a member of A_3 and s is longer than p , the pumping lemma guarantees that s can be split into three pieces, $s = xyz$, satisfying the three conditions of the pumping lemma.

The third condition tells us that $|xy| \leq p$. Furthermore, $p < 2^p$ and so $|y| < 2^p$. Therefore, $|xyyz| = |xyz| + |y| < 2^p + 2^p = 2^{p+1}$. The second condition requires $|y| > 0$ so $2^p < |xyyz| < 2^{p+1}$. The length of $xyyz$ cannot be a power of 2. Hence $xyyz$ is not a member of A_3 , a contradiction. Therefore, A_3 is not regular.

- 1.40 (a)** Let $M = (Q, \Sigma, \delta, q_0, F)$ be a DFA recognizing A , where A is some regular language. Construct $M' = (Q', \Sigma, \delta', q_0', F')$ recognizing $NOPREFIX(A)$ as follows:

1. $Q' = Q$.
2. For $r \in Q'$ and $a \in \Sigma$, define $\delta'(r, a) = \begin{cases} \{\delta(r, a)\} & \text{if } r \notin F \\ \emptyset & \text{if } r \in F. \end{cases}$
3. $q_0' = q_0$.
4. $F' = F$.

1.44 Let $M_B = (Q_B, \Sigma, \delta_B, q_B, F_B)$ and $M_C = (Q_C, \Sigma, \delta_C, q_C, F_C)$ be DFAs recognizing B and C , respectively. Construct NFA $M = (Q, \Sigma, \delta, q_0, F)$ that recognizes $B \stackrel{1}{\leftarrow} C$ as follows. To decide whether its input w is in $B \stackrel{1}{\leftarrow} C$, the machine M checks that $w \in B$, and in parallel nondeterministically guesses a string y that contains the same number of 1s as contained in w and checks that $y \in C$.

1. $Q = Q_B \times Q_C$.
2. For $(q, r) \in Q$ and $a \in \Sigma_\epsilon$, define

$$\delta((q, r), a) = \begin{cases} \{(\delta_B(q, 0), r)\} & \text{if } a = 0 \\ \{(\delta_B(q, 1), \delta_C(r, 1))\} & \text{if } a = 1 \\ \{(q, \delta_C(r, 0))\} & \text{if } a = \epsilon. \end{cases}$$

3. $q_0 = (q_B, q_C)$.
4. $F = F_B \times F_C$.

1.46 (b) Let $B = \{0^m 1^n \mid m \neq n\}$. Observe that $\overline{B} \cap 0^* 1^* = \{0^k 1^k \mid k \geq 0\}$. If B were regular, then $\overline{B}$ would be regular and so would $\overline{B} \cap 0^* 1^*$. But we already know that $\{0^k 1^k \mid k \geq 0\}$ isn't regular, so B cannot be regular.

Alternatively, we can prove B to be nonregular by using the pumping lemma directly, though doing so is trickier. Assume that $B = \{0^m 1^n \mid m \neq n\}$ is regular. Let p be the pumping length given by the pumping lemma. Observe that $p!$ is divisible by all integers from 1 to p , where $p! = p(p-1)(p-2) \cdots 1$. The string $s = 0^{p!} 1^{p+p!} \in B$, and $|s| \geq p$. Thus the pumping lemma implies that s can be divided as xyz with $x = 0^a$, $y = 0^b$, and $z = 0^c 1^{p+p!}$, where $b \geq 1$ and $a+b+c = p$. Let s' be the string $xy^{i+1}z$, where $i = p!/b$. Then $y^i = 0^{p!}$ so $y^{i+1} = 0^{b+p!}$, and so $s' = 0^{a+b+c+p!} 1^{p+p!}$. That gives $s' = 0^{p+p!} 1^{p+p!} \notin B$, a contradiction.

1.50 Assume to the contrary that some FST T outputs w^R on input w . Consider the input strings 00 and 01. On input 00, T must output 00, and on input 01, T must output 10. In both cases, the first input bit is a 0 but the first output bits differ. Operating in this way is impossible for an FST because it produces its first output bit before it reads its second input. Hence no such FST can exist.

1.52 (a) We prove this assertion by contradiction. Let M be a k -state DFA that recognizes L . Suppose for a contradiction that L has index greater than k . That means some set X with more than k elements is pairwise distinguishable by L . Because M has k states, the pigeonhole principle implies that X contains two distinct strings x and y , where $\delta(q_0, x) = \delta(q_0, y)$. Here $\delta(q_0, x)$ is the state that M is in after starting in the start state q_0 and reading input string x . Then, for any string $z \in \Sigma^*$, $\delta(q_0, xz) = \delta(q_0, yz)$. Therefore, either both xz and yz are in L or neither are in L . But then x and y aren't distinguishable by L , contradicting our assumption that X is pairwise distinguishable by L .

(b) Let $X = \{s_1, \dots, s_k\}$ be pairwise distinguishable by L . We construct DFA $M = (Q, \Sigma, \delta, q_0, F)$ with k states recognizing L . Let $Q = \{q_1, \dots, q_k\}$, and define $\delta(q_i, a)$ to be q_j , where $s_j \equiv_L s_i a$ (the relation $\equiv_L$ is defined in Problem 1.51). Note that $s_j \equiv_L s_i a$ for some $s_j \in X$; otherwise, $X \cup s_i a$ would have $k+1$ elements and would be pairwise distinguishable by L , which would contradict the assumption that L has index k . Let $F = \{q_i \mid s_i \in L\}$. Let the start state q_0 be the q_i such that $s_i \equiv_L \epsilon$. M is constructed so that for any state q_i , $\{s \mid \delta(q_0, s) = q_i\} = \{s \mid s \equiv_L s_i\}$. Hence M recognizes L .

(c) Suppose that L is regular and let k be the number of states in a DFA recognizing L . Then from part (a), L has index at most k . Conversely, if L has index k , then by part (b) it is recognized by a DFA with k states and thus is regular. To show that the index of L is the size of the smallest DFA accepting it, suppose that L 's index is *exactly* k . Then, by part (b), there is a k -state DFA accepting L . That is the smallest such DFA because if it were any smaller, then we could show by part (a) that the index of L is less than k .

1.55 (a) The minimum pumping length is 4. The string 000 is in the language but cannot be pumped, so 3 is not a pumping length for this language. If s has length 4 or more, it contains 1s. By dividing s into xyz , where x is 000 and y is the first 1 and z is everything afterward, we satisfy the pumping lemma's three conditions.

(b) The minimum pumping length is 1. The pumping length cannot be 0 because the string ϵ is in the language and it cannot be pumped. Every nonempty string in the language can be divided into xyz , where x , y , and z are ϵ , the first character, and the remainder, respectively. This division satisfies the three conditions.

(d) The minimum pumping length is 3. The pumping length cannot be 2 because the string 11 is in the language and it cannot be pumped. Let s be a string in the language of length at least 3. If s is generated by $0^*1^*0^*1^*$ and s begins either 0 or 11, write $s = xyz$ where $x = \epsilon$, y is the first symbol, and z is the remainder of s . If s is generated by $0^*1^*0^*1^*$ and s begins 10, write $s = xyz$ where $x = 10$, y is the next symbol, and z is the remainder of s . Breaking s up in this way shows that it can be pumped. If s is generated by 10^*1 , we can write it as xyz where $x = 1$, $y = 0$, and z is the remainder of s . This division gives a way to pump s .

2

CONTEXT-FREE LANGUAGES

In Chapter 1 we introduced two different, though equivalent, methods of describing languages: *finite automata* and *regular expressions*. We showed that many languages can be described in this way but that some simple languages, such as $\{0^n 1^n \mid n \geq 0\}$, cannot.

In this chapter we present **context-free grammars**, a more powerful method of describing languages. Such grammars can describe certain features that have a recursive structure, which makes them useful in a variety of applications.

Context-free grammars were first used in the study of human languages. One way of understanding the relationship of terms such as *noun*, *verb*, and *preposition* and their respective phrases leads to a natural recursion because noun phrases may appear inside verb phrases and vice versa. Context-free grammars help us organize and understand these relationships.

An important application of context-free grammars occurs in the specification and compilation of programming languages. A grammar for a programming language often appears as a reference for people trying to learn the language syntax. Designers of compilers and interpreters for programming languages often start by obtaining a grammar for the language. Most compilers and interpreters contain a component called a **parser** that extracts the meaning of a program prior to generating the compiled code or performing the interpreted execution. A number of methodologies facilitate the construction of a parser once a context-free grammar is available. Some tools even automatically generate the parser from the grammar.

The collection of languages associated with context-free grammars are called the ***context-free languages***. They include all the regular languages and many additional languages. In this chapter, we give a formal definition of context-free grammars and study the properties of context-free languages. We also introduce ***pushdown automata***, a class of machines recognizing the context-free languages. Pushdown automata are useful because they allow us to gain additional insight into the power of context-free grammars.

2.1

CONTEXT-FREE GRAMMARS

The following is an example of a context-free grammar, which we call G_1 .

$$\begin{array}{l} A \rightarrow 0A1 \\ A \rightarrow B \\ B \rightarrow \# \end{array}$$

A grammar consists of a collection of *substitution rules*, also called *productions*. Each rule appears as a line in the grammar, comprising a symbol and a string separated by an arrow. The symbol is called a *variable*. The string consists of variables and other symbols called *terminals*. The variable symbols often are represented by capital letters. The terminals are analogous to the input alphabet and often are represented by lowercase letters, numbers, or special symbols. One variable is designated as the *start variable*. It usually occurs on the left-hand side of the topmost rule. For example, grammar G_1 contains three rules. G_1 's variables are A and B , where A is the start variable. Its terminals are 0, 1, and #.

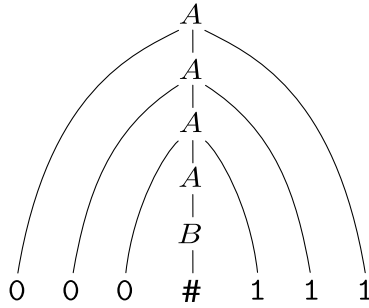
You use a grammar to describe a language by generating each string of that language in the following manner.

1. Write down the start variable. It is the variable on the left-hand side of the top rule, unless specified otherwise.
2. Find a variable that is written down and a rule that starts with that variable. Replace the written down variable with the right-hand side of that rule.
3. Repeat step 2 until no variables remain.

For example, grammar G_1 generates the string 000#111. The sequence of substitutions to obtain a string is called a **derivation**. A derivation of string 000#111 in grammar G_1 is

$$A \Rightarrow 0A1 \Rightarrow 00A11 \Rightarrow 000A111 \Rightarrow 000B111 \Rightarrow 000\#111.$$

You may also represent the same information pictorially with a *parse tree*. An example of a parse tree is shown in Figure 2.1.

**FIGURE 2.1**

Parse tree for 000#111 in grammar G_1

All strings generated in this way constitute the *language of the grammar*. We write $L(G_1)$ for the language of grammar G_1 . Some experimentation with the grammar G_1 shows us that $L(G_1)$ is $\{0^n\#1^n \mid n \geq 0\}$. Any language that can be generated by some context-free grammar is called a *context-free language* (CFL). For convenience when presenting a context-free grammar, we abbreviate several rules with the same left-hand variable, such as $A \rightarrow 0A1$ and $A \rightarrow B$, into a single line $A \rightarrow 0A1 \mid B$, using the symbol “ $\mid$ ” as an “or”.

The following is a second example of a context-free grammar, called G_2 , which describes a fragment of the English language.

```

<SENTENCE> → <NOUN-PHRASE><VERB-PHRASE>
<NOUN-PHRASE> → <CMPLX-NOUN> | <CMPLX-NOUN><PREP-PHRASE>
<VERB-PHRASE> → <CMPLX-VERB> | <CMPLX-VERB><PREP-PHRASE>
<PREP-PHRASE> → <PREP><CMPLX-NOUN>
<CMPLX-NOUN> → <ARTICLE><NOUN>
<CMPLX-VERB> → <VERB> | <VERB><NOUN-PHRASE>
<ARTICLE> → a | the
<NOUN> → boy | girl | flower
<VERB> → touches | likes | sees
<PREP> → with
  
```

Grammar G_2 has 10 variables (the capitalized grammatical terms written inside brackets); 27 terminals (the standard English alphabet plus a space character); and 18 rules. Strings in $L(G_2)$ include:

```

a boy sees
the boy sees a flower
a girl with a flower likes the boy
  
```

Each of these strings has a derivation in grammar G_2 . The following is a derivation of the first string on this list.

$$\begin{aligned}
\langle \text{SENTENCE} \rangle &\Rightarrow \langle \text{NOUN-PHRASE} \rangle \langle \text{VERB-PHRASE} \rangle \\
&\Rightarrow \langle \text{CMPLX-NOUN} \rangle \langle \text{VERB-PHRASE} \rangle \\
&\Rightarrow \langle \text{ARTICLE} \rangle \langle \text{NOUN} \rangle \langle \text{VERB-PHRASE} \rangle \\
&\Rightarrow a \langle \text{NOUN} \rangle \langle \text{VERB-PHRASE} \rangle \\
&\Rightarrow a \text{ boy } \langle \text{VERB-PHRASE} \rangle \\
&\Rightarrow a \text{ boy } \langle \text{CMPLX-VERB} \rangle \\
&\Rightarrow a \text{ boy } \langle \text{VERB} \rangle \\
&\Rightarrow a \text{ boy sees}
\end{aligned}$$

FORMAL DEFINITION OF A CONTEXT-FREE GRAMMAR

Let's formalize our notion of a context-free grammar (CFG).

DEFINITION 2.2

A *context-free grammar* is a 4-tuple (V, Σ, R, S) , where

1. V is a finite set called the *variables*,
2. Σ is a finite set, disjoint from V , called the *terminals*,
3. R is a finite set of *rules*, with each rule being a variable and a string of variables and terminals, and
4. $S \in V$ is the start variable.

If u , v , and w are strings of variables and terminals, and $A \rightarrow w$ is a rule of the grammar, we say that uAv *yields* uwv , written $uAv \Rightarrow uwv$. Say that u *derives* v , written $u \Rightarrow^* v$, if $u = v$ or if a sequence $u_1, u_2, \dots, u_k$ exists for $k \geq 0$ and

$$u \Rightarrow u_1 \Rightarrow u_2 \Rightarrow \dots \Rightarrow u_k \Rightarrow v.$$

The *language of the grammar* is $\{w \in \Sigma^* \mid S \Rightarrow^* w\}$.

In grammar G_1 , $V = \{A, B\}$, $\Sigma = \{0, 1, \#\}$, $S = A$, and R is the collection of the three rules appearing on page 102. In grammar G_2 ,

$$\begin{aligned}
V = \{ &\langle \text{SENTENCE} \rangle, \langle \text{NOUN-PHRASE} \rangle, \langle \text{VERB-PHRASE} \rangle, \\
&\langle \text{PREP-PHRASE} \rangle, \langle \text{CMPLX-NOUN} \rangle, \langle \text{CMPLX-VERB} \rangle, \\
&\langle \text{ARTICLE} \rangle, \langle \text{NOUN} \rangle, \langle \text{VERB} \rangle, \langle \text{PREP} \rangle \},
\end{aligned}$$

and $\Sigma = \{a, b, c, \dots, z, " \}$. The symbol $"$ is the blank symbol, placed invisibly after each word (a, boy, etc.), so the words won't run together.

Often we specify a grammar by writing down only its rules. We can identify the variables as the symbols that appear on the left-hand side of the rules and the terminals as the remaining symbols. By convention, the start variable is the variable on the left-hand side of the first rule.

EXAMPLES OF CONTEXT-FREE GRAMMARS

EXAMPLE 2.3

Consider grammar $G_3 = (\{S\}, \{a, b\}, R, S)$. The set of rules, R , is

$$S \rightarrow aSb \mid SS \mid \varepsilon.$$

This grammar generates strings such as $abab$, $aaabbb$, and $aababb$. You can see more easily what this language is if you think of a as a left parenthesis “(” and b as a right parenthesis “)”. Viewed in this way, $L(G_3)$ is the language of all strings of properly nested parentheses. Observe that the right-hand side of a rule may be the empty string ε . ■

EXAMPLE 2.4

Consider grammar $G_4 = (V, \Sigma, R, \langle \text{EXPR} \rangle)$.

V is $\{\langle \text{EXPR} \rangle, \langle \text{TERM} \rangle, \langle \text{FACTOR} \rangle\}$ and Σ is $\{a, +, \times, (,)\}$. The rules are

$$\begin{aligned}\langle \text{EXPR} \rangle &\rightarrow \langle \text{EXPR} \rangle + \langle \text{TERM} \rangle \mid \langle \text{TERM} \rangle \\ \langle \text{TERM} \rangle &\rightarrow \langle \text{TERM} \rangle \times \langle \text{FACTOR} \rangle \mid \langle \text{FACTOR} \rangle \\ \langle \text{FACTOR} \rangle &\rightarrow (\langle \text{EXPR} \rangle) \mid a\end{aligned}$$

The two strings $a+a \times a$ and $(a+a) \times a$ can be generated with grammar G_4 . The parse trees are shown in the following figure.

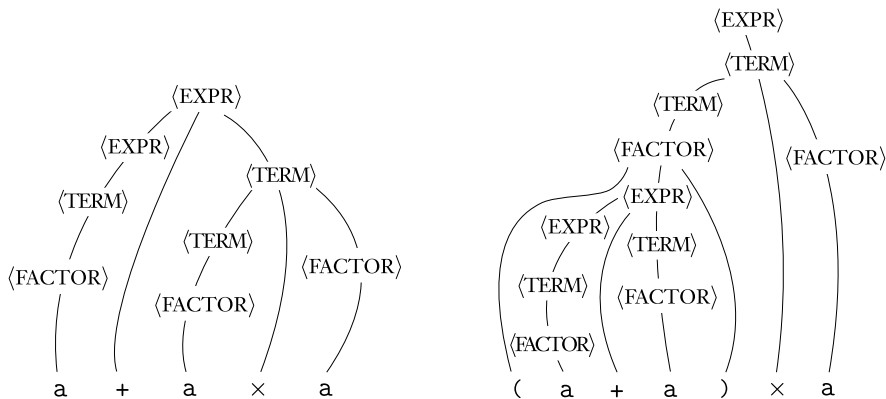


FIGURE 2.5

Parse trees for the strings $a+a \times a$ and $(a+a) \times a$

A compiler translates code written in a programming language into another form, usually one more suitable for execution. To do so, the compiler extracts

the meaning of the code to be compiled in a process called *parsing*. One representation of this meaning is the parse tree for the code, in the context-free grammar for the programming language. We discuss an algorithm that parses context-free languages later in Theorem 7.16 and in Problem 7.45.

Grammar G_4 describes a fragment of a programming language concerned with arithmetic expressions. Observe how the parse trees in Figure 2.5 “group” the operations. The tree for $a+axa$ groups the $\times$ operator and its operands (the second two a ’s) as one operand of the $+$ operator. In the tree for $(a+a) \times a$, the grouping is reversed. These groupings fit the standard precedence of multiplication before addition and the use of parentheses to override the standard precedence. Grammar G_4 is designed to capture these precedence relations. ■

DESIGNING CONTEXT-FREE GRAMMARS

As with the design of finite automata, discussed in Section 1.1 (page 41), the design of context-free grammars requires creativity. Indeed, context-free grammars are even trickier to construct than finite automata because we are more accustomed to programming a machine for specific tasks than we are to describing languages with grammars. The following techniques are helpful, singly or in combination, when you’re faced with the problem of constructing a CFG.

First, many CFLs are the union of simpler CFLs. If you must construct a CFG for a CFL that you can break into simpler pieces, do so and then construct individual grammars for each piece. These individual grammars can be easily merged into a grammar for the original language by combining their rules and then adding the new rule $S \rightarrow S_1 \mid S_2 \mid \dots \mid S_k$, where the variables S_i are the start variables for the individual grammars. Solving several simpler problems is often easier than solving one complicated problem.

For example, to get a grammar for the language $\{0^n 1^n \mid n \geq 0\} \cup \{1^n 0^n \mid n \geq 0\}$, first construct the grammar

$$S_1 \rightarrow 0S_11 \mid \varepsilon$$

for the language $\{0^n 1^n \mid n \geq 0\}$ and the grammar

$$S_2 \rightarrow 1S_20 \mid \varepsilon$$

for the language $\{1^n 0^n \mid n \geq 0\}$ and then add the rule $S \rightarrow S_1 \mid S_2$ to give the grammar

$$\begin{aligned} S &\rightarrow S_1 \mid S_2 \\ S_1 &\rightarrow 0S_11 \mid \varepsilon \\ S_2 &\rightarrow 1S_20 \mid \varepsilon. \end{aligned}$$

Second, constructing a CFG for a language that happens to be regular is easy if you can first construct a DFA for that language. You can convert any DFA into an equivalent CFG as follows. Make a variable R_i for each state q_i of the DFA. Add the rule $R_i \rightarrow aR_j$ to the CFG if $\delta(q_i, a) = q_j$ is a transition in the DFA. Add the rule $R_i \rightarrow \epsilon$ if q_i is an accept state of the DFA. Make R_0 the start variable of the grammar, where q_0 is the start state of the machine. Verify on your own that the resulting CFG generates the same language that the DFA recognizes.

Third, certain context-free languages contain strings with two substrings that are “linked” in the sense that a machine for such a language would need to remember an unbounded amount of information about one of the substrings to verify that it corresponds properly to the other substring. This situation occurs in the language $\{0^n 1^n \mid n \geq 0\}$ because a machine would need to remember the number of 0s in order to verify that it equals the number of 1s. You can construct a CFG to handle this situation by using a rule of the form $R \rightarrow uRv$, which generates strings wherein the portion containing the u ’s corresponds to the portion containing the v ’s.

Finally, in more complex languages, the strings may contain certain structures that appear recursively as part of other (or the same) structures. That situation occurs in the grammar that generates arithmetic expressions in Example 2.4. Any time the symbol a appears, an entire parenthesized expression might appear recursively instead. To achieve this effect, place the variable symbol generating the structure in the location of the rules corresponding to where that structure may recursively appear.

AMBIGUITY

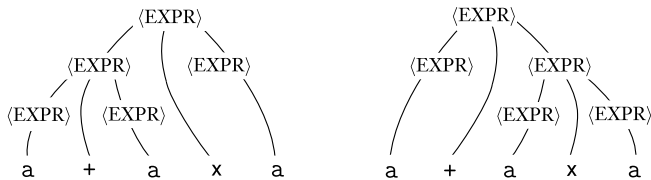
Sometimes a grammar can generate the same string in several different ways. Such a string will have several different parse trees and thus several different meanings. This result may be undesirable for certain applications, such as programming languages, where a program should have a unique interpretation.

If a grammar generates the same string in several different ways, we say that the string is derived *ambiguously* in that grammar. If a grammar generates some string ambiguously, we say that the grammar is *ambiguous*.

For example, consider grammar G_5 :

$$\langle \text{EXPR} \rangle \rightarrow \langle \text{EXPR} \rangle + \langle \text{EXPR} \rangle \mid \langle \text{EXPR} \rangle \times \langle \text{EXPR} \rangle \mid (\langle \text{EXPR} \rangle) \mid a$$

This grammar generates the string $a+a \times a$ ambiguously. The following figure shows the two different parse trees.

**FIGURE 2.6**

The two parse trees for the string $a+a \times a$ in grammar G_5

This grammar doesn't capture the usual precedence relations and so may group the $+$ before the $\times$ or vice versa. In contrast, grammar G_4 generates exactly the same language, but every generated string has a unique parse tree. Hence G_4 is unambiguous, whereas G_5 is ambiguous.

Grammar G_2 (page 103) is another example of an ambiguous grammar. The sentence *the girl touches the boy with the flower* has two different derivations. In Exercise 2.8 you are asked to give the two parse trees and observe their correspondence with the two different ways to read that sentence.

Now we formalize the notion of ambiguity. When we say that a grammar generates a string ambiguously, we mean that the string has two different parse trees, not two different derivations. Two derivations may differ merely in the order in which they replace variables yet not in their overall structure. To concentrate on structure, we define a type of derivation that replaces variables in a fixed order. A derivation of a string w in a grammar G is a **leftmost derivation** if at every step the leftmost remaining variable is the one replaced. The derivation preceding Definition 2.2 (page 104) is a leftmost derivation.

DEFINITION 2.7

A string w is derived **ambiguously** in context-free grammar G if it has two or more different leftmost derivations. Grammar G is **ambiguous** if it generates some string ambiguously.

Sometimes when we have an ambiguous grammar we can find an unambiguous grammar that generates the same language. Some context-free languages, however, can be generated only by ambiguous grammars. Such languages are called **inherently ambiguous**. Problem 2.29 asks you to prove that the language $\{a^i b^j c^k \mid i = j \text{ or } j = k\}$ is inherently ambiguous.

CHOMSKY NORMAL FORM

When working with context-free grammars, it is often convenient to have them in simplified form. One of the simplest and most useful forms is called the

Chomsky normal form. Chomsky normal form is useful in giving algorithms for working with context-free grammars, as we do in Chapters 4 and 7.

DEFINITION 2.8

A context-free grammar is in *Chomsky normal form* if every rule is of the form

$$\begin{aligned} A &\rightarrow BC \\ A &\rightarrow a \end{aligned}$$

where a is any terminal and A , B , and C are any variables—except that B and C may not be the start variable. In addition, we permit the rule $S \rightarrow \epsilon$, where S is the start variable.

THEOREM 2.9

Any context-free language is generated by a context-free grammar in Chomsky normal form.

PROOF IDEA We can convert any grammar G into Chomsky normal form. The conversion has several stages wherein rules that violate the conditions are replaced with equivalent ones that are satisfactory. First, we add a new start variable. Then, we eliminate all ϵ -rules of the form $A \rightarrow \epsilon$. We also eliminate all *unit rules* of the form $A \rightarrow B$. In both cases we patch up the grammar to be sure that it still generates the same language. Finally, we convert the remaining rules into the proper form.

PROOF First, we add a new start variable S_0 and the rule $S_0 \rightarrow S$, where S was the original start variable. This change guarantees that the start variable doesn't occur on the right-hand side of a rule.

Second, we take care of all ϵ -rules. We remove an ϵ -rule $A \rightarrow \epsilon$, where A is not the start variable. Then for each occurrence of an A on the right-hand side of a rule, we add a new rule with that occurrence deleted. In other words, if $R \rightarrow uAv$ is a rule in which u and v are strings of variables and terminals, we add rule $R \rightarrow uv$. We do so for each *occurrence* of an A , so the rule $R \rightarrow uAvAw$ causes us to add $R \rightarrow uvAw$, $R \rightarrow uAvw$, and $R \rightarrow uvw$. If we have the rule $R \rightarrow A$, we add $R \rightarrow \epsilon$ unless we had previously removed the rule $R \rightarrow \epsilon$. We repeat these steps until we eliminate all ϵ -rules not involving the start variable.

Third, we handle all unit rules. We remove a unit rule $A \rightarrow B$. Then, whenever a rule $B \rightarrow u$ appears, we add the rule $A \rightarrow u$ unless this was a unit rule previously removed. As before, u is a string of variables and terminals. We repeat these steps until we eliminate all unit rules.

Finally, we convert all remaining rules into the proper form. We replace each rule $A \rightarrow u_1u_2 \cdots u_k$, where $k \geq 3$ and each u_i is a variable or terminal symbol,

with the rules $A \rightarrow u_1 A_1$, $A_1 \rightarrow u_2 A_2$, $A_2 \rightarrow u_3 A_3$, $\dots$, and $A_{k-2} \rightarrow u_{k-1} u_k$. The A_i 's are new variables. We replace any terminal u_i in the preceding rule(s) with the new variable U_i and add the rule $U_i \rightarrow u_i$.

EXAMPLE 2.10

Let G_6 be the following CFG and convert it to Chomsky normal form by using the conversion procedure just given. The series of grammars presented illustrates the steps in the conversion. Rules shown in bold have just been added. Rules shown in gray have just been removed.

1. The original CFG G_6 is shown on the left. The result of applying the first step to make a new start variable appears on the right.

$$\begin{array}{ll} S \rightarrow ASA \mid \mathbf{aB} & S_0 \rightarrow S \\ A \rightarrow B \mid S & S \rightarrow ASA \mid \mathbf{aB} \\ B \rightarrow \mathbf{b} \mid \epsilon & A \rightarrow B \mid S \\ & B \rightarrow \mathbf{b} \mid \epsilon \end{array}$$

2. Remove ϵ -rules $B \rightarrow \epsilon$, shown on the left, and $A \rightarrow \epsilon$, shown on the right.

$$\begin{array}{ll} S_0 \rightarrow S & S_0 \rightarrow S \\ S \rightarrow ASA \mid \mathbf{aB} \mid \mathbf{a} & S \rightarrow ASA \mid \mathbf{aB} \mid \mathbf{a} \mid \mathbf{SA} \mid \mathbf{AS} \mid S \\ A \rightarrow B \mid S \mid \epsilon & A \rightarrow B \mid S \mid \epsilon \\ B \rightarrow \mathbf{b} \mid \epsilon & B \rightarrow \mathbf{b} \end{array}$$

- 3a. Remove unit rules $S \rightarrow S$, shown on the left, and $S_0 \rightarrow S$, shown on the right.

$$\begin{array}{ll} S_0 \rightarrow S & S_0 \rightarrow \mathbf{S} \mid \mathbf{ASA} \mid \mathbf{aB} \mid \mathbf{a} \mid \mathbf{SA} \mid \mathbf{AS} \\ S \rightarrow ASA \mid \mathbf{aB} \mid \mathbf{a} \mid \mathbf{SA} \mid \mathbf{AS} \mid S & S \rightarrow ASA \mid \mathbf{aB} \mid \mathbf{a} \mid \mathbf{SA} \mid \mathbf{AS} \\ A \rightarrow B \mid S & A \rightarrow B \mid S \\ B \rightarrow \mathbf{b} & B \rightarrow \mathbf{b} \end{array}$$

- 3b. Remove unit rules $A \rightarrow B$ and $A \rightarrow S$.

$$\begin{array}{ll} S_0 \rightarrow ASA \mid \mathbf{aB} \mid \mathbf{a} \mid \mathbf{SA} \mid \mathbf{AS} & S_0 \rightarrow ASA \mid \mathbf{aB} \mid \mathbf{a} \mid \mathbf{SA} \mid \mathbf{AS} \\ S \rightarrow ASA \mid \mathbf{aB} \mid \mathbf{a} \mid \mathbf{SA} \mid \mathbf{AS} & S \rightarrow ASA \mid \mathbf{aB} \mid \mathbf{a} \mid \mathbf{SA} \mid \mathbf{AS} \\ A \rightarrow \mathbf{B} \mid S \mid \mathbf{b} & A \rightarrow \mathbf{S} \mid \mathbf{b} \mid \mathbf{ASA} \mid \mathbf{aB} \mid \mathbf{a} \mid \mathbf{SA} \mid \mathbf{AS} \\ B \rightarrow \mathbf{b} & B \rightarrow \mathbf{b} \end{array}$$

With the addition of a stack component we obtain a schematic representation of a pushdown automaton, as shown in the following figure.

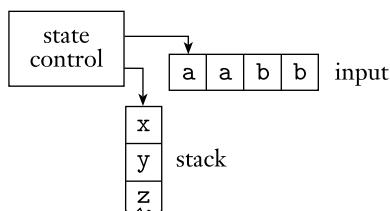


FIGURE 2.12
Schematic of a pushdown automaton

A pushdown automaton (PDA) can write symbols on the stack and read them back later. Writing a symbol “pushes down” all the other symbols on the stack. At any time the symbol on the top of the stack can be read and removed. The remaining symbols then move back up. Writing a symbol on the stack is often referred to as *pushing* the symbol, and removing a symbol is referred to as *popping* it. Note that all access to the stack, for both reading and writing, may be done only at the top. In other words a stack is a “last in, first out” storage device. If certain information is written on the stack and additional information is written afterward, the earlier information becomes inaccessible until the later information is removed.

Plates on a cafeteria serving counter illustrate a stack. The stack of plates rests on a spring so that when a new plate is placed on top of the stack, the plates below it move down. The stack on a pushdown automaton is like a stack of plates, with each plate having a symbol written on it.

A stack is valuable because it can hold an unlimited amount of information. Recall that a finite automaton is unable to recognize the language $\{0^n 1^n \mid n \geq 0\}$ because it cannot store very large numbers in its finite memory. A PDA is able to recognize this language because it can use its stack to store the number of 0s it has seen. Thus the unlimited nature of a stack allows the PDA to store numbers of unbounded size. The following informal description shows how the automaton for this language works.

Read symbols from the input. As each 0 is read, push it onto the stack. As soon as 1s are seen, pop a 0 off the stack for each 1 read. If reading the input is finished exactly when the stack becomes empty of 0s, accept the input. If the stack becomes empty while 1s remain or if the 1s are finished while the stack still contains 0s or if any 0s appear in the input following 1s, reject the input.

As mentioned earlier, pushdown automata may be nondeterministic. Deterministic and nondeterministic pushdown automata are *not* equivalent in power.

Nondeterministic pushdown automata recognize certain languages that no deterministic pushdown automata can recognize, as we will see in Section 2.4. We give languages requiring nondeterminism in Examples 2.16 and 2.18. Recall that deterministic and nondeterministic finite automata do recognize the same class of languages, so the pushdown automata situation is different. We focus on nondeterministic pushdown automata because these automata are equivalent in power to context-free grammars.

FORMAL DEFINITION OF A PUSHDOWN AUTOMATON

The formal definition of a pushdown automaton is similar to that of a finite automaton, except for the stack. The stack is a device containing symbols drawn from some alphabet. The machine may use different alphabets for its input and its stack, so now we specify both an input alphabet Σ and a stack alphabet Γ .

At the heart of any formal definition of an automaton is the transition function, which describes its behavior. Recall that $\Sigma_\epsilon = \Sigma \cup \{\epsilon\}$ and $\Gamma_\epsilon = \Gamma \cup \{\epsilon\}$. The domain of the transition function is $Q \times \Sigma_\epsilon \times \Gamma_\epsilon$. Thus the current state, next input symbol read, and top symbol of the stack determine the next move of a pushdown automaton. Either symbol may be ϵ , causing the machine to move without reading a symbol from the input or without reading a symbol from the stack.

For the range of the transition function we need to consider what to allow the automaton to do when it is in a particular situation. It may enter some new state and possibly write a symbol on the top of the stack. The function δ can indicate this action by returning a member of Q together with a member of Γ_ϵ , that is, a member of $Q \times \Gamma_\epsilon$. Because we allow nondeterminism in this model, a situation may have several legal next moves. The transition function incorporates nondeterminism in the usual way, by returning a set of members of $Q \times \Gamma_\epsilon$, that is, a member of $\mathcal{P}(Q \times \Gamma_\epsilon)$. Putting it all together, our transition function δ takes the form $\delta: Q \times \Sigma_\epsilon \times \Gamma_\epsilon \rightarrow \mathcal{P}(Q \times \Gamma_\epsilon)$.

DEFINITION 2.13

A **pushdown automaton** is a 6-tuple $(Q, \Sigma, \Gamma, \delta, q_0, F)$, where Q , Σ , Γ , and F are all finite sets, and

1. Q is the set of states,
2. Σ is the input alphabet,
3. Γ is the stack alphabet,
4. $\delta: Q \times \Sigma_\epsilon \times \Gamma_\epsilon \rightarrow \mathcal{P}(Q \times \Gamma_\epsilon)$ is the transition function,
5. $q_0 \in Q$ is the start state, and
6. $F \subseteq Q$ is the set of accept states.

A pushdown automaton $M = (Q, \Sigma, \Gamma, \delta, q_0, F)$ computes as follows. It accepts input w if w can be written as $w = w_1 w_2 \cdots w_m$, where each $w_i \in \Sigma_\epsilon$ and sequences of states $r_0, r_1, \dots, r_m \in Q$ and strings $s_0, s_1, \dots, s_m \in \Gamma^*$ exist that satisfy the following three conditions. The strings s_i represent the sequence of stack contents that M has on the accepting branch of the computation.

1. $r_0 = q_0$ and $s_0 = \epsilon$. This condition signifies that M starts out properly, in the start state and with an empty stack.
2. For $i = 0, \dots, m - 1$, we have $(r_{i+1}, b) \in \delta(r_i, w_{i+1}, a)$, where $s_i = at$ and $s_{i+1} = bt$ for some $a, b \in \Gamma_\epsilon$ and $t \in \Gamma^*$. This condition states that M moves properly according to the state, stack, and next input symbol.
3. $r_m \in F$. This condition states that an accept state occurs at the input end.

EXAMPLES OF PUSHDOWN AUTOMATA

EXAMPLE 2.14

The following is the formal description of the PDA (page 112) that recognizes the language $\{0^n 1^n \mid n \geq 0\}$. Let M_1 be $(Q, \Sigma, \Gamma, \delta, q_1, F)$, where

$$Q = \{q_1, q_2, q_3, q_4\},$$

$$\Sigma = \{0, 1\},$$

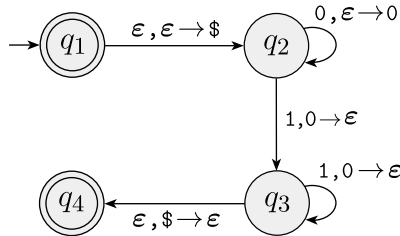
$$\Gamma = \{0, \$\},$$

$$F = \{q_1, q_4\}, \text{ and}$$

δ is given by the following table, wherein blank entries signify $\emptyset$.

Input:	0			1			ϵ		
Stack:	0	\$	ϵ	0	\$	ϵ	0	\$	ϵ
q_1									$\{(q_2, \$)\}$
q_2			$\{(q_2, 0)\}$			$\{(q_3, \epsilon)\}$			
q_3						$\{(q_3, \epsilon)\}$			$\{(q_4, \epsilon)\}$
q_4									

We can also use a state diagram to describe a PDA, as in Figures 2.15, 2.17, and 2.19. Such diagrams are similar to the state diagrams used to describe finite automata, modified to show how the PDA uses its stack when going from state to state. We write “ $a, b \rightarrow c$ ” to signify that when the machine is reading an a from the input, it may replace the symbol b on the top of the stack with a c . Any of a , b , and c may be ϵ . If a is ϵ , the machine may make this transition without reading any symbol from the input. If b is ϵ , the machine may make this transition without reading and popping any symbol from the stack. If c is ϵ , the machine does not write any symbol on the stack when going along this transition.

**FIGURE 2.15**

State diagram for the PDA M_1 that recognizes $\{0^n 1^n \mid n \geq 0\}$

The formal definition of a PDA contains no explicit mechanism to allow the PDA to test for an empty stack. This PDA is able to get the same effect by initially placing a special symbol $\$$ on the stack. Then if it ever sees the $\$$ again, it knows that the stack effectively is empty. Subsequently, when we refer to testing for an empty stack in an informal description of a PDA, we implement the procedure in the same way.

Similarly, PDAs cannot test explicitly for having reached the end of the input string. This PDA is able to achieve that effect because the accept state takes effect only when the machine is at the end of the input. Thus from now on, we assume that PDAs can test for the end of the input, and we know that we can implement it in the same manner.

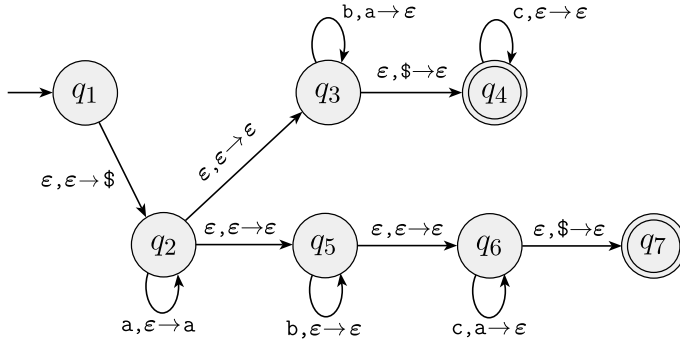
EXAMPLE 2.16

This example illustrates a pushdown automaton that recognizes the language

$$\{a^i b^j c^k \mid i, j, k \geq 0 \text{ and } i = j \text{ or } i = k\}.$$

Informally, the PDA for this language works by first reading and pushing the a's. When the a's are done, the machine has all of them on the stack so that it can match, them with either the b's or the c's. This maneuver is a bit tricky because the machine doesn't know in advance whether to match the a's with the b's or the c's. Nondeterminism comes in handy here.

Using its nondeterminism, the PDA can guess whether to match the a's with the b's or with the c's, as shown in Figure 2.17. Think of the machine as having two branches of its nondeterminism, one for each possible guess. If either of them matches, that branch accepts and the entire machine accepts. Problem 2.57 asks you to show that nondeterminism is essential for recognizing this language with a PDA.

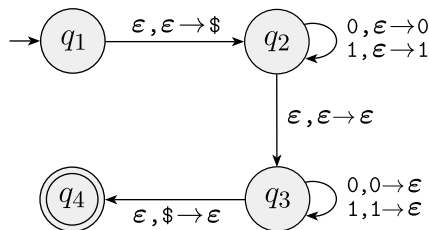
**FIGURE 2.17**

State diagram for PDA M_2 that recognizes $\{a^i b^j c^k \mid i, j, k \geq 0 \text{ and } i = j \text{ or } i = k\}$

EXAMPLE 2.18

In this example we give a PDA M_3 recognizing the language $\{ww^R \mid w \in \{0,1\}^*\}$. Recall that w^R means w written backwards. The informal description and state diagram of the PDA follow.

Begin by pushing the symbols that are read onto the stack. At each point, nondeterministically guess that the middle of the string has been reached and then change into popping off the stack for each symbol read, checking to see that they are the same. If they were always the same symbol and the stack empties at the same time as the input is finished, accept; otherwise reject.

**FIGURE 2.19**

State diagram for the PDA M_3 that recognizes $\{ww^R \mid w \in \{0,1\}^*\}$

Problem 2.58 shows that this language requires a nondeterministic PDA.

EQUIVALENCE WITH CONTEXT-FREE GRAMMARS

In this section we show that context-free grammars and pushdown automata are equivalent in power. Both are capable of describing the class of context-free languages. We show how to convert any context-free grammar into a pushdown automaton that recognizes the same language and vice versa. Recalling that we defined a context-free language to be any language that can be described with a context-free grammar, our objective is the following theorem.

THEOREM 2.20

A language is context free if and only if some pushdown automaton recognizes it.

As usual for “if and only if” theorems, we have two directions to prove. In this theorem, both directions are interesting. First, we do the easier forward direction.

LEMMA 2.21

If a language is context free, then some pushdown automaton recognizes it.

PROOF IDEA Let A be a CFL. From the definition we know that A has a CFG, G , generating it. We show how to convert G into an equivalent PDA, which we call P .

The PDA P that we now describe will work by accepting its input w , if G generates that input, by determining whether there is a derivation for w . Recall that a derivation is simply the sequence of substitutions made as a grammar generates a string. Each step of the derivation yields an *intermediate string* of variables and terminals. We design P to determine whether some series of substitutions using the rules of G can lead from the start variable to w .

One of the difficulties in testing whether there is a derivation for w is in figuring out which substitutions to make. The PDA’s nondeterminism allows it to guess the sequence of correct substitutions. At each step of the derivation, one of the rules for a particular variable is selected nondeterministically and used to substitute for that variable.

The PDA P begins by writing the start variable on its stack. It goes through a series of intermediate strings, making one substitution after another. Eventually it may arrive at a string that contains only terminal symbols, meaning that it has used the grammar to derive a string. Then P accepts if this string is identical to the string it has received as input.

Implementing this strategy on a PDA requires one additional idea. We need to see how the PDA stores the intermediate strings as it goes from one to another. Simply using the stack for storing each intermediate string is tempting. However, that doesn’t quite work because the PDA needs to find the variables in the intermediate string and make substitutions. The PDA can access only the top

symbol on the stack and that may be a terminal symbol instead of a variable. The way around this problem is to keep only *part* of the intermediate string on the stack: the symbols starting with the first variable in the intermediate string. Any terminal symbols appearing before the first variable are matched immediately with symbols in the input string. The following figure shows the PDA P .

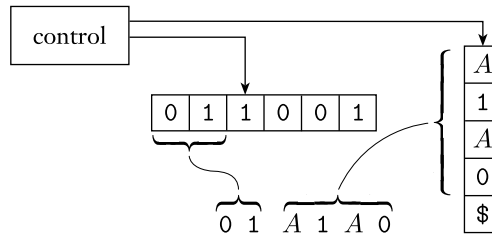


FIGURE 2.22

P representing the intermediate string 01A1A0

The following is an informal description of P .

1. Place the marker symbol \$ and the start variable on the stack.
2. Repeat the following steps forever.
 - a. If the top of stack is a variable symbol A , nondeterministically select one of the rules for A and substitute A by the string on the right-hand side of the rule.
 - b. If the top of stack is a terminal symbol a , read the next symbol from the input and compare it to a . If they match, repeat. If they do not match, reject on this branch of the nondeterminism.
 - c. If the top of stack is the symbol \$, enter the accept state. Doing so accepts the input if it has all been read.

PROOF We now give the formal details of the construction of the pushdown automaton $P = (Q, \Sigma, \Gamma, \delta, q_{\text{start}}, F)$. To make the construction clearer, we use shorthand notation for the transition function. This notation provides a way to write an entire string on the stack in one step of the machine. We can simulate this action by introducing additional states to write the string one symbol at a time, as implemented in the following formal construction.

Let q and r be states of the PDA and let a be in Σ_ϵ and s be in Γ_ϵ . Say that we want the PDA to go from q to r when it reads a and pops s . Furthermore, we want it to push the entire string $u = u_1 \cdots u_l$ on the stack at the same time. We can implement this action by introducing new states $q_1, \dots, q_{l-1}$ and setting the

transition function as follows:

$$\begin{aligned}\delta(q, a, s) &\text{ to contain } (q_1, u_l), \\ \delta(q_1, \varepsilon, \varepsilon) &= \{(q_2, u_{l-1})\}, \\ \delta(q_2, \varepsilon, \varepsilon) &= \{(q_3, u_{l-2})\}, \\ &\vdots \\ \delta(q_{l-1}, \varepsilon, \varepsilon) &= \{(r, u_1)\}.\end{aligned}$$

We use the notation $(r, u) \in \delta(q, a, s)$ to mean that when q is the state of the automaton, a is the next input symbol, and s is the symbol on the top of the stack, the PDA may read the a and pop the s , then push the string u onto the stack and go on to the state r . The following figure shows this implementation.

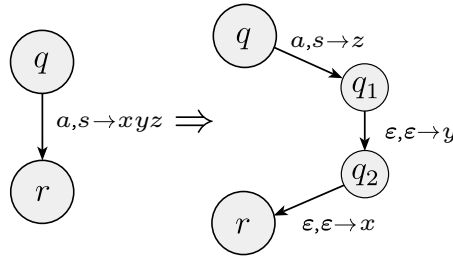


FIGURE 2.23

Implementing the shorthand $(r, xyz) \in \delta(q, a, s)$

The states of P are $Q = \{q_{\text{start}}, q_{\text{loop}}, q_{\text{accept}}\} \cup E$, where E is the set of states we need for implementing the shorthand just described. The start state is q_{start} . The only accept state is q_{accept} .

The transition function is defined as follows. We begin by initializing the stack to contain the symbols $\$$ and S , implementing step 1 in the informal description: $\delta(q_{\text{start}}, \varepsilon, \varepsilon) = \{(q_{\text{loop}}, S\$)\}$. Then we put in transitions for the main loop of step 2.

First, we handle case (a) wherein the top of the stack contains a variable. Let $\delta(q_{\text{loop}}, \varepsilon, A) = \{(q_{\text{loop}}, w) \mid \text{where } A \rightarrow w \text{ is a rule in } R\}$.

Second, we handle case (b) wherein the top of the stack contains a terminal. Let $\delta(q_{\text{loop}}, a, a) = \{(q_{\text{loop}}, \varepsilon)\}$.

Finally, we handle case (c) wherein the empty stack marker $\$$ is on the top of the stack. Let $\delta(q_{\text{loop}}, \varepsilon, \$) = \{(q_{\text{accept}}, \varepsilon)\}$.

The state diagram is shown in Figure 2.24.

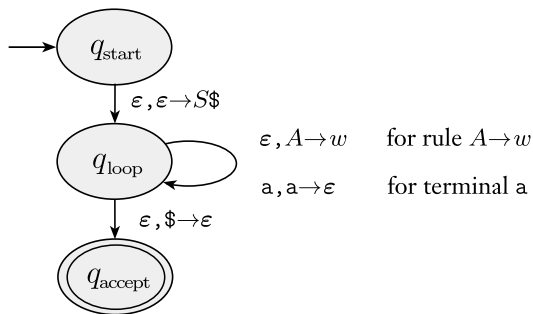


FIGURE 2.24
State diagram of P

That completes the proof of Lemma 2.21.

EXAMPLE 2.25

We use the procedure developed in Lemma 2.21 to construct a PDA P_1 from the following CFG G .

$$\begin{aligned} S &\rightarrow aTb \mid b \\ T &\rightarrow Ta \mid \epsilon \end{aligned}$$

The transition function is shown in the following diagram.

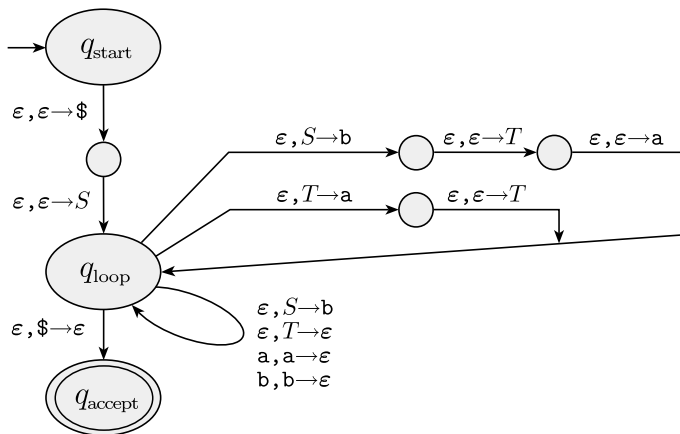


FIGURE 2.26
State diagram of P_1

Now we prove the reverse direction of Theorem 2.20. For the forward direction, we gave a procedure for converting a CFG into a PDA. The main idea was to design the automaton so that it simulates the grammar. Now we want to give a procedure for going the other way: converting a PDA into a CFG. We design the grammar to simulate the automaton. This task is challenging because “programming” an automaton is easier than “programming” a grammar.

LEMMA 2.27

If a pushdown automaton recognizes some language, then it is context free.

PROOF IDEA We have a PDA P , and we want to make a CFG G that generates all the strings that P accepts. In other words, G should generate a string if that string causes the PDA to go from its start state to an accept state.

To achieve this outcome, we design a grammar that does somewhat more. For each pair of states p and q in P , the grammar will have a variable A_{pq} . This variable generates all the strings that can take P from p with an empty stack to q with an empty stack. Observe that such strings can also take P from p to q , regardless of the stack contents at p , leaving the stack at q in the same condition as it was at p .

First, we simplify our task by modifying P slightly to give it the following three features.

1. It has a single accept state, q_{accept} .
2. It empties its stack before accepting.
3. Each transition either pushes a symbol onto the stack (a *push* move) or pops one off the stack (a *pop* move), but it does not do both at the same time.

Giving P features 1 and 2 is easy. To give it feature 3, we replace each transition that simultaneously pops and pushes with a two transition sequence that goes through a new state, and we replace each transition that neither pops nor pushes with a two transition sequence that pushes then pops an arbitrary stack symbol.

To design G so that A_{pq} generates all strings that take P from p to q , starting and ending with an empty stack, we must understand how P operates on these strings. For any such string x , P 's first move on x must be a push, because every move is either a push or a pop and P can't pop an empty stack. Similarly, the last move on x must be a pop because the stack ends up empty.

Two possibilities occur during P 's computation on x . Either the symbol popped at the end is the symbol that was pushed at the beginning, or not. If so, the stack could be empty only at the beginning and end of P 's computation on x . If not, the initially pushed symbol must get popped at some point before the end of x and thus the stack becomes empty at this point. We simulate the former possibility with the rule $A_{pq} \rightarrow aA_{rs}b$, where a is the input read at the first move, b is the input read at the last move, r is the state following p , and s is the state preceding q . We simulate the latter possibility with the rule $A_{pq} \rightarrow A_{pr}A_{rq}$, where r is the state when the stack becomes empty.

PROOF Say that $P = (Q, \Sigma, \Gamma, \delta, q_0, \{q_{\text{accept}}\})$ and construct G . The variables of G are $\{A_{pq} \mid p, q \in Q\}$. The start variable is $A_{q_0, q_{\text{accept}}}$. Now we describe G 's rules in three parts.

1. For each $p, q, r, s \in Q$, $u \in \Gamma$, and $a, b \in \Sigma_\epsilon$, if $\delta(p, a, \epsilon)$ contains (r, u) and $\delta(s, b, u)$ contains (q, ϵ) , put the rule $A_{pq} \rightarrow aA_{rs}b$ in G .
2. For each $p, q, r \in Q$, put the rule $A_{pq} \rightarrow A_{pr}A_{rq}$ in G .
3. Finally, for each $p \in Q$, put the rule $A_{pp} \rightarrow \epsilon$ in G .

You may gain some insight for this construction from the following figures.

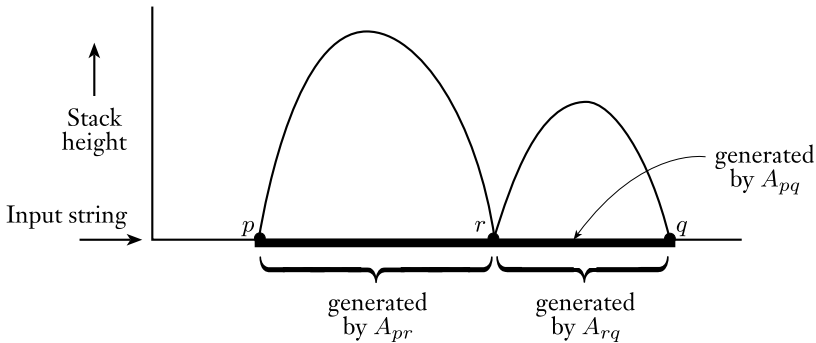


FIGURE 2.28

PDA computation corresponding to the rule $A_{pq} \rightarrow A_{pr}A_{rq}$

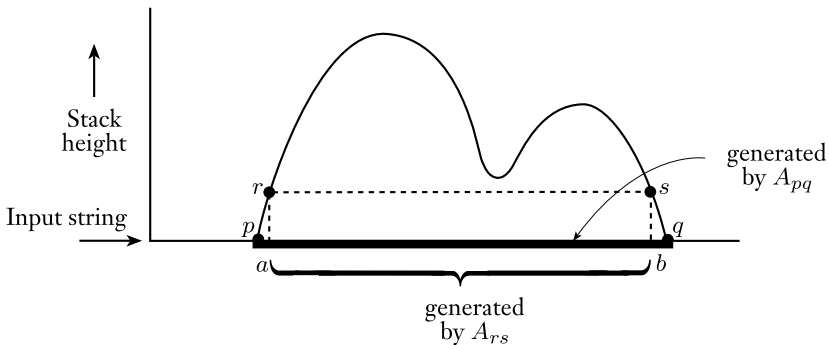


FIGURE 2.29

PDA computation corresponding to the rule $A_{pq} \rightarrow aA_{rs}b$

Now we prove that this construction works by demonstrating that A_{pq} generates x if and only if (iff) x can bring P from p with empty stack to q with empty stack. We consider each direction of the iff as a separate claim.

CLAIM 2.30

If A_{pq} generates x , then x can bring P from p with empty stack to q with empty stack.

We prove this claim by induction on the number of steps in the derivation of x from A_{pq} .

Basis: The derivation has 1 step.

A derivation with a single step must use a rule whose right-hand side contains no variables. The only rules in G where no variables occur on the right-hand side are $A_{pp} \rightarrow \epsilon$. Clearly, input ϵ takes P from p with empty stack to p with empty stack so the basis is proved.

Induction step: Assume true for derivations of length at most k , where $k \geq 1$, and prove true for derivations of length $k + 1$.

Suppose that $A_{pq} \xRightarrow{*} x$ with $k + 1$ steps. The first step in this derivation is either $A_{pq} \Rightarrow aA_{rs}b$ or $A_{pq} \Rightarrow A_{pr}A_{rq}$. We handle these two cases separately.

In the first case, consider the portion y of x that A_{rs} generates, so $x = ayb$. Because $A_{rs} \xRightarrow{*} y$ with k steps, the induction hypothesis tells us that P can go from r on empty stack to s on empty stack. Because $A_{pq} \rightarrow aA_{rs}b$ is a rule of G , $\delta(p, a, \epsilon)$ contains (r, u) and $\delta(s, b, u)$ contains (q, ϵ) , for some stack symbol u . Hence, if P starts at p with empty stack, after reading a it can go to state r and push u onto the stack. Then reading string y can bring it to s and leave u on the stack. Then after reading b it can go to state q and pop u off the stack. Therefore, x can bring it from p with empty stack to q with empty stack.

In the second case, consider the portions y and z of x that A_{pr} and A_{rq} respectively generate, so $x = yz$. Because $A_{pr} \xRightarrow{*} y$ in at most k steps and $A_{rq} \xRightarrow{*} z$ in at most k steps, the induction hypothesis tells us that y can bring P from p to r , and z can bring P from r to q , with empty stacks at the beginning and end. Hence x can bring it from p with empty stack to q with empty stack. This completes the induction step.

CLAIM 2.31

If x can bring P from p with empty stack to q with empty stack, A_{pq} generates x .

We prove this claim by induction on the number of steps in the computation of P that goes from p to q with empty stacks on input x .

Basis: The computation has 0 steps.

If a computation has 0 steps, it starts and ends at the same state—say, p . So we must show that $A_{pp} \xRightarrow{*} x$. In 0 steps, P cannot read any characters, so $x = \epsilon$. By construction, G has the rule $A_{pp} \rightarrow \epsilon$, so the basis is proved.

Induction step: Assume true for computations of length at most k , where $k \geq 0$, and prove true for computations of length $k + 1$.

Suppose that P has a computation wherein x brings p to q with empty stacks in $k + 1$ steps. Either the stack is empty only at the beginning and end of this computation, or it becomes empty elsewhere, too.

In the first case, the symbol that is pushed at the first move must be the same as the symbol that is popped at the last move. Call this symbol u . Let a be the input read in the first move, b be the input read in the last move, r be the state after the first move, and s be the state before the last move. Then $\delta(p, a, \epsilon)$ contains (r, u) and $\delta(s, b, u)$ contains (q, ϵ) , and so rule $A_{pq} \rightarrow aA_{rs}b$ is in G .

Let y be the portion of x without a and b , so $x = ayb$. Input y can bring P from r to s without touching the symbol u that is on the stack and so P can go from r with an empty stack to s with an empty stack on input y . We have removed the first and last steps of the $k + 1$ steps in the original computation on x so the computation on y has $(k + 1) - 2 = k - 1$ steps. Thus the induction hypothesis tells us that $A_{rs} \xRightarrow{*} y$. Hence $A_{pq} \xRightarrow{*} x$.

In the second case, let r be a state where the stack becomes empty other than at the beginning or end of the computation on x . Then the portions of the computation from p to r and from r to q each contain at most k steps. Say that y is the input read during the first portion and z is the input read during the second portion. The induction hypothesis tells us that $A_{pr} \xRightarrow{*} y$ and $A_{rq} \xRightarrow{*} z$. Because rule $A_{pq} \rightarrow A_{pr}A_{rq}$ is in G , $A_{pq} \xRightarrow{*} x$, and the proof is complete.

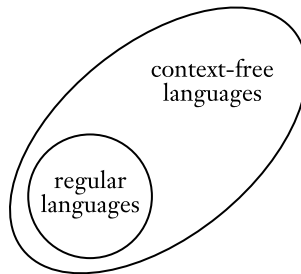
That completes the proof of Lemma 2.27 and of Theorem 2.20.

.....

We have just proved that pushdown automata recognize the class of context-free languages. This proof allows us to establish a relationship between the regular languages and the context-free languages. Because every regular language is recognized by a finite automaton and every finite automaton is automatically a pushdown automaton that simply ignores its stack, we now know that every regular language is also a context-free language.

COROLLARY 2.32

Every regular language is context free.

**FIGURE 2.33**

Relationship of the regular and context-free languages

2.3

NON-CONTEXT-FREE LANGUAGES

In this section we present a technique for proving that certain languages are not context free. Recall that in Section 1.4 we introduced the pumping lemma for showing that certain languages are not regular. Here we present a similar pumping lemma for context-free languages. It states that every context-free language has a special value called the **pumping length** such that all longer strings in the language can be “pumped.” This time the meaning of *pumped* is a bit more complex. It means that the string can be divided into five parts so that the second and the fourth parts may be repeated together any number of times and the resulting string still remains in the language.

THE PUMPING LEMMA FOR CONTEXT-FREE LANGUAGES

THEOREM 2.34

Pumping lemma for context-free languages If A is a context-free language, then there is a number p (the pumping length) where, if s is any string in A of length at least p , then s may be divided into five pieces $s = uvxyz$ satisfying the conditions

1. for each $i \geq 0$, $uv^ixy^iz \in A$,
2. $|vy| > 0$, and
3. $|vxy| \leq p$.

When s is being divided into $uvxyz$, condition 2 says that either v or y is not the empty string. Otherwise the theorem would be trivially true. Condition 3

states that the pieces v , x , and y together have length at most p . This technical condition sometimes is useful in proving that certain languages are not context free.

PROOF IDEA Let A be a CFL and let G be a CFG that generates it. We must show that any sufficiently long string s in A can be pumped and remain in A . The idea behind this approach is simple.

Let s be a very long string in A . (We make clear later what we mean by “very long.”) Because s is in A , it is derivable from G and so has a parse tree. The parse tree for s must be very tall because s is very long. That is, the parse tree must contain some long path from the start variable at the root of the tree to one of the terminal symbols at a leaf. On this long path, some variable symbol R must repeat because of the pigeonhole principle. As the following figure shows, this repetition allows us to replace the subtree under the second occurrence of R with the subtree under the first occurrence of R and still get a legal parse tree. Therefore, we may cut s into five pieces $uvxyz$ as the figure indicates, and we may repeat the second and fourth pieces and obtain a string still in the language. In other words, $uv^i xy^i z$ is in A for any $i \geq 0$.

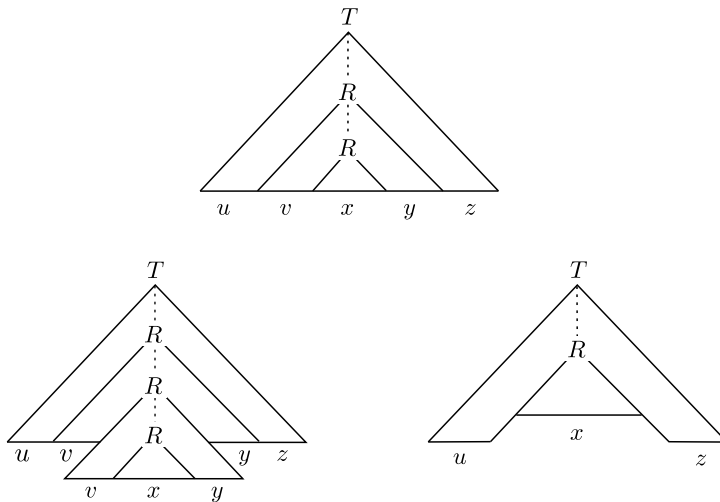


FIGURE 2.35
Surgery on parse trees

Let's now turn to the details to obtain all three conditions of the pumping lemma. We also show how to calculate the pumping length p .

PROOF Let G be a CFG for CFL A . Let b be the maximum number of symbols in the right-hand side of a rule (assume at least 2). In any parse tree using this grammar, we know that a node can have no more than b children. In other words, at most b leaves are 1 step from the start variable; at most b^2 leaves are within 2 steps of the start variable; and at most b^h leaves are within h steps of the start variable. So, if the height of the parse tree is at most h , the length of the string generated is at most b^h . Conversely, if a generated string is at least $b^h + 1$ long, each of its parse trees must be at least $h + 1$ high.

Say $|V|$ is the number of variables in G . We set p , the pumping length, to be $b^{|V|+1}$. Now if s is a string in A and its length is p or more, its parse tree must be at least $|V| + 1$ high, because $b^{|V|+1} \geq b^{|V|} + 1$.

To see how to pump any such string s , let τ be one of its parse trees. If s has several parse trees, choose τ to be a parse tree that has the smallest number of nodes. We know that τ must be at least $|V| + 1$ high, so its longest path from the root to a leaf has length at least $|V| + 1$. That path has at least $|V| + 2$ nodes; one at a terminal, the others at variables. Hence that path has at least $|V| + 1$ variables. With G having only $|V|$ variables, some variable R appears more than once on that path. For convenience later, we select R to be a variable that repeats among the lowest $|V| + 1$ variables on this path.

We divide s into $uvxyz$ according to Figure 2.35. Each occurrence of R has a subtree under it, generating a part of the string s . The upper occurrence of R has a larger subtree and generates vxy , whereas the lower occurrence generates just x with a smaller subtree. Both of these subtrees are generated by the same variable, so we may substitute one for the other and still obtain a valid parse tree. Replacing the smaller by the larger repeatedly gives parse trees for the strings $uv^i xy^i z$ at each $i > 1$. Replacing the larger by the smaller generates the string uxz . That establishes condition 1 of the lemma. We now turn to conditions 2 and 3.

To get condition 2, we must be sure that v and y are not both ε . If they were, the parse tree obtained by substituting the smaller subtree for the larger would have fewer nodes than τ does and would still generate s . This result isn't possible because we had already chosen τ to be a parse tree for s with the smallest number of nodes. That is the reason for selecting τ in this way.

In order to get condition 3, we need to be sure that vxy has length at most p . In the parse tree for s the upper occurrence of R generates vxy . We chose R so that both occurrences fall within the bottom $|V| + 1$ variables on the path, and we chose the longest path in the parse tree, so the subtree where R generates vxy is at most $|V| + 1$ high. A tree of this height can generate a string of length at most $b^{|V|+1} = p$.

For some tips on using the pumping lemma to prove that languages are not context free, review the text preceding Example 1.73 (page 80) where we discuss the related problem of proving nonregularity with the pumping lemma for regular languages.

EXAMPLE 2.36

Use the pumping lemma to show that the language $B = \{a^n b^n c^n \mid n \geq 0\}$ is not context free.

We assume that B is a CFL and obtain a contradiction. Let p be the pumping length for B that is guaranteed to exist by the pumping lemma. Select the string $s = a^p b^p c^p$. Clearly s is a member of B and of length at least p . The pumping lemma states that s can be pumped, but we show that it cannot. In other words, we show that no matter how we divide s into $uvxyz$, one of the three conditions of the lemma is violated.

First, condition 2 stipulates that either v or y is nonempty. Then we consider one of two cases, depending on whether substrings v and y contain more than one type of alphabet symbol.

1. When both v and y contain only one type of alphabet symbol, v does not contain both a's and b's or both b's and c's, and the same holds for y . In this case, the string uv^2xy^2z cannot contain equal numbers of a's, b's, and c's. Therefore, it cannot be a member of B . That violates condition 1 of the lemma and is thus a contradiction.
2. When either v or y contains more than one type of symbol, uv^2xy^2z may contain equal numbers of the three alphabet symbols but not in the correct order. Hence it cannot be a member of B and a contradiction occurs.

One of these cases must occur. Because both cases result in a contradiction, a contradiction is unavoidable. So the assumption that B is a CFL must be false. Thus we have proved that B is not a CFL. ■

EXAMPLE 2.37

Let $C = \{a^i b^j c^k \mid 0 \leq i \leq j \leq k\}$. We use the pumping lemma to show that C is not a CFL. This language is similar to language B in Example 2.36, but proving that it is not context free is a bit more complicated.

Assume that C is a CFL and obtain a contradiction. Let p be the pumping length given by the pumping lemma. We use the string $s = a^p b^p c^p$ that we used earlier, but this time we must “pump down” as well as “pump up.” Let $s = uvxyz$ and again consider the two cases that occurred in Example 2.36.

1. When both v and y contain only one type of alphabet symbol, v does not contain both a's and b's or both b's and c's, and the same holds for y . Note that the reasoning used previously in case 1 no longer applies. The reason is that C contains strings with unequal numbers of a's, b's, and c's as long as the numbers are not decreasing. We must analyze the situation more carefully to show that s cannot be pumped. Observe that because v and y contain only one type of alphabet symbol, one of the symbols a, b, or c doesn't appear in v or y . We further subdivide this case into three subcases according to which symbol does not appear.

- a. *The a's do not appear.* Then we try pumping down to obtain the string $uv^0xy^0z = uxz$. That contains the same number of a's as s does, but it contains fewer b's or fewer c's. Therefore, it is not a member of C , and a contradiction occurs.
 - b. *The b's do not appear.* Then either a's or c's must appear in v or y because both can't be the empty string. If a's appear, the string uv^2xy^2z contains more a's than b's, so it is not in C . If c's appear, the string uv^0xy^0z contains more b's than c's, so it is not in C . Either way, a contradiction occurs.
 - c. *The c's do not appear.* Then the string uv^2xy^2z contains more a's or more b's than c's, so it is not in C , and a contradiction occurs.
2. When either v or y contains more than one type of symbol, uv^2xy^2z will not contain the symbols in the correct order. Hence it cannot be a member of C , and a contradiction occurs.

Thus we have shown that s cannot be pumped in violation of the pumping lemma and that C is not context free. ■

EXAMPLE 2.38

Let $D = \{ww \mid w \in \{0,1\}^*\}$. Use the pumping lemma to show that D is not a CFL. Assume that D is a CFL and obtain a contradiction. Let p be the pumping length given by the pumping lemma.

This time choosing string s is less obvious. One possibility is the string 0^p10^p1 . It is a member of D and has length greater than p , so it appears to be a good candidate. But this string *can* be pumped by dividing it as follows, so it is not adequate for our purposes.

$$\overbrace{000 \cdots 000}^{0^p 1} \underbrace{0}_{u} \underbrace{1}_{v} \underbrace{0}_{x} \overbrace{000 \cdots 0001}^{0^p 1} \underbrace{0}_{y} \underbrace{000 \cdots 0001}_{z}$$

Let's try another candidate for s . Intuitively, the string $0^p1^p0^p1^p$ seems to capture more of the “essence” of the language D than the previous candidate did. In fact, we can show that this string does work, as follows.

We show that the string $s = 0^p1^p0^p1^p$ cannot be pumped. This time we use condition 3 of the pumping lemma to restrict the way that s can be divided. It says that we can pump s by dividing $s = uvxyz$, where $|vxy| \leq p$.

First, we show that the substring vxy must straddle the midpoint of s . Otherwise, if the substring occurs only in the first half of s , pumping s up to uv^2xy^2z moves a 1 into the first position of the second half, and so it cannot be of the form ww . Similarly, if vxy occurs in the second half of s , pumping s up to uv^2xy^2z moves a 0 into the last position of the first half, and so it cannot be of the form ww .

But if the substring vxy straddles the midpoint of s , when we try to pump s down to uxz it has the form $0^p1^i0^j1^p$, where i and j cannot both be p . This string is not of the form ww . Thus s cannot be pumped, and D is not a CFL. ■

2.4

DETERMINISTIC CONTEXT-FREE LANGUAGES

As you recall, deterministic finite automata and nondeterministic finite automata are equivalent in language recognition power. In contrast, nondeterministic pushdown automata are more powerful than their deterministic counterparts. We will show that certain context-free languages cannot be recognized by deterministic PDAs—these languages require nondeterministic PDAs. The languages that are recognizable by deterministic pushdown automata (DPDAs) are called deterministic context-free languages (DCFLs). This subclass of the context-free languages is relevant to practical applications, such as the design of parsers in compilers for programming languages, because the parsing problem is generally easier for DCFLs than for CFLs. This section gives a short overview of this important and beautiful subject.

In defining DPDAs, we conform to the basic principle of determinism: at each step of its computation, the DPDA has at most one way to proceed according to its transition function. Defining DPDAs is more complicated than defining DFAs because DPDAs may read an input symbol without popping a stack symbol, and vice versa. Accordingly, we allow ε -moves in the DPDA's transition function even though ε -moves are prohibited in DFAs. These ε -moves take two forms: **ε -input moves** corresponding to $\delta(q, \varepsilon, x)$, and **ε -stack moves** corresponding to $\delta(q, a, \varepsilon)$. A move may combine both forms, corresponding to $\delta(q, \varepsilon, \varepsilon)$. If a DPDA can make an ε -move in a certain situation, it is prohibited from making a move in that same situation that involves processing a symbol instead of ε . Otherwise multiple valid computation branches might occur, leading to nondeterministic behavior. The formal definition follows.

DEFINITION 2.39

A **deterministic pushdown automaton** is a 6-tuple $(Q, \Sigma, \Gamma, \delta, q_0, F)$, where Q , Σ , Γ , and F are all finite sets, and

1. Q is the set of states,
2. Σ is the input alphabet,
3. Γ is the stack alphabet,
4. $\delta: Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \rightarrow (Q \times \Gamma_\varepsilon) \cup \{\emptyset\}$ is the transition function,
5. $q_0 \in Q$ is the start state, and
6. $F \subseteq Q$ is the set of accept states.

The transition function δ must satisfy the following condition. For every $q \in Q$, $a \in \Sigma$, and $x \in \Gamma$, exactly one of the values

$$\delta(q, a, x), \delta(q, a, \varepsilon), \delta(q, \varepsilon, x), \text{ and } \delta(q, \varepsilon, \varepsilon)$$

is not $\emptyset$.

The transition function may output either a single move of the form (r, y) or it may indicate no action by outputting $\emptyset$. To illustrate these possibilities, let's consider an example. Suppose a DPDA M with transition function δ is in state q , has a as its next input symbol, and has symbol x on the top of its stack. If $\delta(q, a, x) = (r, y)$ then M reads a , pops x off the stack, enters state r , and pushes y on the stack. Alternatively, if $\delta(q, a, x) = \emptyset$ then when M is in state q , it has no move that reads a and pops x . In that case, the condition on δ requires that one of $\delta(q, \epsilon, x)$, $\delta(q, a, \epsilon)$, or $\delta(q, \epsilon, \epsilon)$ is nonempty, and then M moves accordingly. The condition enforces deterministic behavior by preventing the DPDA from taking two different actions in the same situation, such as would be the case if both $\delta(q, a, x) \neq \emptyset$ and $\delta(q, a, \epsilon) \neq \emptyset$. A DPDA has exactly one legal move in every situation where its stack is nonempty. If the stack is empty, a DPDA can move only if the transition function specifies a move that pops ϵ . Otherwise the DPDA has no legal move and it rejects without reading the rest of the input.

Acceptance for DPDAs works in the same way it does for PDAs. If a DPDA enters an accept state after it has read the last input symbol of an input string, it accepts that string. In all other cases, it rejects that string. Rejection occurs if the DPDA reads the entire input but doesn't enter an accept state when it is at the end, or if the DPDA fails to read the entire input string. The latter case may arise if the DPDA tries to pop an empty stack or if the DPDA makes an endless sequence of ϵ -input moves without reading the input past a certain point.

The language of a DPDA is called a *deterministic context-free language*.

EXAMPLE 2.40

The language $\{0^n 1^n \mid n \geq 0\}$ in Example 2.14 is a DCFL. We can easily modify its PDA M_1 to be a DPDA by adding transitions for any missing state, input symbol, and stack symbol combinations to a "dead" state from which acceptance isn't possible.

Examples 2.16 and 2.18 give CFLs $\{a^i b^j c^k \mid i, j, k \geq 0 \text{ and } i = j \text{ or } i = k\}$ and $\{ww^R \mid w \in \{0,1\}^*\}$, which are not DCFLs. Problems 2.57 and 2.58 show that nondeterminism is necessary for recognizing these languages. ■

Arguments involving DPDAs tend to be somewhat technical in nature, and though we strive to emphasize the primary ideas behind the constructions, readers may find this section to be more challenging than other sections in the first few chapters. Later material in the book doesn't depend on this section, so it may be skipped if desired.

We'll begin with a technical lemma that will simplify the discussion later on. As noted, DPDAs may reject inputs by failing to read the entire input string, but such DPDAs introduce messy cases. Fortunately, the next lemma shows that we can convert a DPDA into one that avoids this inconvenient behavior.

LEMMA 2.41

Every DPDA has an equivalent DPDA that always reads the entire input string.

PROOF IDEA A DPDA may fail to read the entire input if it tries to pop an empty stack or because it makes an endless sequence of ε -input moves. Call the first situation *hanging* and the second situation *looping*. We solve the hanging problem by initializing the stack with a special symbol. If that symbol is later popped from the stack before the end of the input, the DPDA reads to the end of the input and rejects. We solve the looping problem by identifying the looping situations, i.e., those from which no further input symbol is ever read, and re-programming the DPDA so that it reads and rejects the input instead of looping. We must adjust these modifications to accommodate the case where hanging or looping occurs on the last symbol of the input. If the DPDA enters an accept state at any point after it has read the last symbol, the modified DPDA accepts instead of rejects.

PROOF Let $P = (Q, \Sigma, \Gamma, \delta, q_0, F)$ be a DPDA. First, add a new start state q_{start} , an additional accept state q_{accept} , a new state q_{reject} , as well as other new states as described. Perform the following changes for every $r \in Q$, $a \in \Sigma_\varepsilon$, and $x, y \in \Gamma_\varepsilon$.

First modify P so that, once it enters an accept state, it remains in accepting states until it reads the next input symbol. Add a new accept state q_a for every $q \in Q$. For each $q \in Q$, if $\delta(q, \varepsilon, x) = (r, y)$, set $\delta(q_a, \varepsilon, x) = (r_a, y)$, and then if $q \in F$, also change δ so that $\delta(q, \varepsilon, x) = (r_a, y)$. For each $q \in Q$ and $a \in \Gamma$, if $\delta(q, a, x) = (r, y)$ set $\delta(q_a, a, x) = (r, y)$. Let F' be the set of new and old accept states.

Next, modify P to reject when it tries to pop an empty stack, by initializing the stack with a special new stack symbol $\$$. If P subsequently detects $\$$ while in a non-accepting state, it enters q_{reject} and scans the input to the end. If P detects $\$$ while in an accept state, it enters q_{accept} . Then, if any input remains unread, it enters q_{reject} and scans the input to the end. Formally, set $\delta(q_{\text{start}}, \varepsilon, \varepsilon) = (q_0, \$)$. For $x \in \Gamma$ and $\delta(q, a, x) \neq \emptyset$, if $q \notin F'$ then set $\delta(q, a, \$) = (q_{\text{reject}}, \varepsilon)$, and if $q \in F'$ then set $\delta(q, a, \$) = (q_{\text{accept}}, \varepsilon)$. For $a \in \Sigma$, set $\delta(q_{\text{reject}}, a, \varepsilon) = (q_{\text{reject}}, \varepsilon)$ and $\delta(q_{\text{accept}}, a, \varepsilon) = (q_{\text{reject}}, \varepsilon)$.

Lastly, modify P to reject instead of making an endless sequence of ε -input moves prior to the end of the input. For every $q \in Q$ and $x \in \Gamma$, call (q, x) a *looping situation* if, when P is started in state q with $x \in \Gamma$ on the top of the stack, it never pops anything below x and it never reads an input symbol. Say the looping situation is *accepting* if P enters an accept state during its subsequent moves, and otherwise it is *rejecting*. If (q, x) is an accepting looping situation, set $\delta(q, \varepsilon, x) = (q_{\text{accept}}, \varepsilon)$, whereas if (q, x) is a rejecting looping situation, set $\delta(q, \varepsilon, x) = (q_{\text{reject}}, \varepsilon)$.

For simplicity, we'll assume henceforth that DPDAs read their input to the end.

PROPERTIES OF DCFLS

We'll explore closure and nonclosure properties of the class of DCFLs, and use these to exhibit a CFL that is not a DCFL.

THEOREM 2.42

The class of DCFLs is closed under complementation.

PROOF IDEA Swapping the accept and non-accept states of a DFA yields a new DFA that recognizes the complementary language, thereby proving that the class of regular languages is closed under complementation. The same approach works for DPDAs, except for one problem. The DPDA may accept its input by entering both accept and non-accept states in a sequence of moves at the end of the input string. Interchanging accept and non-accept states would still accept in this case.

We fix this problem by modifying the DPDA to limit when acceptance can occur. For each symbol of the input, the modified DPDA can enter an accept state only when it is about to read the next symbol. In other words, only reading states—states that always read an input symbol—may be accept states. Then, by swapping acceptance and non-acceptance only among these reading states, we invert the output of the DPDA.

PROOF First modify P as described in the proof of Lemma 2.41 and let $(Q, \Sigma, \Gamma, \delta, q_0, F)$ be the resulting machine. This machine always reads the entire input string. Moreover, once enters an accept state, it remains in accept states until it reads the next input symbol.

In order to carry out the proof idea, we need to identify the reading states. If the DPDA in state q reads an input symbol $a \in \Sigma$ without popping the stack, i.e., $\delta(q, a, \epsilon) \neq \emptyset$, designate q to be a reading state. However, if it reads and also pops, the decision to read may depend on the popped symbol, so divide that step into two: a pop and then a read. Thus if $\delta(q, a, x) = (r, y)$ for $a \in \Sigma$ and $x \in \Gamma$, add a new state q_x and modify δ so $\delta(q, \epsilon, x) = (q_x, \epsilon)$ and $\delta(q_x, a, \epsilon) = (r, y)$. Designate q_x to be a reading state. The states q_x never pop the stack, so their action is independent of the stack contents. Assign q_x to be an accept state if $q \in F$. Finally, remove the accepting state designation from any state which isn't a reading state. The modified DPDA is equivalent to P , but it enters an accept state at most once per input symbol, when it is about to read the next symbol.

Now, invert which reading states are classified as accepting. The resulting DPDA recognizes the complementary language.

This theorem implies that some CFLs are not DCFLs. Any CFL whose complement isn't a CFL isn't a DCFL. Thus $A = \{a^i b^j c^k \mid i \neq j \text{ or } j \neq k \text{ where } i, j, k \geq 0\}$ is a CFL but not a DCFL. Otherwise $\bar{A}$ would be a CFL, so the result of Problem 2.18 would incorrectly imply that $\bar{A} \cap a^* b^* c^* = \{a^n b^n c^n \mid n \geq 0\}$ is context free.

Problem 2.53 asks you to show that the class of DCFLs isn't closed under other familiar operations such as union, intersection, star, and reversal.

To simplify arguments, we will occasionally consider *endmarked inputs* whereby the special endmarker symbol $\sqcup$ is appended to the input string. Here we add $\sqcup$ to the DPDA's input alphabet. As we show in the next theorem, adding endmarkers doesn't change the power of DPDAs. However, designing DPDAs on endmarked inputs is often easier because we can take advantage of knowing when the input string ends. For any language A , we write the *endmarked language* $A\sqcup$ to be the collection of strings $w\sqcup$ where $w \in A$.

THEOREM 2.43

A is a DCFL if and only if $A\sqcup$ is a DCFL.

PROOF IDEA Proving the forward direction of this theorem is routine. Say DPDA P recognizes A . Then DPDA P' recognizes $A\sqcup$ by simulating P until P' reads $\sqcup$. At that point, P' accepts if P had entered an accept state during the previous symbol. P' doesn't read any symbols after $\sqcup$.

To prove the reverse direction, let DPDA P recognize $A\sqcup$ and construct a DPDA P' that recognizes A . As P' reads its input, it simulates P . Prior to reading each input symbol, P' determines whether P would accept if that symbol were $\sqcup$. If so, P' enters an accept state. Observe that P may operate the stack after it reads $\sqcup$, so determining whether it accepts after reading $\sqcup$ may depend on the stack contents. Of course, P' cannot afford to pop the entire stack at every input symbol, so it must determine what P would do after reading $\sqcup$, but without popping the stack. Instead, P' stores additional information on the stack that allows P' to determine immediately whether P would accept. This information indicates from which states P would eventually accept while (possibly) manipulating the stack, but without reading further input.

PROOF We give proof details of the reverse direction only. As we described in the proof idea, let DPDA $P = (Q, \Sigma \cup \{\sqcup\}, \Gamma, \delta, q_0, F)$ recognize $A\sqcup$ and construct a DPDA $P' = (Q', \Sigma, \Gamma', \delta', q_0', F')$ that recognizes A . First, modify P so that each of its moves does exactly one of the following operations: read an input symbol; push a symbol onto the stack; or pop a symbol from the stack. Making this modification is straightforward by introducing new states.

P' simulates P , while maintaining a copy of its stack contents interleaved with additional information on the stack. Every time P' pushes one of P 's stack symbols, P' follows that by pushing a symbol that represents a subset of P 's states. Thus we set $\Gamma' = \Gamma \cup \mathcal{P}(Q)$. The stack in P' interleaves members of Γ with members of $\mathcal{P}(Q)$. If $R \in \mathcal{P}(Q)$ is the top stack symbol, then by starting P in any one of R 's states, P will eventually accept without reading any more input.

Initially, P' pushes the set R_0 on the stack, where R_0 contains every state q such that when P is started in q with an empty stack, it eventually accepts without reading any input symbols. Then P' begins simulating P . To simulate a pop move, P' first pops and discards the set of states that appears as the top stack symbol, then it pops again to obtain the symbol that P would have popped at this point, and uses it to determine the next move of P . Simulating a push move $\delta(q, \varepsilon, \varepsilon) = (r, x)$, where P pushes x as it goes from state q to state r , goes as follows. First P' examines the set of states R on the top of its stack, and then it pushes x and after that the set S , where $q \in S$ if $q \in F$ or if $\delta(q, \varepsilon, x) = (r, \varepsilon)$ and $r \in R$. In other words, S is the set of states that are either accepting immediately, or that would lead to a state in R after popping x . Lastly, P' simulates a read move $\delta(q, a, \varepsilon) = (r, \varepsilon)$, by examining the set R on the top of the stack and entering an accept state if $r \in R$. If P' is at the end of the input string when it enters this state, it will accept the input. If it is not at the end of the input string, it will continue simulating P , so this accept state must also record P 's state. Thus we create this state as a second copy of P 's original state, marking it as an accept state in P' .

DETERMINISTIC CONTEXT-FREE GRAMMARS

This section defines deterministic context-free grammars, the counterpart to deterministic pushdown automata. We will show that these two models are equivalent in power, provided that we restrict our attention to endmarked languages, where all strings are terminated with $\#$. Thus the correspondence isn't quite as strong as we saw in regular expressions and finite automata, or in CFGs and PDAs, where the generating model and the recognizing model describe exactly the same class of languages without the need for endmarkers. However, in the case of DPDAs and DCFGs, the endmarkers are necessary because equivalence doesn't hold otherwise.

In a deterministic automaton, each step in a computation determines the next step. The automaton cannot make choices about how it proceeds because only a single possibility is available at every point. To define determinism in a grammar, observe that computations in automata correspond to derivations in grammars. In a deterministic grammar, derivations are constrained, as you will see.

Derivations in CFGs begin with the start variable and proceed “top down” with a series of substitutions according to the grammar's rules, until the derivation obtains a string of terminals. For defining DCFGs we take a “bottom up” approach, by starting with a string of terminals and processing the derivation in reverse, employing a series of reduce steps until reaching the start variable. Each **reduce step** is a reversed substitution, whereby the string of terminals and variables on the right-hand side of a rule is replaced by the variable on the corresponding left-hand side. The string replaced is called the **reducing string**. We call the entire reversed derivation a **reduction**. Deterministic CFGs are defined in terms of reductions that have a certain property.

More formally, if u and v are strings of variables and terminals, write $u \rightarrow v$ to mean that v can be obtained from u by a reduce step. In other words, $u \rightarrow v$ means the same as $v \Rightarrow u$. A **reduction from u to v** is a sequence

$$u = u_1 \rightarrow u_2 \rightarrow \dots \rightarrow u_k = v$$

and we say that u is **reducible to v** , written $u \xrightarrow{*} v$. Thus $u \xrightarrow{*} v$ whenever $v \Rightarrow^* u$. A **reduction from u** is a reduction from u to the start variable. In a **leftmost reduction**, each reducing string is reduced only after all other reducing strings that lie entirely to its left. With a little thought we can see that a leftmost reduction is a rightmost derivation in reverse.

Here's the idea behind determinism in CFGs. In a CFG with start variable S and string w in its language, say that a leftmost reduction of w is

$$w = u_1 \rightarrow u_2 \rightarrow \dots \rightarrow u_k = S.$$

First, we stipulate that every u_i determines the next reduce step and hence u_{i+1} . Thus w determines its entire leftmost reduction. This requirement implies only that the grammar is unambiguous. To get determinism, we need to go further. In each u_i , the next reduce step must be uniquely determined by the prefix of u_i up through and including the reducing string h of that reduce step. In other words, the leftmost reduce step in u_i doesn't depend on the symbols in u_i to the right of its reducing string.

Introducing terminology will help us make this idea precise. Let w be a string in the language of CFG G , and let u_i appear in a leftmost reduction of w . In the reduce step $u_i \rightarrow u_{i+1}$, say that rule $T \rightarrow h$ was applied in reverse. That means we can write $u_i = xhy$ and $u_{i+1} = xTy$, where h is the reducing string, x is the part of u_i that appears leftward of h , and y is the part of u_i that appears rightward of h . Pictorially,

$$u_i = \overbrace{x_1 \cdots x_j}^x \overbrace{h_1 \cdots h_k}^h \overbrace{y_1 \cdots y_l}^y \rightarrow \overbrace{x_1 \cdots x_j}^x \overbrace{T}^T \overbrace{y_1 \cdots y_l}^y = u_{i+1}.$$

FIGURE 2.44

Expanded view of $xhy \rightarrow xTy$

We call h , together with its reducing rule $T \rightarrow h$, a **handle** of u_i . In other words, a handle of a string u_i that appears in a leftmost reduction of $w \in L(G)$ is the occurrence of the reducing string in u_i , together with the reducing rule for u_i in this reduction. Occasionally we associate a handle with its reducing string only, when we aren't concerned with the reducing rule. A string that appears in a leftmost reduction of some string in $L(G)$ is called a **valid string**. We define handles only for valid strings.

A valid string may have several handles, but only if the grammar is ambiguous. Unambiguous grammars may generate strings by one parse tree only, and

therefore the leftmost reductions, and hence the handles, are also unique. In that case, we may refer to *the* handle of a valid string.

Observe that y , the portion of u_i following a handle, is always a string of terminals because the reduction is leftmost. Otherwise, y would contain a variable symbol and that could arise only from a previous reduce step whose reducing string was completely to the right of h . But then the leftmost reduction should have reduced the handle at an earlier step.

EXAMPLE 2.45

Consider the grammar G_1 :

$$\begin{aligned} R &\rightarrow S \mid T \\ S &\rightarrow aSb \mid ab \\ T &\rightarrow aTbb \mid abb \end{aligned}$$

Its language is $B \cup C$ where $B = \{a^m b^m \mid m \geq 1\}$ and $C = \{a^m b^{2m} \mid m \geq 1\}$. In this leftmost reduction of the string $aaabbb \in L(G_1)$, we've underlined the handle at each step:

$$aa\underline{ab}bb \rightarrow aa\underline{aSb}b \rightarrow \underline{aSb} \rightarrow \underline{S} \rightarrow R.$$

Similarly, this is a leftmost reduction of the string $aaabbbbbb$:

$$aa\underline{ab}bbbbb \rightarrow aa\underline{aTbbb}b \rightarrow \underline{aTbb} \rightarrow \underline{T} \rightarrow R.$$

In both cases, the leftmost reduction shown happens to be the only reduction possible; but in other grammars where several reductions may occur, we must use a leftmost reduction to define the handles. Notice that the handles of $aaabbb$ and $aaabbbbbb$ are unequal, even though the initial parts of these strings agree. We'll discuss this point in more detail shortly when we define DCFGs.

A PDA can recognize $L(G_1)$ by using its nondeterminism to guess whether its input is in B or in C . Then, after it pushes the a 's on the stack, it pops the a 's and matches each one with b or bb accordingly. Problem 2.55 asks you to show that $L(G_1)$ is not a DCFL. If you try to make a DPDA that recognizes this language, you'll see that the machine cannot know in advance whether the input is in B or in C so it doesn't know how to match the a 's with the b 's. Contrast this grammar with grammar G_2 :

$$\begin{aligned} R &\rightarrow 1S \mid 2T \\ S &\rightarrow aSb \mid ab \\ T &\rightarrow aTbb \mid abb \end{aligned}$$

where the first symbol in the input provides this information. Our definition of DCFGs must include G_2 yet exclude G_1 . ■

EXAMPLE 2.46

Let G_3 be the following grammar:

$$\begin{aligned} S &\rightarrow T\text{--}\vdash \\ T &\rightarrow T(T) \mid \varepsilon \end{aligned}$$

This grammar illustrates several points. First, it generates an endmarked language. We will focus on endmarked languages later on when we prove the equivalence between DPDAs and DCFGs. Second, ε handles may occur in reductions, as indicated with short underscores in the leftmost reduction of the string $()() \text{--}\vdash$:

$$\text{.}()() \text{--}\vdash \rightarrow T\text{.}()() \text{--}\vdash \rightarrow \underline{T(T)}() \text{--}\vdash \rightarrow T\text{.}() \text{--}\vdash \rightarrow \underline{T(T)} \text{--}\vdash \rightarrow \underline{T\text{--}\vdash} \rightarrow S.$$

Handles play an important role in defining DCFGs because handles determine reductions. Once we know the handle of a string, we know the next reduce step. To make sense of the coming definition, keep our goal in mind: we aim to define DCFGs so that they correspond to DPDAs. We'll establish that correspondence by showing how to convert DCFGs to equivalent DPDAs, and vice versa. For this conversion to work, the DPDA needs to find handles so that it can find reductions. But finding a handle may be tricky. It seems that we need to know a string's next reduce step to identify its handle, but a DPDA doesn't know the reduction in advance. We'll solve this by restricting handles in a DCFG so that the DPDA can find them more easily.

To motivate the definition, consider ambiguous grammars, where some strings have several handles. Selecting a specific handle may require advance knowledge of which parse tree derives the string, information that is certainly unavailable to the DPDA. We'll see that DCFGs are unambiguous so handles are unique. However, uniqueness alone is unsatisfactory for defining DCFGs as grammar G_1 in Example 2.45 shows.

Why don't unique handles imply that we have a DCFG? The answer is evident by examining the handles in G_1 . If $w \in B$, the handle is ab , whereas if $w \in C$, the handle is abb . Though w determines which of these cases applies, discovering which of ab or abb is the handle may require examining all of w , and a DPDA hasn't read the entire input when it needs to select the handle.

In order to define DCFGs that correspond to DPDAs, we impose a stronger requirement on the handles. The initial part of a valid string, up to and including its handle, must be sufficient to determine the handle. Thus, if we are reading a valid string from left to right, as soon as we read the handle we know we have it. We don't need to read beyond the handle in order to identify the handle. Recall that the unread part of the valid string contains only terminals because the valid string has been obtained by a leftmost reduction of an initial string of terminals, and the unread part hasn't been processed yet. Accordingly, we say that a handle h of a valid string $v = xhy$ is a **forced handle** if h is the unique handle in *every* valid string $xh\hat{y}$ where $\hat{y} \in \Sigma^*$.

DEFINITION 2.47

A *deterministic context-free grammar* is a context-free grammar such that every valid string has a forced handle.

For simplicity, we'll assume throughout this section on deterministic context-free languages that the start variable of a CFG doesn't appear on the right-hand side of any rule and that every variable in a grammar appears in a reduction of some string in the grammar's language, i.e., grammars contain no useless variables.

Though our definition of DCFGs is mathematically precise, it doesn't give any obvious way to determine whether a CFG is deterministic. Next we'll present a procedure to do exactly that, called the *DK*-test. We'll also use the construction underlying the *DK*-test to enable a DPDA to find handles, when we show how to convert a DCFG to a DPDA.

The *DK*-test relies on one simple but surprising fact. For any CFG G we can construct an associated DFA DK that can identify handles. Specifically, DK accepts its input z if

1. z is the prefix of some valid string $v = zy$, and
2. z ends with a handle of v .

Moreover, each accept state of DK indicates the associated reducing rule(s). In a general CFG, multiple reducing rules may apply, depending on which valid v extends z . But in a DCFG, as we'll see, each accept state corresponds to exactly one reducing rule.

We will describe the *DK*-test after we've presented DK formally and established its properties, but here's the plan. In a DCFG, all handles are forced. Thus if zy is a valid string with a prefix z that ends in a handle of zy , that handle is unique, and it is also the handle for all valid strings $z\hat{y}$. For these properties to hold, each of DK 's accept states must be associated with a single handle and hence with a single applicable reducing rule. Moreover, the accept state must not have an outgoing path that leads to an accept state by reading a string in Σ^* . Otherwise, the handle of zy would not be unique or it would depend on y . In the *DK*-test, we construct DK and then conclude that G is deterministic if all of its accept states have these properties.

To construct DFA DK , we'll construct an equivalent NFA K and convert K to DK ¹ via the subset construction introduced in Theorem 1.39. To understand K , first consider an NFA J that performs a simpler task. It accepts every input string that ends with the right-hand side of any rule. Constructing J is easy. It guesses which rule to use and it also guesses the point at which to start matching the input with that rule's right-hand side. As it matches the input, J keeps track

¹The name *DK* is a mnemonic for "deterministic K " but it also stands for Donald Knuth, who first proposed this idea.

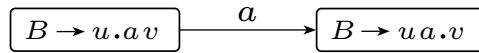
of its progress through the chosen right-hand side. We represent this progress by placing a dot in the corresponding point in the rule, yielding a *dotted rule*, also called an *item* in some other treatments of this material. Thus for each rule $B \rightarrow u_1 u_2 \cdots u_k$ with k symbols on the right-hand side, we get $k + 1$ dotted rules:

$$\begin{aligned} B &\rightarrow \cdot u_1 u_2 \cdots u_k \\ B &\rightarrow u_1 \cdot u_2 \cdots u_k \\ &\vdots \\ B &\rightarrow u_1 u_2 \cdots \cdot u_k \\ B &\rightarrow u_1 u_2 \cdots u_k \cdot \end{aligned}$$

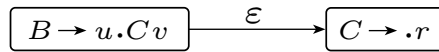
Each of these dotted rules corresponds to one state of J . We indicate the state associated with the dotted rule $B \rightarrow u.v$ with a box around it, $\boxed{B \rightarrow u.v}$. The accept states $\boxed{\boxed{B \rightarrow u \cdot}}$ correspond to the *completed rules* that have the dot at the end. We add a separate start state with a self-loop on all symbols and an ϵ -move to $\boxed{B \rightarrow \cdot u}$ for each rule $B \rightarrow u$. Thus J accepts if the match completes successfully at the end of the input. If a mismatch occurs or if the end of the match doesn't coincide with the end of the input, this branch of J 's computation rejects.

NFA K operates similarly, but it is more judicious about choosing a rule for matching. Only potential reducing rules are allowed. Like J , its states correspond to all dotted rules. It has a special start state that has an ϵ -move to $\boxed{S_1 \rightarrow \cdot u}$ for every rule involving the start variable S_1 . On each branch of its computation, K matches a potential reducing rule with a substring of the input. If that rule's right-hand side contains a variable, K may nondeterministically switch to some rule that expands that variable. Lemma 2.48 formalizes this idea. First we describe K in detail.

The transitions come in two varieties: shift-moves and ϵ -moves. The shift-moves appear for every a that is a terminal or variable, and every rule $B \rightarrow uav$:



The ϵ -moves appear for all rules $B \rightarrow uCv$ and $C \rightarrow r$:



The accept states are all $\boxed{\boxed{B \rightarrow u \cdot}}$ corresponding to a completed rule. Accept states have no outgoing transitions and are written with a double box.

The next lemma and its corollary prove that K accepts all strings z that end with handles for some valid extension of z . Because K is nondeterministic, we say that it “may” enter a state to mean that K does enter that state on some branch of its nondeterminism.

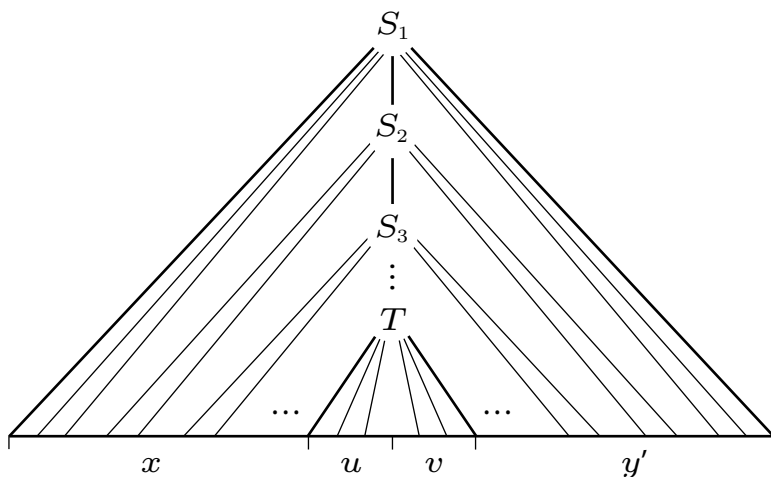
LEMMA 2.48

K may enter state $\boxed{T \rightarrow u.v}$ on reading input z iff $z = xu$ and $xuvy$ is a valid string with handle uv and reducing rule $T \rightarrow uv$, for some $y \in \Sigma^*$.

PROOF IDEA K operates by matching a selected rule's right-hand side with a portion of the input. If that match completes successfully, it accepts. If that right-hand side contains a variable C , either of two situations may arise. If C is the next input symbol, then matching the selected rule simply continues. If C has been expanded, the input will contain symbols derived from C , so K nondeterministically selects a substitution rule for C and starts matching from the beginning of the right-hand side of that rule. It accepts when the right-hand side of the currently selected rule has been matched completely.

PROOF First we prove the forward direction. Assume that K on w enters $\boxed{T \rightarrow u.v}$. Examine K 's path from its start state to $\boxed{T \rightarrow u.v}$. Think of the path as runs of shift-moves separated by ε -moves. The shift-moves are transitions between states sharing the same rule, shifting the dot rightward over symbols read from the input. In the i^{th} run, say that the rule is $S_i \rightarrow u_i S_{i+1} v_i$, where S_{i+1} is the variable expanded in the next run. The penultimate run is for rule $S_l \rightarrow u_l T v_l$, and the final run has rule $T \rightarrow uv$.

Input z must then equal $u_1 u_2 \dots u_l u = xu$ because the strings u_i and u were the shift-move symbols read from the input. Letting $y' = v_l \dots v_2 v_1$, we see that $xuvy'$ is derivable in G because the rules above give the derivation as shown in the parse tree illustrated in Figure 2.49.

**FIGURE 2.49**

Parse tree leading to $xuvy'$

To obtain a valid string, fully expand all variables that appear in y' until each variable derives some string of terminals, and call the resulting string y . The string $xuvy$ is valid because it occurs in a leftmost reduction of $w \in L(G)$, a string of terminals obtained by fully expanding all variables in $xuvy$.

As is evident from the figure below, uv is the handle in the reduction and its reducing rule is $T \rightarrow uv$.

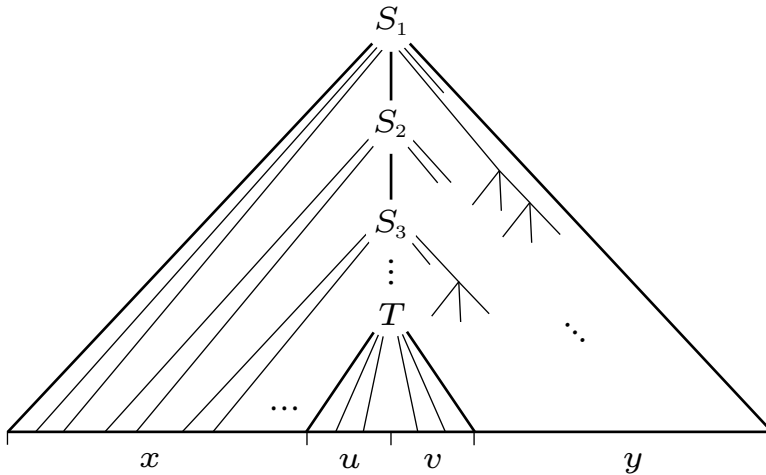


FIGURE 2.50

Parse tree leading to valid string $xuvy$ with handle uv

Now we prove the reverse direction of the lemma. Assume that string $xuvy$ is a valid string with handle uv and reducing rule $T \rightarrow uv$. Show that K on input xu may enter state $\boxed{T \rightarrow u.v}$.

The parse tree for $xuvy$ appears in the preceding figure. It is rooted at the start variable S_1 and it must contain the variable T because $T \rightarrow uv$ is the first reduce step in the reduction of $xuvy$. Let $S_2, \dots, S_l$ be the variables on the path from S_1 to T as shown. Note that all variables in the parse tree that appear leftward of this path must be unexpanded, or else uv wouldn't be the handle.

In this parse tree, each S_i leads to S_{i+1} by some rule $S_i \rightarrow u_i S_{i+1} v_i$. Thus the grammar must contain the following rules for some strings u_i and v_i .

$$\begin{aligned} S_1 &\rightarrow u_1 S_2 v_1 \\ S_2 &\rightarrow u_2 S_3 v_2 \\ &\vdots \\ S_l &\rightarrow u_l T v_l \\ T &\rightarrow uv \end{aligned}$$

K contains the following path from its start state to state $\boxed{T \rightarrow u.v}$ on reading input $z = xu$. First, K makes an ϵ -move to $\boxed{S_1 \rightarrow .u_1 S_2 v_1}$. Then, while reading the symbols of u_1 , it performs the corresponding shift-moves until it enters $\boxed{S_1 \rightarrow u_1 . S_2 v_1}$ at the end of u_1 . Then it makes an ϵ -move to $\boxed{S_2 \rightarrow .u_2 S_3 v_2}$ and continues with shift-moves on reading u_2 until it reaches $\boxed{S_2 \rightarrow u_2 . S_3 v_2}$ and so on. After reading u_i it enters $\boxed{S_i \rightarrow u_i . T v_i}$ which leads by an ϵ -move to $\boxed{T \rightarrow .uv}$ and finally after reading u it is in $\boxed{T \rightarrow u.v}$.

The following corollary shows that K accepts all strings ending with a handle of some valid extension. It follows from Lemma 2.48 by taking $u = h$ and $v = \epsilon$.

COROLLARY 2.51

K may enter accept state $\boxed{T \rightarrow h.}$ on input z iff $z = xh$ and h is a handle of some valid string xhy with reducing rule $T \rightarrow h$.

Finally, we convert NFA K to DFA DK by using the subset construction in the proof of Theorem 1.39 on page 55 and then removing all states that are unreachable from the start state. Each of DK 's states thus contains one or more dotted rules. Each accept state contains at least one completed rule. We can apply Lemma 2.48 and Corollary 2.51 to DK by referring to the states that contain the indicated dotted rules.

Now we are ready to describe the ***DK-test***.

Starting with a CFG G , construct the associated DFA DK . Determine whether G is deterministic by examining DK 's accept states. The ***DK-test*** stipulates that every accept state contains

1. exactly one completed rule, and
2. no dotted rule in which a terminal symbol immediately follows the dot, i.e., no dotted rule of the form $B \rightarrow u.av$ for $a \in \Sigma$.

THEOREM 2.52

G passes the ***DK-test*** iff G is a DCFG.

PROOF IDEA We'll show that the ***DK-test*** passes if and only if all handles are forced. Equivalently, the test fails iff some handle isn't forced. First, suppose that some valid string has an unforced handle. If we run DK on this string, Corollary 2.51 says that DK enters an accept state at the end of the handle. The ***DK-test*** fails because that accept state has either a second completed rule or an outgoing path leading to an accept state, where the outgoing path begins with a terminal symbol. In the latter case, the accept state would contain a dotted rule with a terminal symbol following the dot.

Conversely, if the *DK*-test fails because an accept state has two completed rules, extend the associated string to two valid strings with differing handles at that point. Similarly, if it has a completed rule and a dotted rule with a terminal following the dot, employ Lemma 2.48 to get two valid extensions with differing handles at that point. Constructing the valid extension corresponding to the second rule is a bit delicate.

PROOF Start with the forward direction. Assume that G isn't deterministic and show that it fails the *DK*-test. Take a valid string xhy that has an unforced handle h . Hence some valid string xhy' has a different handle $\hat{h} \neq h$, where y' is a string of terminals. We can thus write xhy' as $xhy' = \hat{x}\hat{h}\hat{y}$.

If $xh = \hat{x}\hat{h}$, the reducing rules differ because h and $\hat{h}$ aren't the same handle. Therefore, input xh sends *DK* to a state that contains two completed rules, a violation of the *DK*-test.

If $xh \neq \hat{x}\hat{h}$, one of these extends the other. Assume that xh is the proper prefix of $\hat{x}\hat{h}$. The argument is the same with the strings interchanged and y in place of y' , if $\hat{x}\hat{h}$ is the shorter string. Let q be the state that *DK* enters on input xh . State q must be accepting because h is a handle of xhy . A transition arrow must exit q because $\hat{x}\hat{h}$ sends *DK* to an accept state via q . Furthermore, that transition arrow is labeled with a terminal symbol, because $y' \in \Sigma^+$. Here $y' \neq \epsilon$ because $\hat{x}\hat{h}$ extends xh . Hence q contains a dotted rule with a terminal symbol immediately following the dot, violating the *DK*-test.

To prove the reverse direction, assume G fails the *DK*-test at some accept state q , and show that G isn't deterministic by exhibiting an unforced handle. Because q is accepting, it has a completed rule $T \rightarrow h..$ Let z be a string that leads *DK* to q . Then $z = xh$ where some valid string xhy has handle h with reducing rule $T \rightarrow h$, for $y \in \Sigma^*$. Now we consider two cases, depending on how the *DK*-test fails.

First, say q has another completed rule $B \rightarrow \hat{h}..$ Then some valid string xhy' must have a different handle $\hat{h}$ with reducing rule $B \rightarrow \hat{h}$. Therefore, h isn't a forced handle.

Second, say q contains a rule $B \rightarrow u.av$ where $a \in \Sigma$. Because xh takes *DK* to q , we have $xh = \hat{x}u$, where $\hat{x}uav\hat{y}$ is valid and has a handle uav with reducing rule $B \rightarrow uav$, for some $\hat{y} \in \Sigma^*$. To show that h is unforced, fully expand all variables in v to get the result $v' \in \Sigma^*$, then let $y' = av'\hat{y}$ and notice that $y' \in \Sigma^*$. The following leftmost reduction shows that xhy' is a valid string and h is not the handle.

$$xhy' = xhav'\hat{y} = \hat{x}uav'\hat{y} \xrightarrow{*} \hat{x}uav\hat{y} \rightarrow \hat{x}B\hat{y} \xrightarrow{*} S$$

where S is the start variable. We know that $\hat{x}uav\hat{y}$ is valid and we can obtain $\hat{x}uav'\hat{y}$ from it by using a rightmost derivation so $\hat{x}uav'\hat{y}$ is also valid. Moreover, the handle of $\hat{x}uav'\hat{y}$ either lies inside v' (if $v \neq v'$) or is uav (if $v = v'$). In either case, the handle includes a and thus cannot be h because h fully precedes a . Hence h isn't a forced handle.

When building the DFA DK in practice, a direct construction may be faster than first constructing the NFA K . Begin by adding a dot at the initial point in all rules involving the start variable and place these now-dotted rules into DK 's start state. If a dot precedes a variable C in any of these rules, place dots at the initial position in all rules that have C on the left-hand side and add these rules to the state, continuing this process until no new dotted rules are obtained. For any symbol c that follows a dot, add an outgoing edge labeled c to a new state containing the dotted rules obtained by shifting the dot across the c in any of the dotted rules where the dot precedes the c , and add rules corresponding to the rules where a dot precedes a variable as before.

EXAMPLE 2.53

Here we illustrate how the DK -test fails for the following grammar.

$$\begin{aligned} S &\rightarrow E \neg \\ E &\rightarrow E + T \mid T \\ T &\rightarrow T \times a \mid a \end{aligned}$$

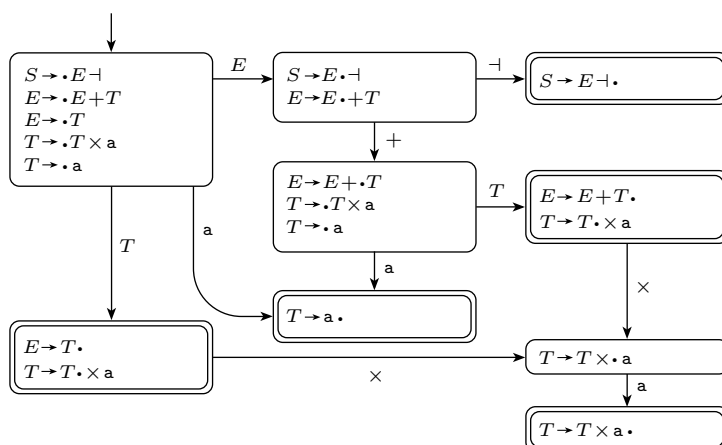


FIGURE 2.54

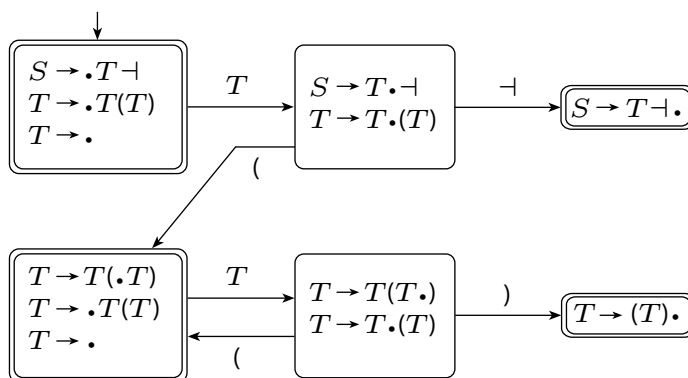
Example of a failed DK -test

Notice the two problematic states at the lower left and the second from the top right, where an accept state contains a dotted rule where a terminal symbol follows the dot. ■

EXAMPLE 2.55

Here is the DFA DK showing that the grammar below is a DCFG.

$$\begin{aligned} S &\rightarrow T\text{--} \\ T &\rightarrow T(T) \mid \varepsilon \end{aligned}$$

**FIGURE 2.56**

Example of a DK -test that passes

Observe that all accept states satisfy the DK -test conditions. ■

RELATIONSHIP OF DPDAS AND DCFGs

In this section we will show that DPDAs and DCFGs describe the same class of endmarked languages. First, we will demonstrate how to convert DCFGs to equivalent DPDAs. This conversion works in all cases. Second, we will show how to do the reverse conversion, from DPDAs to equivalent DCFGs. The latter conversion works only for endmarked languages. We restrict the equivalence to endmarked languages, because the models are not equivalent without this restriction. We showed earlier that endmarkers don't affect the class of languages that DPDAs recognize, but they do affect the class of languages that DCFGs generate. Without endmarkers, DCFGs generate only a subclass of the DCFGs—those that are prefix-free (see Problem 2.52). Note that every endmarked language is prefix-free.

THEOREM 2.57

An endmarked language is generated by a deterministic context-free grammar if and only if it is deterministic context free.

We have two directions to prove. First we will show that every DCFG has an equivalent DPDA. Then we will show that every DPDA that recognizes an end-marked language has an equivalent DCFG. We handle these two directions in separate lemmas.

LEMMA 2.58

Every DCFG has an equivalent DPDA.

PROOF IDEA We show how to convert a DCFG G to an equivalent DPDA P . P uses the DFA DK to operate as follows. It simulates DK on the symbols it reads from the input until DK accepts. As shown in the proof of Theorem 2.52, DK 's accept state indicates a specific dotted rule because G is deterministic, and that rule identifies a handle for some valid string extending the input it has seen so far. Moreover, this handle applies to *every* valid extension because G is deterministic, and in particular it will apply to the full input to P , if that input is in $L(G)$. So P can use this handle to identify the first reduce step for its input string, even though it has read only a part of its input at this point.

How does P identify the second and subsequent reduce steps? One idea is to perform the reduce step directly on the input string, and then run the modified input through DK as we did above. But the input can be neither modified nor reread so this idea doesn't work. Another approach would be to copy the input to the stack and carry out the reduce step there, but then P would need to pop the entire stack to run the modified input through DK and so the modified input would not remain available for later steps.

The trick here is to store the states of DK on the stack, instead of storing the input string there. Every time P reads an input symbol and simulates a move in DK , it records DK 's state by pushing it on the stack. When it performs a reduce step using reducing rule $T \rightarrow u$, it pops $|u|$ states off the stack, revealing the state DK was in prior to reading u . It resets DK to that state, then simulates it on input T and pushes the resulting state on the stack. Then P proceeds by reading and processing input symbols as before.

When P pushes the start variable on the stack, it has found a reduction of its input to the start variable, so it enters an accept state.

Next we prove the other direction of Theorem 2.57.

LEMMA 2.59

Every DPDA that recognizes an endmarked language has an equivalent DCFG.

PROOF IDEA This proof is a modification of the construction in Lemma 2.27 on page 121 that describes the conversion of a PDA P to an equivalent CFG G .

Here P and G are deterministic. In the proof idea for Lemma 2.27, we altered P to empty its stack and enter a specific accept state q_{accept} when it accepts. A PDA cannot directly determine that it is at the end of its input, so P uses its nondeterminism to guess that it is in that situation. We don't want to introduce nondeterminism in constructing DPDA P . Instead we use the assumption that $L(P)$ is endmarked. We modify P to empty its stack and enter q_{accept} when it enters one of its original accept states after it has read the endmarker $\sqcup$.

Next we apply the grammar construction to obtain G . Simply applying the original construction to a DPDA produces a nearly deterministic grammar because the CFG's derivations closely correspond to the DPDA's computations. That grammar fails to be deterministic in one minor, fixable way.

The original construction introduces rules of the form $A_{pq} \rightarrow A_{pr}A_{rq}$ and these may cause ambiguity. These rules cover the case where A_{pq} generates a string that takes P from state p to state q with its stack empty at both ends, and the stack empties midway. The substitution corresponds to dividing the computation at that point. But if the stack empties several times, several divisions are possible. Each of these divisions yields different parse trees, so the resulting grammar is ambiguous. We fix this problem by modifying the grammar to divide the computation only at the very last point where the stack empties midway, thereby removing this ambiguity. For illustration, a similar but simpler situation occurs in the ambiguous grammar

$$\begin{aligned} S &\rightarrow T\sqcup \\ T &\rightarrow TT \mid (T) \mid \epsilon \end{aligned}$$

which is equivalent to the unambiguous, and deterministic, grammar

$$\begin{aligned} S &\rightarrow T\sqcup \\ T &\rightarrow T(T) \mid \epsilon. \end{aligned}$$

Next we show the modified grammar is deterministic by using the *DK*-test. The grammar is designed to simulate the DPDA. As we proved in Lemma 2.27, A_{pq} generates exactly those strings on which P goes from state p on empty stack to state q on empty stack. We'll prove G 's determinism using P 's determinism so we will find it useful to define P 's computation on valid strings to observe its action on handles. Then we can use P 's deterministic behavior to show that handles are forced.

PROOF Say that $P = (Q, \Sigma, \Gamma, \delta, q_0, \{q_{\text{accept}}\})$ and construct G . The start variable is $A_{q_0, q_{\text{accept}}}$. The construction on page 121 contains parts 1, 2, and 3, repeated here for convenience.

1. For each $p, q, r, s \in Q$, $u \in \Gamma$, and $a, b \in \Sigma_\varepsilon$, if $\delta(p, a, \varepsilon)$ contains (r, u) and $\delta(s, b, u)$ contains (q, ε) , put the rule $A_{pq} \rightarrow aA_{rs}b$ in G .
2. For each $p, q, r \in Q$, put the rule $A_{pq} \rightarrow A_{pr}A_{rq}$ in G .
3. For each $p \in Q$, put the rule $A_{pp} \rightarrow \varepsilon$ in G .

We modify the construction to avoid introducing ambiguity, by combining rules of types 1 and 2 into a single type 1-2 rule that achieves the same effect.

- 1-2. For each $p, q, r, s, t \in Q$, $u \in \Gamma$, and $a, b \in \Sigma_\varepsilon$, if $\delta(r, a, \varepsilon) = (s, u)$ and $\delta(t, b, u) = (q, \varepsilon)$, put the rule $A_{pq} \rightarrow A_{pr}aA_{st}b$ in G .

To see that the modified grammar generates the same language, consider any derivation in the original grammar. For each substitution due to a type 2 rule $A_{pq} \rightarrow A_{pr}A_{rq}$, we can assume that r is P 's state when it is at the rightmost point where the stack becomes empty midway by modifying the proof of Claim 2.31 on page 123 to select r in this way. Then the subsequent substitution of A_{rq} must expand it using a type 1 rule $A_{rq} \rightarrow aA_{st}b$. We can combine these two substitutions into a single type 1-2 rule $A_{pq} \rightarrow A_{pr}aA_{st}b$.

Conversely, in a derivation using the modified grammar, if we replace each type 1-2 rule $A_{pq} \rightarrow A_{pr}aA_{st}b$ by the type 2 rule $A_{pq} \rightarrow A_{pr}A_{rq}$ followed by the type 1 rule $A_{rq} \rightarrow aA_{st}b$, we get the same result.

Now we use the *DK*-test to show that G is deterministic. To do that, we'll analyze how P operates on valid strings by extending its input alphabet and transition function to process variable symbols in addition to terminal symbols. We add all symbols A_{pq} to P 's input alphabet and we extend its transition function δ by defining $\delta(p, A_{pq}, \varepsilon) = (q, \varepsilon)$. Set all other transitions involving A_{pq} to $\emptyset$. To preserve P 's deterministic behavior, if P reads A_{pq} from the input then disallow an ε -input move.

The following claim applies to a derivation of any string w in $L(G)$ such as

$$A_{q_0, q_{\text{accept}}} = v_0 \Rightarrow v_1 \Rightarrow \cdots \Rightarrow v_i \Rightarrow \cdots \Rightarrow v_k = w.$$

CLAIM 2.60

If P reads v_i containing a variable A_{pq} , it enters state p just prior to reading A_{pq} .

The proof uses induction on i , the number of steps to derive v_i from $A_{q_0, q_{\text{accept}}}$.

Basis: $i = 0$.

In this case, $v_i = A_{q_0, q_{\text{accept}}}$ and P starts in state q_0 so the basis is true.

Induction step: Assume the claim for i and prove it for $i + 1$.

First consider the case where $v_i = xA_{pq}y$ and A_{pq} is the variable substituted in the step $v_i \Rightarrow v_{i+1}$. The induction hypothesis implies that P enters state p after it reads x , prior to reading symbol A_{pq} . According to G 's construction the substitution rules may be of two types:

1. $A_{pq} \rightarrow A_{pr}aA_{st}b$ or
2. $A_{pp} \rightarrow \varepsilon$.

Thus either $v_{i+1} = xA_{pr}aA_{st}by$ or $v_{i+1} = xy$, depending on which type of rule was used. In the first case, when P reads $A_{pr}aA_{st}b$ in v_{i+1} , we know it starts in state p , because it has just finished reading x . As P reads $A_{pr}aA_{st}b$ in v_{i+1} , it enters the sequence of states r , s , t , and q , due to the substitution rule's construction. Therefore, it enters state p just prior to reading A_{pr} and it enters state s just prior to reading A_{st} , thereby establishing the claim for these two occurrences of variables. The claim holds on occurrences of variables in the y part because, after P reads b it enters state q and then it reads string y . On input v_i , it also enters q just before reading y , so the computations agree on the y parts of v_i and v_{i+1} . Obviously, the computations agree on the x parts. Therefore, the claim holds for v_{i+1} . In the second case, no new variables are introduced, so we only need to observe that the computations agree on the x and y parts of v_i and v_{i+1} . This proves the claim.

CLAIM 2.61

G passes the DK -test.

We show that each of DK 's accept states satisfies the DK -test requirements. Select one of these accept states. It contains a completed rule R . This completed rule may have one of two forms:

1. $A_{pq} \rightarrow A_{pr}aA_{st}b$.
2. $A_{pp} \rightarrow \cdot$.

In both situations, we need to show that the accept state cannot contain

- a. another completed rule, and
- b. a dotted rule that has a terminal symbol immediately after the dot.

We consider each of these four cases separately. In each case, we start by considering a string z on which DK goes to the accept state we selected above.

Case 1a. Here R is a completed type 1-2 rule. For any rule in this accept state, z must end with the symbols preceding the dot in that rule because DK goes to that state on z . Hence the symbols preceding the dot must be consistent in all such rules. These symbols are $A_{pr}aA_{st}b$ in R so any other type 1-2 completed rule must have exactly the same symbols on the right-hand side. It follows that the variables on the left-hand side must also agree, so the rules must be the same.

Suppose the accept state contains R and some type 3 completed ε -rule T . From R we know that z ends with $A_{pr}aA_{st}b$. Moreover, we know that P pops its stack at the very end of z because a pop occurs at that point in R , due to G 's construction. According to the way we build DK , a completed ε -rule in a state must derive from a dotted rule that resides in the same state, where the dot isn't at the very beginning and the dot immediately precedes some variable. (An

exception occurs at DK 's start state, where this dot may occur at the beginning of the rule, but this accept state cannot be the start state because it contains a completed type 1-2 rule.) In G , that means T derives from a type 1-2 dotted rule where the dot precedes the second variable. From G 's construction a push occurs just before the dot. This implies that P does a push move at the very end of z , contradicting our previous statement. Thus the completed ε -rule T cannot exist. Either way, a second completed rule of either type cannot occur in this accept state.

Case 2a. Here R is a completed ε -rule $A_{pp} \rightarrow \cdot$. We show that no other completed ε -rule $A_{qq} \rightarrow \cdot$ can coexist with R . If it does, the preceding claim shows that P must be in p after reading z and it must also be in q after reading z . Hence $p = q$ and therefore the two completed ε -rules are the same.

Case 1b. Here R is a completed type 1-2 rule. From Case 1a, we know that P pops its stack at the end of z . Suppose the accept state also contains a dotted rule T where a terminal symbol immediately follows the dot. From T we know that P doesn't pop its stack at the end of z . This contradiction shows that this situation cannot arise.

Case 2b. Here R is a completed ε -rule. Assume that the accept state also contains a dotted rule T where a terminal symbol immediately follows the dot. Because T is of type 1-2, a variable symbol immediately precedes the dot, and thus z ends with that variable symbol. Moreover, after P reads z it is prepared to read a non- ε input symbol because a terminal follows the dot. As in Case 1a, the completed ε -rule R derives from a type 1-2 dotted rule S where the dot immediately precedes the second variable. (Again this accept state cannot be DK 's start state because the dot doesn't occur at the beginning of T .) Thus some symbol $\hat{a} \in \Sigma_\varepsilon$ immediately precedes the dot in S and so z ends with $\hat{a}$. Either $\hat{a} \in \Sigma$ or $\hat{a} = \varepsilon$, but because z ends with a variable symbol, $\hat{a} \notin \Sigma$ so $\hat{a} = \varepsilon$. Therefore, after P reads z but before it makes the ε -input move to process $\hat{a}$, it is prepared to read an ε input. We also showed above that P is prepared to read a non- ε input symbol at this point. But a DPDA isn't allowed to make both an ε -input move and a move that reads a non- ε input symbol at a given state and stack, so the above situation is impossible. Thus this situation cannot occur.

.....

PARSING AND LR(K) GRAMMARS

Deterministic context-free languages are of major practical importance. Their algorithms for membership and parsing are based on DPDAs and are therefore efficient, and they encompass a rich class of CFLs that include most programming languages. However, DCFGs are sometimes inconvenient for expressing particular DCFLs. The requirement that all handles are forced is often an obstacle to designing intuitive DCFGs.

Fortunately, a broader class of grammars called the $LR(k)$ grammars gives us the best of both worlds. They are close enough to DCFGs to allow direct conversion into DPDAs. Yet they are expressive enough for many applications.

Algorithms for $LR(k)$ grammars introduce *lookahead*. In a DCFG, all handles are forced. A handle depends only on the symbols in a valid string up through and including the handle, but not on terminal symbols that follow the handle. In an $LR(k)$ grammar, a handle may also depend on symbols that follow the handle, but only on the first k of these. The acronym $LR(k)$ stands for: Left to right input processing, Rightmost derivations (or equivalently, leftmost reductions), and k symbols of lookahead.

To make this precise, let h be a handle of a valid string $v = xhy$. Say that h is **forced by lookahead k** if h is the unique handle of every valid string $xh\hat{y}$ where $\hat{y} \in \Sigma^*$ and where y and $\hat{y}$ agree on their first k symbols. (If either string is shorter than k , the strings must agree up to the length of the shorter one.)

DEFINITION 2.62

An **$LR(k)$ grammar** is a context-free grammar such that the handle of every valid string is forced by lookahead k .

Thus a DCFG is the same as an $LR(0)$ grammar. We can show that for every k we can convert $LR(k)$ grammars to DPDAs. We've already shown that DPDAs are equivalent to $LR(0)$ grammars. Hence $LR(k)$ grammars are equivalent in power for all k and all describe exactly the DCFs. The following example shows that $LR(1)$ grammars are more convenient than DCFGs for specifying certain languages.

To avoid cumbersome notation and technical details, we will show how to convert $LR(k)$ grammars to DPDAs only for the special case where $k = 1$. The conversion in the general case works in essentially the same way.

To begin, we'll present a variant of the *DK*-test, modified for $LR(1)$ grammars. We call it the *DK*-test with lookahead 1, or simply the DK_1 -test. As before, we'll construct an NFA, called K_1 here, and convert it to a DFA DK_1 . Each of K_1 's states has a dotted rule $T \rightarrow u.v$ and now also a terminal symbol a , called the **lookahead symbol**, shown as $\boxed{T \rightarrow u.v \ a}$. This state indicates that K_1 has recently read the string u , which would be a part of a handle uv provided that v follows after u and a follows after v .

The formal construction works much as before. The start state has an ϵ -move to $\boxed{S_1 \rightarrow .u \ a}$ for every rule involving the start variable S_1 and every $a \in \Sigma$. The shift transitions take $\boxed{T \rightarrow u.xv \ a}$ to $\boxed{T \rightarrow ux.v \ a}$ on input x where x is a variable symbol or terminal symbol. The ϵ -transitions take $\boxed{T \rightarrow u.Cv \ a}$ to $\boxed{C \rightarrow .r \ b}$ for each rule $C \rightarrow r$, where b is the first symbol of any string of terminals that can be derived from v . If v derives ϵ , add $b = a$. The accept states are all $\boxed{B \rightarrow u. \ a}$ for completed rules $B \rightarrow u.$ and $a \in \Sigma$.

Let R_1 be a completed rule with lookahead symbol a_1 , and let R_2 be a dotted rule with lookahead symbol a_2 . Say that R_1 and R_2 are **consistent** if

1. R_2 is completed and $a_1 = a_2$, or
2. R_2 is not completed and a_1 immediately follows its dot.

Now we are ready to describe the **DK_1 -test**. Construct the DFA DK_1 . The test stipulates that every accept state must not contain any two consistent dotted rules.

THEOREM 2.63

G passes the DK_1 -test iff G is an LR(1) grammar.

PROOF IDEA Corollary 2.51 still applies to DK_1 because we can ignore the lookahead symbols.

EXAMPLE 2.64

This example shows that the following grammar passes the DK_1 -test. Recall that in Example 2.53 this grammar was shown to fail the DK -test. Hence it is an example of a grammar that is LR(1) but not a DCFG.

$$\begin{aligned} S &\rightarrow E \dashv \\ E &\rightarrow E + T \mid T \\ T &\rightarrow T \times a \mid a \end{aligned}$$

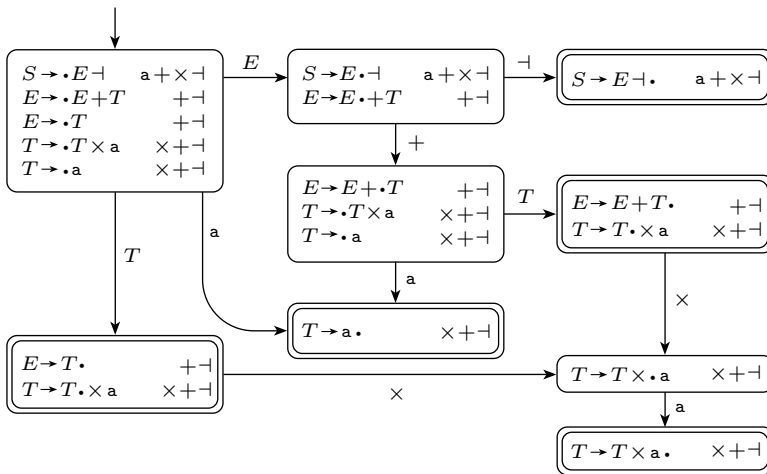


FIGURE 2.65
Passing the DK_1 -test

THEOREM 2.66

An endmarked language is generated by an LR(1) grammar iff it is a DCFL.

We've already shown that every DCFL has an LR(0) grammar, because an LR(0) grammar is the same as a DCFG. That proves the reverse direction of the theorem. What remains is the following lemma, which shows how to convert an LR(1) grammar to a DPDA.

LEMMA 2.67

Every LR(1) grammar has an equivalent DPDA.

PROOF IDEA We construct P_1 , a modified version of the DPDA P that we presented in Lemma 2.67. P_1 reads its input and simulates DK_1 , while using the stack to keep track of the state DK_1 would be in if all reduce steps were applied to this input up to this point. Moreover, P_1 reads 1 symbol ahead and stores this lookahead information in its finite state memory. Whenever DK_1 reaches an accept state, P_1 consults its lookahead to see whether to perform a reduce step, and which step to do if several possibilities appear in this state. Only one option can apply because the grammar is LR(1).

EXERCISES

- 2.1 Recall the CFG G_4 that we gave in Example 2.4. For convenience, let's rename its variables with single letters as follows.

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T \times F \mid F \\ F &\rightarrow (E) \mid a \end{aligned}$$

Give parse trees and derivations for each string.

- | | |
|---|---|
| <p>a. a</p> <p>b. $a+a$</p> | <p>c. $a+a+a$</p> <p>d. $((a))$</p> |
|---|---|
- 2.2
- a. Use the languages $A = \{a^m b^n c^n \mid m, n \geq 0\}$ and $B = \{a^n b^n c^m \mid m, n \geq 0\}$ together with Example 2.36 to show that the class of context-free languages is not closed under intersection.
 - b. Use part (a) and DeMorgan's law (Theorem 0.20) to show that the class of context-free languages is not closed under complementation.

^A2.3 Answer each part for the following context-free grammar G .

$$\begin{aligned} R &\rightarrow XRX \mid S \\ S &\rightarrow aTb \mid bTa \\ T &\rightarrow XTX \mid X \mid \varepsilon \\ X &\rightarrow a \mid b \end{aligned}$$

- a. What are the variables of G ?
 - b. What are the terminals of G ?
 - c. Which is the start variable of G ?
 - d. Give three strings in $L(G)$.
 - e. Give three strings *not* in $L(G)$.
 - f. True or False: $T \Rightarrow aba$.
 - g. True or False: $T \stackrel{*}{\Rightarrow} aba$.
 - h. True or False: $T \Rightarrow T$.
 - i. True or False: $T \stackrel{*}{\Rightarrow} T$.
 - j. True or False: $XXX \stackrel{*}{\Rightarrow} aba$.
 - k. True or False: $X \stackrel{*}{\Rightarrow} aba$.
 - l. True or False: $T \stackrel{*}{\Rightarrow} XX$.
 - m. True or False: $T \stackrel{*}{\Rightarrow} XXX$.
 - n. True or False: $S \stackrel{*}{\Rightarrow} \varepsilon$.
 - o. Give a description in English of $L(G)$.
- 2.4 Give context-free grammars that generate the following languages. In all parts, the alphabet Σ is $\{0,1\}$.
- ^Aa. $\{w \mid w \text{ contains at least three 1s}\}$
 - b. $\{w \mid w \text{ starts and ends with the same symbol}\}$
 - c. $\{w \mid \text{the length of } w \text{ is odd}\}$
 - ^Ad. $\{w \mid \text{the length of } w \text{ is odd and its middle symbol is a 0}\}$
 - e. $\{w \mid w = w^R, \text{ that is, } w \text{ is a palindrome}\}$
 - f. The empty set
- 2.5 Give informal descriptions and state diagrams of pushdown automata for the languages in Exercise 2.4.
- 2.6 Give context-free grammars generating the following languages.
- ^Aa. The set of strings over the alphabet $\{a,b\}$ with more a's than b's
 - b. The complement of the language $\{a^n b^n \mid n \geq 0\}$
 - ^Ac. $\{w\#x \mid w^R \text{ is a substring of } x \text{ for } w, x \in \{0,1\}^*\}$
 - d. $\{x_1\#x_2\#\cdots\#x_k \mid k \geq 1, \text{ each } x_i \in \{a,b\}^*, \text{ and for some } i \text{ and } j, x_i = x_j^R\}$
- ^A2.7 Give informal English descriptions of PDAs for the languages in Exercise 2.6.
- ^A2.8 Show that the string `the girl touches the boy with the flower` has two different leftmost derivations in grammar G_2 on page 103. Describe in English the two different meanings of this sentence.
- 2.9 Give a context-free grammar that generates the language

$$A = \{a^i b^j c^k \mid i = j \text{ or } j = k \text{ where } i, j, k \geq 0\}.$$

Is your grammar ambiguous? Why or why not?

- 2.10 Give an informal description of a pushdown automaton that recognizes the language A in Exercise 2.9.
- 2.11 Convert the CFG G_4 given in Exercise 2.1 to an equivalent PDA, using the procedure given in Theorem 2.20.

- 2.12 Convert the CFG G given in Exercise 2.3 to an equivalent PDA, using the procedure given in Theorem 2.20.
- 2.13 Let $G = (V, \Sigma, R, S)$ be the following grammar. $V = \{S, T, U\}$; $\Sigma = \{0, \#\}$; and R is the set of rules:

$$\begin{aligned} S &\rightarrow TT \mid U \\ T &\rightarrow 0T \mid T0 \mid \# \\ U &\rightarrow 0U00 \mid \# \end{aligned}$$

- a. Describe $L(G)$ in English.
- b. Prove that $L(G)$ is not regular.
- 2.14 Convert the following CFG into an equivalent CFG in Chomsky normal form, using the procedure given in Theorem 2.9.

$$\begin{aligned} A &\rightarrow BAB \mid B \mid \epsilon \\ B &\rightarrow 00 \mid \epsilon \end{aligned}$$

- 2.15 Give a counterexample to show that the following construction fails to prove that the class of context-free languages is closed under star. Let A be a CFL that is generated by the CFG $G = (V, \Sigma, R, S)$. Add the new rule $S \rightarrow SS$ and call the resulting grammar G' . This grammar is supposed to generate A^* .
- 2.16 Show that the class of context-free languages is closed under the regular operations, union, concatenation, and star.
- 2.17 Use the results of Exercise 2.16 to give another proof that every regular language is context free, by showing how to convert a regular expression directly to an equivalent context-free grammar.



PROBLEMS

- ^A2.18 a. Let C be a context-free language and R be a regular language. Prove that the language $C \cap R$ is context free.
- b. Let $A = \{w \mid w \in \{a, b, c\}^* \text{ and } w \text{ contains equal numbers of } a\text{'s, } b\text{'s, and } c\text{'s}\}$. Use part (a) to show that A is not a CFL.
- *2.19 Let CFG G be the following grammar.

$$\begin{aligned} S &\rightarrow aSb \mid bY \mid Ya \\ Y &\rightarrow bY \mid aY \mid \epsilon \end{aligned}$$

Give a simple description of $L(G)$ in English. Use that description to give a CFG for $\overline{L(G)}$, the complement of $L(G)$.

- 2.20 Let $A/B = \{w \mid wx \in A \text{ for some } x \in B\}$. Show that if A is context free and B is regular, then A/B is context free.
- *2.21 Let $\Sigma = \{a, b\}$. Give a CFG generating the language of strings with twice as many a 's as b 's. Prove that your grammar is correct.
- *2.22 Let $C = \{x\#y \mid x, y \in \{0, 1\}^* \text{ and } x \neq y\}$. Show that C is a context-free language.

- *2.23 Let $D = \{xy \mid x, y \in \{0,1\}^* \text{ and } |x| = |y| \text{ but } x \neq y\}$. Show that D is a context-free language.
- *2.24 Let $E = \{a^i b^j \mid i \neq j \text{ and } 2i \neq j\}$. Show that E is a context-free language.
- 2.25 For any language A , let $SUFFIX(A) = \{v \mid uv \in A \text{ for some string } u\}$. Show that the class of context-free languages is closed under the *SUFFIX* operation.
- 2.26 Show that if G is a CFG in Chomsky normal form, then for any string $w \in L(G)$ of length $n \geq 1$, exactly $2n - 1$ steps are required for any derivation of w .
- *2.27 Let $G = (V, \Sigma, R, \langle \text{STMT} \rangle)$ be the following grammar.

$$\begin{aligned} \langle \text{STMT} \rangle &\rightarrow \langle \text{ASSIGN} \rangle \mid \langle \text{IF-THEN} \rangle \mid \langle \text{IF-THEN-ELSE} \rangle \\ \langle \text{IF-THEN} \rangle &\rightarrow \text{if condition then } \langle \text{STMT} \rangle \\ \langle \text{IF-THEN-ELSE} \rangle &\rightarrow \text{if condition then } \langle \text{STMT} \rangle \text{ else } \langle \text{STMT} \rangle \\ \langle \text{ASSIGN} \rangle &\rightarrow \text{a:=1} \end{aligned}$$

$$\Sigma = \{\text{if, condition, then, else, a:=1}\}$$

$$V = \{\langle \text{STMT} \rangle, \langle \text{IF-THEN} \rangle, \langle \text{IF-THEN-ELSE} \rangle, \langle \text{ASSIGN} \rangle\}$$

G is a natural-looking grammar for a fragment of a programming language, but G is ambiguous.

- a. Show that G is ambiguous.
 - b. Give a new unambiguous grammar for the same language.
- *2.28 Give unambiguous CFGs for the following languages.
- a. $\{w \mid \text{in every prefix of } w \text{ the number of a's is at least the number of b's}\}$
 - b. $\{w \mid \text{the number of a's and the number of b's in } w \text{ are equal}\}$
 - c. $\{w \mid \text{the number of a's is at least the number of b's in } w\}$
- *2.29 Show that the language A in Exercise 2.9 is inherently ambiguous.
- 2.30 Use the pumping lemma to show that the following languages are not context free.
- a. $\{0^n 1^n 0^n 1^n \mid n \geq 0\}$
 - ^Ab. $\{0^n \# 0^{2n} \# 0^{3n} \mid n \geq 0\}$
 - ^Ac. $\{w \# t \mid w \text{ is a substring of } t, \text{ where } w, t \in \{a, b\}^*\}$
 - d. $\{t_1 \# t_2 \# \cdots \# t_k \mid k \geq 2, \text{ each } t_i \in \{a, b\}^*, \text{ and } t_i = t_j \text{ for some } i \neq j\}$
- 2.31 Let B be the language of all palindromes over $\{0,1\}$ containing equal numbers of 0s and 1s. Show that B is not context free.
- 2.32 Let $\Sigma = \{1, 2, 3, 4\}$ and $C = \{w \in \Sigma^* \mid \text{in } w, \text{ the number of 1s equals the number of 2s, and the number of 3s equals the number of 4s}\}$. Show that C is not context free.
- *2.33 Show that $F = \{a^i b^j \mid i = kj \text{ for some positive integer } k\}$ is not context free.
- 2.34 Consider the language $B = L(G)$, where G is the grammar given in Exercise 2.13. The pumping lemma for context-free languages, Theorem 2.34, states the existence of a pumping length p for B . What is the minimum value of p that works in the pumping lemma? Justify your answer.
- 2.35 Let G be a CFG in Chomsky normal form that contains b variables. Show that if G generates some string with a derivation having at least 2^b steps, $L(G)$ is infinite.

2.36 Give an example of a language that is not context free but that acts like a CFL in the pumping lemma. Prove that your example works. (See the analogous example for regular languages in Problem 1.54.)

***2.37** Prove the following stronger form of the pumping lemma, wherein *both* pieces *v* and *y* must be nonempty when the string *s* is broken up.

If *A* is a context-free language, then there is a number *k* where, if *s* is any string in *A* of length at least *k*, then *s* may be divided into five pieces, $s = uvxyz$, satisfying the conditions:

- a. for each $i \geq 0$, $uv^i xy^i z \in A$,
- b. $v \neq \epsilon$ and $y \neq \epsilon$, and
- c. $|vxy| \leq k$.

^A2.38 Refer to Problem 1.41 for the definition of the perfect shuffle operation. Show that the class of context-free languages is not closed under perfect shuffle.

2.39 Refer to Problem 1.42 for the definition of the shuffle operation. Show that the class of context-free languages is not closed under shuffle.

***2.40** Say that a language is *prefix-closed* if all prefixes of every string in the language are also in the language. Let *C* be an infinite, prefix-closed, context-free language. Show that *C* contains an infinite regular subset.

***2.41** Read the definitions of $NOPREFIX(A)$ and $NOEXTEND(A)$ in Problem 1.40.

- a. Show that the class of CFLs is not closed under $NOPREFIX$.
- b. Show that the class of CFLs is not closed under $NOEXTEND$.

***2.42** Let $Y = \{w \mid w = t_1 \# t_2 \# \dots \# t_k \text{ for } k \geq 0, \text{ each } t_i \in 1^*, \text{ and } t_i \neq t_j \text{ whenever } i \neq j\}$. Here $\Sigma = \{1, \#\}$. Prove that *Y* is not context free.

2.43 For strings *w* and *t*, write $w \doteq t$ if the symbols of *w* are a permutation of the symbols of *t*. In other words, $w \doteq t$ if *t* and *w* have the same symbols in the same quantities, but possibly in a different order.

For any string *w*, define $SCRAMBLE(w) = \{t \mid t \doteq w\}$. For any language *A*, let $SCRAMBLE(A) = \{t \mid t \in SCRAMBLE(w) \text{ for some } w \in A\}$.

- a. Show that if $\Sigma = \{0,1\}$, then the $SCRAMBLE$ of a regular language is context free.
- b. What happens in part (a) if Σ contains three or more symbols? Prove your answer.

2.44 If *A* and *B* are languages, define $A \diamond B = \{xy \mid x \in A \text{ and } y \in B \text{ and } |x| = |y|\}$. Show that if *A* and *B* are regular languages, then $A \diamond B$ is a CFL.

***2.45** Let $A = \{wtw^{\mathcal{R}} \mid w, t \in \{0,1\}^* \text{ and } |w| = |t|\}$. Prove that *A* is not a CFL.

2.46 Consider the following CFG *G*:

$$\begin{aligned} S &\rightarrow SS \mid T \\ T &\rightarrow aTb \mid ab \end{aligned}$$

Describe $L(G)$ and show that *G* is ambiguous. Give an unambiguous grammar *H* where $L(H) = L(G)$ and sketch a proof that *H* is unambiguous.

- 2.47 Let $\Sigma = \{0,1\}$ and let B be the collection of strings that contain at least one 1 in their second half. In other words, $B = \{uv \mid u \in \Sigma^*, v \in \Sigma^*1\Sigma^* \text{ and } |u| \geq |v|\}$.
- Give a PDA that recognizes B .
 - Give a CFG that generates B .
- 2.48 Let $\Sigma = \{0,1\}$. Let C_1 be the language of all strings that contain a 1 in their middle third. Let C_2 be the language of all strings that contain two 1s in their middle third. So $C_1 = \{xyz \mid x, z \in \Sigma^* \text{ and } y \in \Sigma^*1\Sigma^*, \text{ where } |x| = |z| \geq |y|\}$ and $C_2 = \{xyz \mid x, z \in \Sigma^* \text{ and } y \in \Sigma^*1\Sigma^*1\Sigma^*, \text{ where } |x| = |z| \geq |y|\}$.
- Show that C_1 is a CFL.
 - Show that C_2 is not a CFL.
- *2.49 We defined the rotational closure of language A to be $RC(A) = \{yx \mid xy \in A\}$. Show that the class of CFLs is closed under rotational closure.
- *2.50 We defined the CUT of language A to be $CUT(A) = \{yxyz \mid xyz \in A\}$. Show that the class of CFLs is not closed under CUT .
- 2.51 Show that every DCFG is an unambiguous CFG.
- ^A2.52 Show that every DCFG generates a prefix-free language.
- *2.53 Show that the class of DCFLs is not closed under the following operations:
- Union
 - Intersection
 - Concatenation
 - Star
 - Reversal

2.54 Let G be the following grammar:

$$\begin{aligned} S &\rightarrow T\text{--} \\ T &\rightarrow T\mathbf{a}T\mathbf{b} \mid T\mathbf{b}T\mathbf{a} \mid \varepsilon \end{aligned}$$

- Show that $L(G) = \{w\text{--} \mid w \text{ contains equal numbers of a's and b's}\}$. Use a proof by induction on the length of w .
 - Use the DK -test to show that G is a DCFG.
 - Describe a DPDA that recognizes $L(G)$.
- 2.55 Let G_1 be the following grammar that we introduced in Example 2.45. Use the DK -test to show that G_1 is not a DCFG.

$$\begin{aligned} R &\rightarrow S \mid T \\ S &\rightarrow \mathbf{a}S\mathbf{b} \mid \mathbf{ab} \\ T &\rightarrow \mathbf{a}T\mathbf{bb} \mid \mathbf{abb} \end{aligned}$$

- A decorative element consisting of a single row of approximately 20 small, light gray squares.

2.3 (a) R, X, S, T ; (b) a, b ; (c) R ; (d) Three strings in $L(G)$ are ab, ba , and aab ; (e) Three strings not in $L(G)$ are a, b , and ε ; (f) False; (g) True; (h) False; (i) True; (j) True; (k) False; (l) True; (m) True; (n) False; (o) $L(G)$ consists of all strings over a and b that are not palindromes.

2.6 (a) $S \rightarrow TaT$
 $T \rightarrow TT \mid aTb \mid bTa \mid a \mid \epsilon$
 T generates all strings with at least as many a's as b's, and S forces an extra a.

(c) $S \rightarrow TX$
 $T \rightarrow 0T0 \mid 1T1 \mid \#X$
 $X \rightarrow 0X \mid 1X \mid \epsilon$

- Copyright 2012 Cengage Learning. All Rights Reserved. May not be copied, scanned, or duplicated, in whole or in part. Due to electronic rights, some third party content may be suppressed from the eBook and/or eChapter(s). Editorial review has deemed that any suppressed content does not materially affect the overall learning experience. Cengage Learning reserves the right to remove additional content at any time if subsequent rights restrictions require it.

2.8 Here is one derivation:

$\langle \text{SENTENCE} \rangle \rightarrow \langle \text{NOUN-PHRASE} \rangle \langle \text{VERB-PHRASE} \rangle \rightarrow$
 $\langle \text{CMPLX-NOUN} \rangle \langle \text{VERB-PHRASE} \rangle \rightarrow$
 $\langle \text{ARTICLE} \rangle \langle \text{NOUN} \rangle \langle \text{VERB-PHRASE} \rangle \rightarrow$
 The $\langle \text{NOUN} \rangle \langle \text{VERB-PHRASE} \rangle \rightarrow$
 The girl $\langle \text{VERB-PHRASE} \rangle \rightarrow$
 The girl $\langle \text{CMPLX-VERB} \rangle \langle \text{PREP-PHRASE} \rangle \rightarrow$
 The girl $\langle \text{VERB} \rangle \langle \text{NOUN-PHRASE} \rangle \langle \text{PREP-PHRASE} \rangle \rightarrow$
 The girl touches $\langle \text{NOUN-PHRASE} \rangle \langle \text{PREP-PHRASE} \rangle \rightarrow$
 The girl touches $\langle \text{CMPLX-NOUN} \rangle \langle \text{PREP-PHRASE} \rangle \rightarrow$
 The girl touches $\langle \text{ARTICLE} \rangle \langle \text{NOUN} \rangle \langle \text{PREP-PHRASE} \rangle \rightarrow$
 The girl touches the $\langle \text{NOUN} \rangle \langle \text{PREP-PHRASE} \rangle \rightarrow$
 The girl touches the boy $\langle \text{PREP-PHRASE} \rangle \rightarrow$
 The girl touches the boy $\langle \text{PREP} \rangle \langle \text{CMPLX-NOUN} \rangle \rightarrow$
 The girl touches the boy with $\langle \text{CMPLX-NOUN} \rangle \rightarrow$
 The girl touches the boy with $\langle \text{ARTICLE} \rangle \langle \text{NOUN} \rangle \rightarrow$
 The girl touches the boy with the $\langle \text{NOUN} \rangle \rightarrow$
 The girl touches the boy with the flower

Here is another leftmost derivation:

$\langle \text{SENTENCE} \rangle \rightarrow \langle \text{NOUN-PHRASE} \rangle \langle \text{VERB-PHRASE} \rangle \rightarrow$
 $\langle \text{CMPLX-NOUN} \rangle \langle \text{VERB-PHRASE} \rangle \rightarrow$
 $\langle \text{ARTICLE} \rangle \langle \text{NOUN} \rangle \langle \text{VERB-PHRASE} \rangle \rightarrow$
 The $\langle \text{NOUN} \rangle \langle \text{VERB-PHRASE} \rangle \rightarrow$
 The girl $\langle \text{VERB-PHRASE} \rangle \rightarrow$
 The girl $\langle \text{CMPLX-VERB} \rangle \rightarrow$
 The girl $\langle \text{VERB} \rangle \langle \text{NOUN-PHRASE} \rangle \rightarrow$
 The girl touches $\langle \text{NOUN-PHRASE} \rangle \rightarrow$
 The girl touches $\langle \text{CMPLX-NOUN} \rangle \langle \text{PREP-PHRASE} \rangle \rightarrow$
 The girl touches $\langle \text{ARTICLE} \rangle \langle \text{NOUN} \rangle \langle \text{PREP-PHRASE} \rangle \rightarrow$
 The girl touches the $\langle \text{NOUN} \rangle \langle \text{PREP-PHRASE} \rangle \rightarrow$
 The girl touches the boy $\langle \text{PREP-PHRASE} \rangle \rightarrow$
 The girl touches the boy $\langle \text{PREP} \rangle \langle \text{CMPLX-NOUN} \rangle \rightarrow$
 The girl touches the boy with $\langle \text{CMPLX-NOUN} \rangle \rightarrow$
 The girl touches the boy with $\langle \text{ARTICLE} \rangle \langle \text{NOUN} \rangle \rightarrow$
 The girl touches the boy with the $\langle \text{NOUN} \rangle \rightarrow$
 The girl touches the boy with the flower

Each of these derivations corresponds to a different English meaning. In the first derivation, the sentence means that the girl used the flower to touch the boy. In the second derivation, the boy is holding the flower when the girl touches her.

2.18 (a) Let C be a context-free language and R be a regular language. Let P be the PDA that recognizes C , and D be the DFA that recognizes R . If Q is the set of states of P and Q' is the set of states of D , we construct a PDA P' that recognizes $C \cap R$ with the set of states $Q \times Q'$. P' will do what P does and also keep track of the states of D . It accepts a string w if and only if it stops at a state $q \in F_P \times F_D$, where F_P is the set of accept states of P and F_D is the set of accept states of D . Since $C \cap R$ is recognized by P' , it is context free.

(b) Let R be the regular language $a^*b^*c^*$. If A were a CFL then $A \cap R$ would be a CFL by part (a). However, $A \cap R = \{a^n b^n c^n \mid n \geq 0\}$, and Example 2.36 proves that $A \cap R$ is not context free. Thus A is not a CFL.

- 2.30 (b)** Let $B = \{0^n \# 0^{2n} \# 0^{3n} \mid n \geq 0\}$. Let p be the pumping length given by the pumping lemma. Let $s = 0^p \# 0^{2p} \# 0^{3p}$. We show that $s = uvxyz$ cannot be pumped.

Neither v nor y can contain $\#$, otherwise uv^2xy^2z contains more than two $\#$ s. Therefore, if we divide s into three segments by $\#$'s: 0^p , 0^{2p} , and 0^{3p} , at least one of the segments is not contained within either v or y . Hence uv^2xy^2z is not in B because the 1 : 2 : 3 length ratio of the segments is not maintained.

(c) Let $C = \{w\#t \mid w \text{ is a substring of } t, \text{ where } w, t \in \{a, b\}^*\}$. Let p be the pumping length given by the pumping lemma. Let $s = a^p b^p \# a^p b^p$. We show that the string $s = uvxyz$ cannot be pumped.

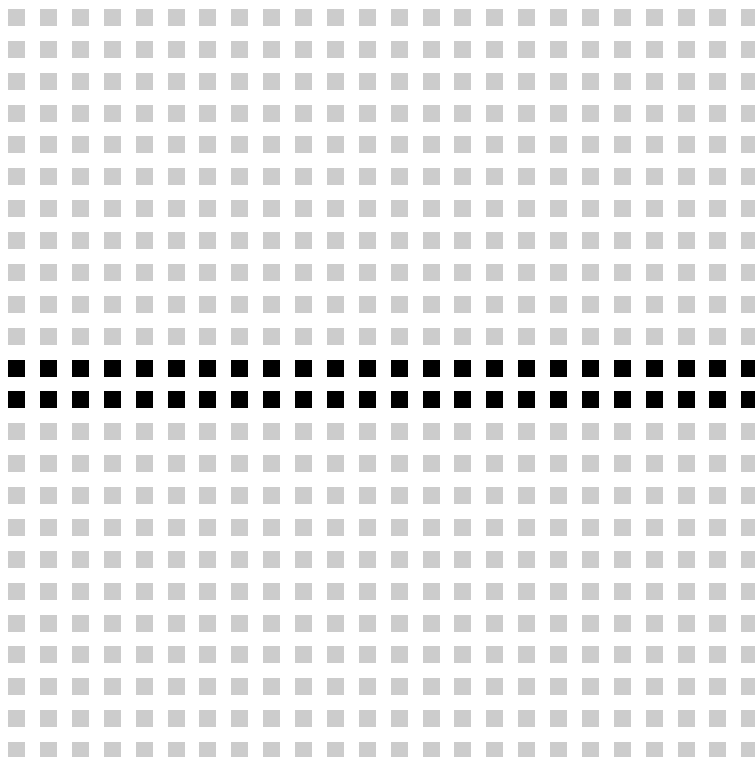
Neither v nor y can contain $\#$, otherwise uv^0xy^0z does not contain $\#$ and therefore is not in C . If both v and y occur on the left-hand side of the $\#$, the string uv^2xy^2z cannot be in C because it is longer on the left-hand side of the $\#$. Similarly, if both strings occur on the right-hand side of the $\#$, the string uv^0xy^0z cannot be in C because it is again longer on the left-hand side of the $\#$. If one of v and y is empty (both cannot be empty), treat them as if both occurred on the same side of the $\#$ as above.

The only remaining case is where both v and y are nonempty and straddle the $\#$. But then v consists of b 's and y consists of a 's because of the third pumping lemma condition $|vxy| \leq p$. Hence, uv^2xy^2z contains more b 's on the left-hand side of the $\#$, so it cannot be a member of C .

- 2.38** Let A be the language $\{0^k 1^k \mid k \geq 0\}$ and let B be the language $\{a^k b^{3k} \mid k \geq 0\}$. The perfect shuffle of A and B is the language $C = \{(0a)^k (0b)^k (1b)^{2k} \mid k \geq 0\}$. Languages A and B are easily seen to be CFLs, but C is not a CFL, as follows. If C were a CFL, let p be the pumping length given by the pumping lemma, and let s be the string $(0a)^p (0b)^p (1b)^{2p}$. Because s is longer than p and $s \in C$, we can divide $s = uvxyz$ satisfying the pumping lemma's three conditions. Strings in C are exactly one-fourth 1s and one-eighth a 's. In order for uv^2xy^2z to have that property, the string vxy must contain both 1s and a 's. But that is impossible, because the 1s and a 's are separated by $2p$ symbols in s yet the third condition says that $|vxy| \leq p$. Hence C is not context free.

- 2.52** We use a proof by contradiction. Assume that w and wz are two unequal strings in $L(G)$, where G is a DCFG. Both are valid strings so both have handles, and these handles must agree because we can write $w = xhy$ and $wz = xhyz = xh\hat{y}$ where h is the handle of w . Hence, the first reduce steps of w and wz produce valid strings u and uz , respectively. We can continue this process until we obtain S_1 and S_1z where S_1 is the start variable. However, S_1 does not appear on the right-hand side of any rule so we cannot reduce S_1z . That gives a contradiction.

PART TWO



C O M P U T A B I L I T Y T H E O R Y